



OPEN A generative AI cybersecurity risks mitigation model for code generation: using ANN-ISM hybrid approach

Hussein A. Al-Hashimi

The increasing reliance on automatic code generation integrated with Generative AI technology has raised new challenges for cybersecurity defense against code injection, insecure code templates, and adversarial manipulation of an AI model. These risks make developing advanced frameworks imperative to ensure secure, reliable, and privacy-preserving code generation processes. The paper presents a novel Hybrid Artificial Neural Network (ANN)-Interpretive Structural Modeling (ISM) Framework to alleviate the cybersecurity risks associated with the automatic code generation using Generative AI. The proposed framework integrates the predictive capability of ANN and structured analysis of ISM for the identification, evaluation, and treatment of common vulnerabilities and risks in automatic code generation. We first conduct a multivocal literature review (MLR) to identify cybersecurity risks and generative AI practices for addressing these risks in automatic code generation. Then we conduct a questionnaire survey to identify and validate the identified risks and practices. An expert panel review was then assigned for the process of ANN-ISM. The ANN model can predict potential security risks by learning from historical data and code generation patterns. ISM is used to (1) structure and visualize (2) relations between identified risks and mitigation approaches and (3) offer a combined, multi-layered risk management methodology. We then perform an in-depth examination of the framework with a case study of an AI-based code generation company. We further determine its practicality and usefulness in real-world settings. The case study results show that the framework efficiently handles the primary cybersecurity challenges, such as injection attacks, code quality, backdoors, and lack of input validation. The analysis characterizes the maturity of several mitigation practices and areas for improvement for security integration with automatic code generation functionality. Advanced risk mitigation is enabled in the framework across multiple process areas, where techniques such as static code analysis, automated penetration testing, and adversarial training hold much promise. The Hybrid ANN-ISM Mechanism is a stable and flexible solution for cybersecurity risk reduction in automatic code generation environments. The coupling of ANN and ISM, in terms of predictive analysis and structured risk management, respectively, contributes effectively towards the security of AI-based code generation tools. More research is required to improve the scalability, privacy preserving, and dynamic integration of the framework with cybersecurity threat intelligence.

Keywords Cybersecurity risks, Automatic code generation, Generative AI practices, Multivocal literature review, Empirical study, ANN-ISM, Case study

The increasing adoption of Generative AI for automatic code generation has revolutionized software development. Code generation synthesizes code from a user-given description¹. Most modern Integrated Development Environments (IDEs) available today are also equipped with facilities that support automatic code generation according to end-user inputs in the form of field names or function headers¹. These methods can be based on any established, common pattern, such as setters and getters, refactorizations, or inheritance. However, such methods may be inefficient. For example, the generation of setters and getters can be time-consuming, and after refactoring, they have to be tested intensively to work correctly. Therefore, although these approaches are widely adopted, in certain scenarios they may prove challenging in terms of both development time and code

College of Computer and Information Sciences, King Saud University, Riyadh, Saudi Arabia. email: halhashimi@ksu.edu.sa

quality. Through the use of technology including machine learning especially Generative AI, developers are able to automatically write large parts of code from overview details speeding up the process of software development and making it less exhaustive². Automatic code generation systems regard being based on machine learning models, especially deep learning methods and technology, which have led to an accelerated development cycle, minimized human error, and improved productivity¹.

However, with the growing complexity and ubiquity of automatic code generation tools comes an increasing security risk. While effective, these tools add new security holes that attackers can exploit when poorly managed³. Automated code generation speeds up the development process at the cost of various cybersecurity issues. These problems are frequently exacerbated by a lack of human oversight, particularly when considering Generative AI models that may produce code based on observed patterns in vast datasets, some of which may be inadvertently insecure or flawed⁴. However, with the increased usage of such systems, the threat of different cybersecurity vulnerabilities is also on the rise⁵. Automated code generators, whilst effective, are liable to have security holes that can be exploited by attackers, with potentially disastrous consequences for security breaches, data loss, or software misbehavior⁶.

Cybersecurity threats in automatic code generation are diverse, including injection attacks, insecure code templates, backdoors, and adversarial perturbation of machine learning models, among others^{7,8}. Those risks are compounded by the complexity and incomprehensibility of AI models, which often work as ‘black boxes’ that are difficult to understand in full-and also hard to understand the implications of what would happen if the code were insecure⁹. Furthermore, the speed at which code is created can, in some cases, exceed the adoption rate of traditional security and maintainability practices, leading to many software systems that are wide open to attack¹⁰.

As organizations increasingly employ AI-based tools to generate code automatically, there is a critical necessity to develop holistic frameworks for the detection and mitigation of cyber risks in these systems. The challenges of automatic code generation and generative AI are largely ignored by traditional cybersecurity solutions, which are designed to address challenges in conventional software development paradigms.

Research problem

Given the above concerns, this paper proposes the Hybrid Artificial Neural Network (ANN)—Interpretive Structural Modeling (ISM) Framework, designed to mitigate cybersecurity risks in an Automatic code generation environment. The hybrid model employs the predictive strength of ANN and the relationship analysis by ISM to identify, evaluate, and control the risks holistically and proactively. ANNs are known for their capability to discover regularities and potential security vulnerabilities by analyzing large amounts of historical data, including known vulnerabilities, attack surfaces, and known secure coding patterns. In addition to ANN, ISM is a structured, hierarchical model that distinguishable risk and shows their interconnections that giving a precise and understandable model of cybersecurity vulnerabilities and riddance policies. Integrating two such methods—ANN for prediction and ISM to structure the analysis—represents a significant advance in improving the security of automated code generators.

Significance of the study

The innovative Hybrid ANN-ISM Framework has several implications. First, we present a new way to combine AI-based risk prediction with a structured risk management model, providing a more systematic and understandable solution for the automatic code generation. Second, it focuses on important cybersecurity vulnerabilities in generative AI, where, to date, relatively minimal research has been conducted on its unique security considerations. The framework is also general and can be applied to different applications and industries, guaranteeing its suitability for protecting several AI-based code generators. This paper aims to bridge this gap by proposing a framework that capitalizes upon the strengths of ANN and ISM to develop a hybrid approach for mitigating cybersecurity risks in automatic code generation. This project aims to bolster the security of AI developer tools, root out their vulnerabilities, and make it safer to deploy AI-powered systems in real-world environments.

Objectives of the study

The primary objectives of this paper are:

- To propose a novel Hybrid ANN-ISM Framework that combines the strengths of ANN and ISM to mitigate cybersecurity risks associated with automatic code generation.
- To evaluate the effectiveness of the proposed framework through a case study, assessing its ability to address common cybersecurity risks such as injection attacks, insecure code templates, backdoors, and insufficient input validation.
- To provide an assessment of the proposed framework's applicability in real-world scenarios, we analyze the maturity levels of generative AI practices within automatic code generation tools and identify areas for improvement.
- To enrich the literature in cybersecurity of AI-based software development by showing the effectiveness of a combined predictive and structural model to alleviate security risks.

Structure of the paper

The structure of the paper is organized as follows:

- Section “[Related work](#)” presents a review of the related work in the field of cybersecurity in automatic code generation, focusing on existing methods, frameworks, and the role of generative AI in software security.

- Section “[Research methodology](#)” provides a detailed explanation of the research methodology and components of the Hybrid ANN-ISM Framework.
- Section “[Results and analysis](#)” discusses the results detailing the hierarchical structure of cybersecurity risks and generative AI mitigation practices.
- Section “[Framework evaluation](#)” presents the evaluation of the Hybrid ANN-ISM Framework for mitigating cybersecurity risks in automatic code generation utilizing generative AI practices.
- Section “[Implications of the study](#)” presents the implications of the study.
- Section “[Limitations of the study](#)” presents the study’s limitations.
- Section “[Conclusion and future research direction](#)” presents the conclusion and direction for future research.

By considering cybersecurity issues within automatic code generation from Generative AI, our work paves the way for future research and development of security-aware development tools within AI, and such a result should further promote the safe practice of AIs in the software industry.

Related work

As software development increasingly becomes automated, the need for addressing cybersecurity risks has gained significant attention. Automatic code generation tools powered by Generative AI and other machine learning techniques are designed to streamline the development process. These tools have several benefits: speed, efficiency, and consistency. Nevertheless, these tools pose new security threats, so there is an increasing number of studies on discovering and mitigating the threats. This section contains an overview of the methods and frameworks on the state of the art and the role of Generative AI to foster software security in automatic code generation.

Before the advances of AI and machine learning-based approaches, code generation was typically based on pre-existing templates, libraries, and pattern-based approaches. Even if these tools fell short in many aspects, they usually included naive protection mechanisms like input validation and crude error handling. Static code analysis tools such as SonarQube and Checkmarx were created to automate the process of evaluating source code for potential security weaknesses, helping developers identify issues within produced code (e.g., buffer overflow, SQL injection, and cross-site scripting (XSS) vulnerabilities)^{11,12}. These tools provide static security vulnerability analysis by analyzing the codebase to discover common vulnerabilities without determining their execution; however, not all vulnerabilities raised by the creation of code automatic tools, even if they have not detected dynamic flaws. In addition, manual code reviews and penetration testing have been used to discover vulnerabilities in the generated code⁹. While these approaches are effective in pinpointing security vulnerabilities, they are time-consuming and not practical for large-scale projects, particularly for those relying on generated code, as procedure-based testing quickly becomes overwhelmed by the sheer amount of code⁹.

As the limitations of traditional approaches became obvious, new automatic methods were developed. Work has concentrated on seamlessly incorporating secure coding practices and automatic vulnerability checking within the code generation process. For instance, secure coding templates can be embedded in the automatic generation framework. Therefore, with templates, we can trust that the code will be written based on best security practices, like sanitizing input and handling sensitive data. Products like Secure Code Warrior offer manageable code snippets with this fixer that automatically replaces insecure structures^{13–15}. Automated penetration testing tools have also been improved to match with generated source code. Solution providers, Veracode and Snyk, for example, have advanced their toolsets to also automatically scan codebases for bugs generated from auto-generated code and third-party dependencies^{16–20}. In typical CI/CD flows, these scanners are integrated into the flow itself, guaranteeing that security checks are executed at each stage of development.

The arrival of Generative AI has changed the game in automatic code generation with new abilities not just to generate, but also to protect the code. Generative AI systems, in particular those based on deep learning, such as transforming models (e.g., OpenAI’s GPT and Google’s BERT), have proven to be highly efficient at generating code that performs a function based on high-level user inputs^{21,22}. On the one hand, it has made the software development process much faster; on the other hand, the utility of generative models includes certain new security concerns to be aware of^{4,23}. One of the significant issues with code generated by AI is that it is inherently opaque. Black-box models like deep neural networks and large language models (LLMs) of the code in code generation risk generating exploitative output without recognizing how individual vulnerabilities or design flaws are being injected into the code²⁴. This lack of interpretability makes it challenging for developers to consider if the AI-created code follows the secure coding principles or contains concealed vulnerabilities, including insecure API invocations and vulnerable data manipulation logic, among others²⁵.

New research has started tackling this issue using AI models for security risk detection. For instance, code-analysis tools based on AI have been constructed to audit generated code for security vulnerabilities in real time¹⁰. In such a related work, code generation made by AI models can be identified by using large code bases to learn patterns for flaws in code, and for that, several models such as CodeBERT and GNNs have been trained²⁶. These solutions enable AI-generated code to be automatically screened and flagged for possible issues before the applications are deployed, helping to minimize the chances of security gaps. Furthermore, Adversarial Machine Learning has also been investigated for the security of AI-supported code generation²⁷. Generative models, like any machine learning system, are exposed to adversarial attacks. Adversarial attacks that involve adding small perturbations in the training data can modify the generated code effectively to inject subtle vulnerabilities that could bypass conventional security checks²⁸. To combat this, efforts are now being made to train generative models and adversarial defenses, improving their resilience against such attacks. Adversarial training addresses the generation of secure code and the discovery of attacking code at the generation phase by training the model with adversarial inputs²⁹.

Hybrid approaches that leverage traditional security technologies with the power of Generative AI have been quite popular in recent years^{10,21,30,31}. For instance Hybrid ANN-ISM Model (proposed in this paper) combines the Artificial Neural Network (ANN) to predict the security risks and Interpretive Structural Modeling (ISM) for undergoing the structured analysis and risk analysis, and mitigation. Integrating AI-based prediction models and systematic risk assessment, the hybrid approach can provide a powerful alternative to code security generation. Other hybrid approaches have been proposed instead of combining static and dynamic analysis²³. For instance, using an AI solution to static code analysis with a dynamic application security testing (DAST) tool means that the code generated is subjected to multi-level scrutiny both pre-deployment and real-time³². This layered combination provides the benefit that vulnerabilities overlooked in one layer are redundant with another^{33,34}.

The continuous advances of AI-aided tools and frameworks for code generation security have outlined several prospective research directions. One promising approach is to leverage reinforcement learning to iteratively enhance the security of generated code on the fly by dynamically adjusting security protection at runtime according to the real-world deployment feedback. In addition, federated learning would be a means to create decentralized AI models, which can enhance defensive security and privacy further without having access to sensitive data in code generation. The increasing interest in Explainable AI (XAI) in the code generation tools domain is significant. Research in the area is moving towards making these AI-engineered code generation tools more interpretable, so developers can understand why specific lines of code were introduced and which security decisions were being made. This will allow more trustworthy use of these tools in production situations where security and transparency matter.

Although Generative AI enables this new kind of automation, it will also add significant cybersecurity risks that must be addressed. Traditional security techniques have been grafted onto the impact of “code that writes code.” Still, emerging AI-based systems are so complex and opaque that entirely new security paradigms are needed. The effect, generative role of AI in software security is both transformative and daunting, and future research in hybrid solutions, adversarial resiliency, and explainability will be crucial to ensure that these technologies can be used safely and securely in practice.

While advanced AI-based models such as CodeBERT and Codex exhibit remarkable capabilities in code prediction and generation, they provide limited transparency and structural understanding of how different cybersecurity risks interact. In contrast, conventional hybrid methods offer valuable interpretive and relational insights through causal or dependency mapping, yet they typically lack the ability to quantitatively evaluate or forecast the magnitude of such risks. This disparity highlights the necessity for a framework that integrates both predictive analytics and structural reasoning. The proposed ANN-ISM hybrid model fulfills this need by combining the learning and prediction strengths of ANN with the hierarchical analysis power of ISM, enabling it to both quantify cybersecurity risks and elucidate their interconnections. Consequently, this approach provides a more comprehensive and interpretable solution for managing cybersecurity threats within Generative AI-driven code generation environments.

Research methodology

In this study, we follow a comprehensive six-phase approach (see Fig. 1) to verify and validate our proposed Hybrid ANN-ISM Framework to reduce the cybersecurity risks in automated code generation. The first phase will comprise of multivocal literature review (MLR) bringing in perspectives from various sources of knowledge, building a strong base for the study. Phase 2 is a field experiment (online questionnaire survey), in which we intend to collect the opinions from practitioners to understand some problems and points of view about the matter. The third phase is an expert panel review to optimize the draft framework through their collective professional wisdom. In the fourth stage, a model is proposed for predicting cybersecurity risks using an Artificial Neural Network (ANN). The fifth stage uses ISM for deeper analysis and structuring the relationships among risk factors. In the end, in the sixth phase, a case study is applied to assess if the proposed approach is achievable and efficient in the real situation. The systematic process guarantees a comprehensive and consistent investigation of the framework capabilities to mitigate cybersecurity risk in automatic code generation.

Phase 1: multivocal literature review (MLR)

Multivocal literature review (MLR) is a comprehensive and systematic literature review from more than one perspective, voice, and source^{35–37}. It represents a spectrum of perspectives, approaches, and results across a field. For this paper, an MLR would entail accessing information from various sources, including peer-reviewed papers, conference papers, industry reports, white papers, and expert opinions. The MLR would focus on cybersecurity risks and Generative AI practices related to automated code generation.

Here are the specific steps of this paper to perform an MLR^{35,38}:

Defining the research questions and scope

- **Establish key research questions:** The first step in MLR is to define the principal questions of the study. Here, the main research questions are:
 - What are the primary cybersecurity risks associated with automatic code generation?
 - What best Generative AI practices and strategies we should adopt to mitigate these risks?
- **Determine the scope:** This will involve deciding on the boundaries of the review, by defining the special generative AI technologies (for example, input validation and sanitization, GANs, etc.) and the scope of the particular cybersecurity risks (possibly, injection attacks, code quality and logic errors, backdoors and malicious code).

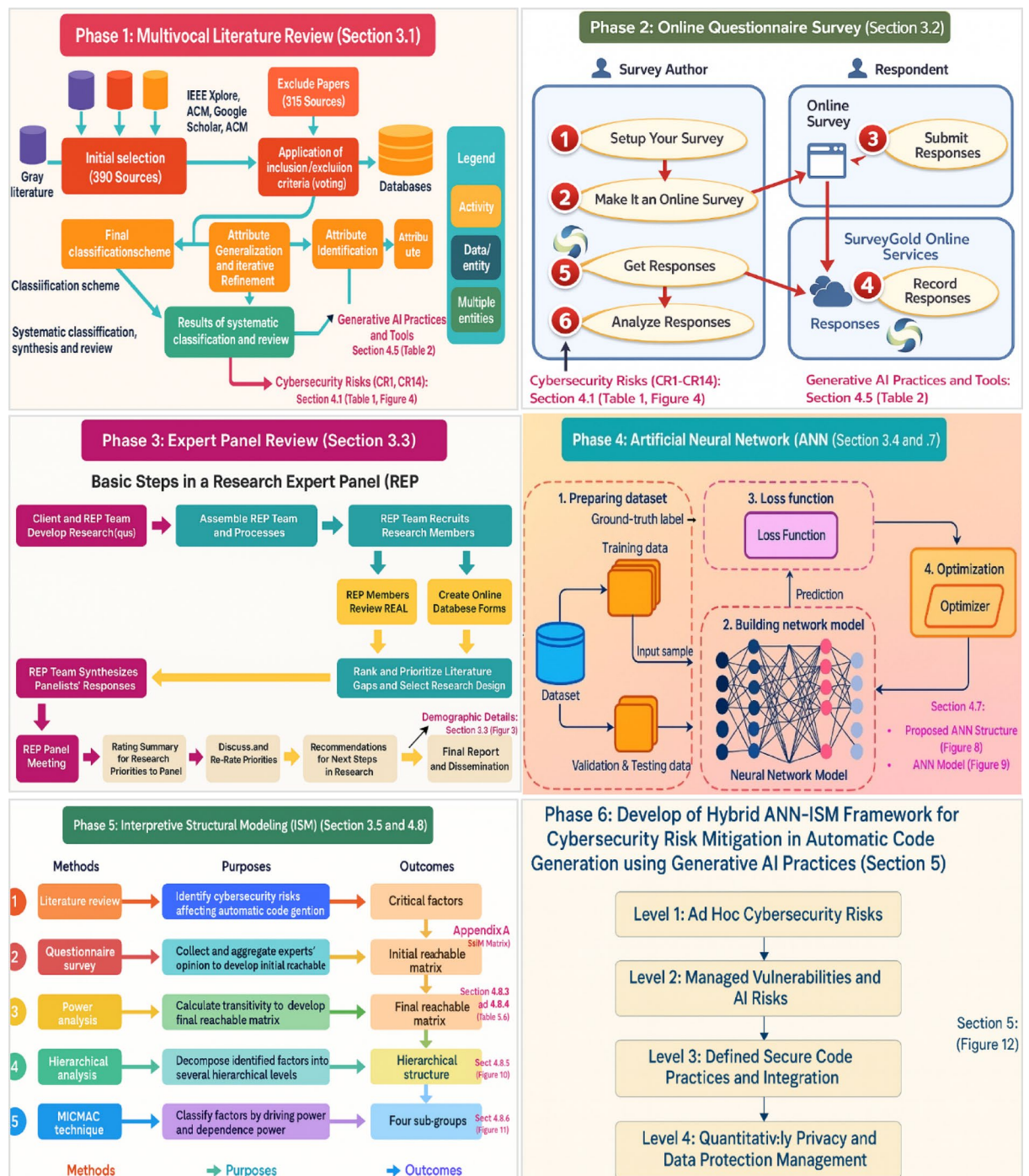


Fig. 1. Research flow framework.

Searching for sources

Find sources: We look for a diverse range of sources:

- Academic references: Papers on cybersecurity, automatic code generation, AI, and Generative Models from high-impact journals and conferences, such as:
 - IEEE Transactions on Cybersecurity*
 - Journal of Experimental and Theoretical Artificial Intelligence (JETAI)*
 - ACM Computing Surveys*
 - International Journal of Information Security*
 - Security and Privacy: (Wiley).*

- Industry reports: Announcements from cybersecurity firms, technology companies, and research institutions. Research papers, reports, and white papers from cybersecurity companies and think-tanks, and organizations like:
 - *Gartner*
 - *McKinsey and Company*
 - *OWASP (Open Web Application Security Project)*
 - *ISACA (Information Systems Audit; Control Association)*
 - *National Institute of Standards and Technology (NIST)*
- Government and regulatory source: Documents from government departments or standards companies, such as:
 - *EU GDPR Reports*
 - *U.S. Cybersecurity and Infrastructure Security Agency (CISA) Advisories*
- Employ databases: Widely used academic databases such as:
 - *Google Scholar, IEEE Xplore, SpringerLink, ACM, Scopus, etc.*
- Search criteria: query syntax: in specific search term:
 - “*cybersecurity risks in automatic code generation*”, “*Generative AI practices*”, “*generative models and vulnerabilities*”, “*risk mitigation in automatic code generation*”

The PRISMA Flowchart of the final sample size is shown in Fig. 2.

Screening and selecting sources

- First-level screening: We screen abstracts and titles to include relevant and reliable sources.
- Inclusion criteria:
 - Literature regarding cybersecurity and AI in the realm of automatic code generation, papers that present solutions to mitigate the identified risks.
 - Recent research papers—Within the past 5–10 years.
 - Consider both cybersecurity threats and AI-specific mitigations.
 - Studies in high-quality peer-reviewed journals and conference proceedings.
 - Updates from reputable cybersecurity firms.
 - Resources and references about generative AI techniques, approaches, models, practices, etc.

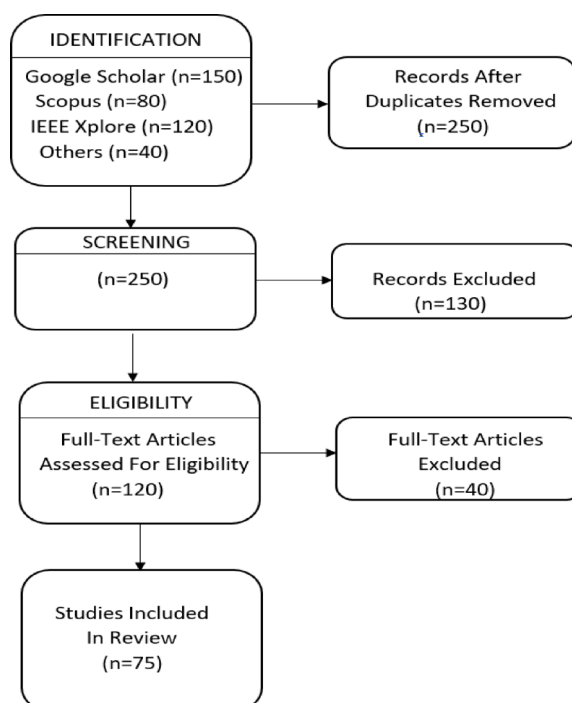


Fig. 2. PRISMA flowchart for final sample size.

- Exclusion criteria:
 - Non-relevant content about cybersecurity risks or generative AI in automatic code generation.
 - Papers over 10 years old (unless they are seminal).
 - Non-peer-reviewed sources or opinion pieces.

Data extraction and synthesis

We extract the following information from the selected papers:

- Cybersecurity risks found: What are the primary cybersecurity risks mentioned in association with code automation?
- Mitigation approaches: What are the proposed generative AI practices, methods, or technologies to mitigate risks?
- Emerging trends: We seek new or novel approaches to secure automated code generation against cybersecurity threats.
- Challenges and gaps: We discuss areas in which our literature review highlighted potential gaps in, or limitations of, the current literature.
- Categorize results: We classify our findings under several categories, such as Cybersecurity risks (e.g., data poisoning, adversarial attacks), Model robustness, and security protocols—best practices and recommendations in generative AI for safeguarding automated code generation.

Analysis and thematic clustering

Identify themes and variations within themes: After organizing the data, we looked for themes and variations within themes across sources. For example:

- Security threats in automatic code generation.
- Threats of the abuse of generative AI in generating deepfakes or counterfeit content.
- Proactive strategies for mitigating bias include adversarial training, model verification, and an AI ethical framework.
- Cross-source comparisons: We contrasted sources' conclusions about cybersecurity risks and mitigation strategies.

Synthesizing results and presenting findings

- Provide a holistic view: We consolidated the most-mentioned cybersecurity risks and the generative AI mitigations practices found across all the sources. Explain the significance of these observations to cybersecurity issues and solutions in generative AI.
- Draw attention to research gaps: We identify unexplored and under-researched topics useful for incoming research. It may be more evidence, new mitigation approaches, or a joint academia-industry partnership.
- Discussion of limitations: We discussed limitations of the current review (e.g., potential bias of sources, restricted availability of databases, or absence of research).

Formulating implications and recommendations

- Develop implications: Drawing from the synthesis, we suggest practical implications for research and practice. For example, it might indicate where additional work is required before we can safely rely on automatic code generation.
- Practice implications: We provide practical recommendations for managing cybersecurity risks in automatic code generation, such as introducing specific security standards, regulatory frameworks, or audits of generative AI.

Writing and structuring the literature review

- Introduction: We present a discussion about cybersecurity risks and generative AI practices importance and limitations in automatic code generation.
- Methods: We describe how the MLR was performed and why various voices and sources are included.
- Main body: We present the results under three main themes (exposure, mitigation, and challenges).
- Conclusions: We conclude by summarizing the main findings, identifying research gaps, and making recommendations for future research and practice.

By taking these steps, the MLR contributes to a well-informed and balanced discussion of cybersecurity risks and generative AI methods for addressing them, in which various voices and perspectives are heard. This will be beneficial both academically and by transferring knowledge between academia, industry, and practical application.

Phase 2: online questionnaire survey

The second step of this research was to design a questionnaire survey, and several essential elements were considered to develop a comprehensive and valuable research. The main objective of the survey is to find out what sorts of cybersecurity risks exist in automatic code generation, and as a secondary task was to understand how generative AI can be applied to address these risks. The following steps were followed in this survey^{39–43}:

- First of all, we set a target audience for this survey. We choose the cybersecurity researchers, the software developers, the AI researchers, and users and developers of automatic programming tools. The participants have some experience with AI, code generation, and cybersecurity. This guarantees the answers are the results of knowledge and applicable to the study's objectives. The final sample size of participants in this survey is 70.
- The next stage was the construction of the questionnaire itself. The survey covers various categories of questions, such as demographic, managing risk in cybersecurity identification questions, artificial intelligence practices for mitigating risk questions, and technology and tools questions in the field. Demographic background questions, how to get basic information about the sample, like their role in the industry, and their working experience. For example, we inquire:
 - How would you describe your role in the organization?"
 - How many years of experience in software development/AI/cybersecurity?
- Furthermore, in our questionnaire, questions on the identification of cybersecurity risk were evaluate the participant's knowledge of typical (security) risks related to automatic code generation, e.g., code injection, data leakage, and insecure APIs. e.g., a question of concern is,
 - What are the most common cybersecurity risks that exist in automatic code generation?
- Fig. 3 presents descriptive statistics of the respondents of the questionnaire. AI practices for risk reduction are indispensable for anyone considering how generative AI is employed to counter threats. We included further questions such as,
 - Have you heard of any AI-driven methods for finding vulnerabilities in code?
 - What are your thoughts on how generative AI can help counter the cybersecurity challenges associated with automatic code generation?
- In addition, technology and tooling-related questions were applied to determine what platforms are in use in automatic code generation and whether they incorporate any AI-related security capabilities.
- The survey format consisted of an introduction, which was a short profile of the study giving a précis of the survey's intent, addressing how the responses would be used, and the amount of time to complete the instrument. Also included is an informed consent with a statement explaining that the information provided is confidential and that participants' identities will remain anonymous. The survey was structured according to the same topics: demographics, cybersecurity risks, AI practices, and technology/tools. At the end, participants were thanked, and if relevant, we mentioned any subsequent events (e.g., a presentation), such as the sharing of results.
- Then, finding the right survey tool is also significant. We use online tools like Google Forms to produce and circulate the survey. This product enables us to develop, distribute, and analyze surveys to fit different levels of functionality. After choosing the survey instrument, a pilot test should be performed with a limited number of subjects. This trial was used to debug question clarity problems, survey length, or tool usability, and to ensure that the survey functions properly once disseminated more broadly.
- After the pilot testing was undertaken and any problems rectified, the survey was distributed to the larger population of interest. Here we resort to professional networks such as LinkedIn, GitHub, stack-overflow or AI/cybersecurity forums to facilitate reaching the appropriate responders. We also commit to a specific survey completion deadline so that you can rest assured that we will collect the data when we should.
- Seizing the data first and then analyzing it. We tracked the responses to check that everything is going well during the survey. Subsequently, the data were examined and read across for trends, patterns, and nuances. Quantitative data were analyzed using statistical packages such as SPSS and Excel, while the open-ended responses were analyzed through thematic analysis to identify themes. This enables us to interpret the survey findings.
- Lastly, the results were presented systematically in this paper. The reports concentrate on the top cybersecurity concerns of the respondents and offer insight into the generative AI methods that are most frequently employed to counter those concerns. Ethical issues were also taken care of by maintaining the privacy and confidentiality of the respondents. The study follows the ethical principles of research involving human subjects, including obtaining informed consent and maintaining confidentiality.

Phase 3: expert panel review

An expert panel review was performed to assess the research presented in this paper. The panel was comprised of 19 experts from multiple domains such as cybersecurity, AI, software development, and automated code generation tools. They were from several sectors: academia, industry, and research labs, with a mix of professionals having experience in:

- Cybersecurity and risk management practice requirements
- AI-driven products, particularly those using generative AI technologies
- Experience in software development, preferably automatic code generation
- Ethical implications of cybersecurity and AI

Between them, these individuals have over a decade of experience, many of whom have advanced degrees and have held professional leadership positions in their particular area. The study design is a rigorous process, involving sequential Delphi rounds. During any given round, the MLR is scrutinized by experts, who critically appraise the

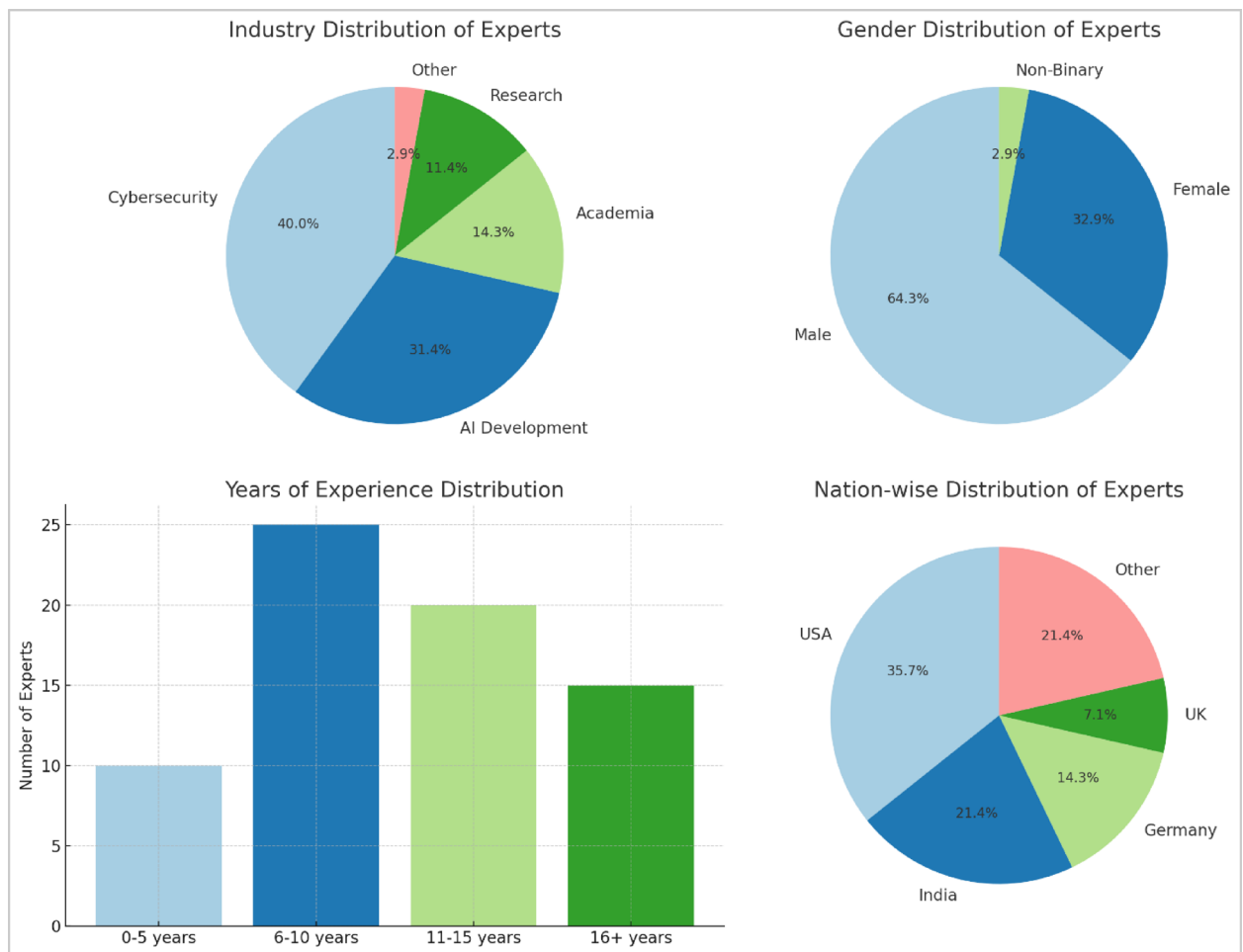


Fig. 3. Demographic details of survey experts.

MLR and results with extensive feedback on the research design and areas for potential improvements. Relevant input from the expert panel is thoroughly integrated into the research, which sharpens the research questions and provides a more specific scope for the ANN-ISM framework.

The experts assessed the cybersecurity risks on a holistic risk scale. Penalty for perceived lower (~5%) and medium importance (~45–50%) risks was set to 1 and 10, respectively. Other risk scores were scored in 5 percent units, creating a stepwise scale for responses. These expert judgements are adopted to construct pairwise matrices, which encapsulate the interrelations of different cybersecurity risks in automatic code generation, which are presented in Table 1.

To ensure the reliability and validity of the research model, ANN and ISM were used as two other analytical methods. Such analyses enable a more in-depth understanding of the results, which helps to illustrate the face validity of the findings and ultimately adds to the strength of the research process.

Phase 4: artificial neural network (ANN)

The ANN process was applied in the fourth stage of this research. It is also easy to adapt to new data sets. One of the advantages of the ANN is that it can work with incomplete or missing data inputs⁴⁴. ANN's predictions, as a rule, are superior to those of other techniques such as SEM, multiple linear regression, MDA, and binary logistic regression. Inductive Structural Modeling (ISM) is commonly employed to identify the implications of predictors on a predictor variable. Still, linear techniques like ISM have limitations in capturing the non-linear process of human decision-making by neglecting higher-level relations^{45,46}. ANN, being a well-known AI model, can overcome this drawback by mimicking decision-making scenarios and nonlinear relationships, as emphasized by Leong et al.⁴⁵. ANN's multi-layer perceptron/structure simulates the relationship between inputs and outputs, similar to how the human brain operates. An important aspect of ANN is its capability to model the nonlinear and non-compensatory links between the attributes⁴⁷.

In summary, ANN models are more accurate than the classical linear approaches and provide remarkable flexibility and generosity⁶⁴. However, ANN is inappropriate for attributive analysis or hypothesis testing^{45,48}. To address this problem, a two-stage approach based on the integration of ISM with ANN has been proposed.

ANN training

When an ANN is trained, we model intrinsic relationships between inputs and outputs by adjusting its internal weights, and the input/output pairs are shown as:

$$S = (d1, x1), (d2, x2), \dots, (dNi, xNi) \quad (1)$$

The input parameters, referred to as x_i , and corresponding output responses, denoted d_i , comprise a random sample. These data sets display the inherent non-linear correspondence between inputs and outputs. The objective is to build an ANN model capable of learning this kind of invariant relationship independently. Typically, the output of the ANN is written as $w_{ij}y_i + b_i$

$$y = y(x, w) \quad (2)$$

where y is the ANN output, x is the input parameters, and w indicates unknown weights. The optimal weights can be found by solving an optimization problem, which minimizes the disparity between the predicted output and the real label. This optimization can be formulated as:

$$w^* = \min \times ET = \min \times i || d_i - \gamma(x_i, w) || \quad (3)$$

where ET represents the sample standard error. There are many ways to approach this problem, and the most widely known is backpropagation, proposed by Hertz et al.⁴⁹. This is a method that adjusts the weights of the network by computing the estimated gradient of the error function with respect to the weights, which leads to better predictions:

Code no	Cybersecurity risks (CRs)	Description	Examples
CR1	Injection attacks ¹⁹	Automatic code generation can inadvertently introduce injection vulnerabilities, such as SQL or command injection, where attackers can inject malicious input to manipulate the code's behavior	Malicious inputs that manipulate database queries or system commands to leak sensitive data or gain unauthorized access
CR2	Code quality and logic errors ⁶⁴	Generated code may contain logical flaws or inefficient code, leading to vulnerabilities. These flaws can arise from the limited contextual understanding of the AI and might result in improper error handling, unchecked data flows, or bad design practices	A bad input validation or throwing exceptions incorrectly might expose the system to be exploited
CR3	Backdoors and malicious code ^{2,7}	Adversaries could inject backdoors or other types of malicious code into the generative AI model during the model training process, or the model can mistakenly output insecure code	A malicious actor could teach an AI system on malicious software, subsequently having the tool create software with embedded vulnerabilities or unauthorized entry points
CR4	Vulnerabilities in reused code (legacy dependencies) ⁶⁵	Most auto-generated code is dependent on existing libraries/frameworks that have a risk profile that is not being managed	Generating code that uses an old version of a known buffer overflow library, which an attacker could abuse
CR5	Insufficient Input Validation ³⁰	Generated code may not include sufficient input validation and could be compromised by users sending carefully crafted input that is designed to cause buffer overflows or execute malicious code	AI-led code for user authentication fails to properly validate input fields, making the code vulnerable to SQL injection or buffer overflow
CR6	Weak authentication and authorization mechanisms ⁶⁶	The code generated may lack a secure, strong authentication/authorization mechanism, exposing the systems to unauthorized access, privilege elevation, etc	A web application created by AI might provide the user a means to circumvent the login, or the access control might be weak or ineffectively implemented, allowing privilege escalation
CR7	Lack of encryption or insecure data handling ⁶⁷	Generated code may also overlook encrypting data or storing it in unsecured ways, which will invite theft or unauthorized access to data	A password management application that stores passwords and/or financial information in clear text. If the application is compromised, sensitive data can be easily captured
CR8	Reusability of vulnerable code ¹⁰	Code sharing in AI models could lead to similar vulnerable code in different projects. The kind of security risks, if not addressed, can spread throughout different apps and enterprises	A commonly used AI software may produce code from an API with security flaws that no one checked for, and that code might have gone unnoticed across multiple systems
CR9	Lack of secure code review and testing ⁶⁸	Some AI-generated code might not be scrutinized in the same detailed ways, so that security vulnerabilities could be missed	The source code is not necessarily subjected to a static analysis for security weaknesses and exploitable vulnerabilities
CR10	Adversarial attacks on AI models ²¹	The AI model running for code generation could be attacked by attackers who abuse the training data to let the AI output malicious or incorrect code	An adversary input can fool the AI into providing code with vulnerabilities or adding back doors to the system
CR11	Over-reliance on AI model ²¹	Developers can rely too much on AI scripts and skip on the due diligence, which results in vulnerable systems or code that is not fully understood	Unthinkingly relying on AI-generated code without scrutinizing it can result in implementing insecure algorithms or suboptimal security methods
CR12	Privacy issues and data leakage ⁶⁹	Two data-level issues appear across different application domains: AI models can produce code that unintentionally interfaces private data with code (resulting in data leakage, or violation of privacy regulations such as GDPR or HIPAA)	An AI-created web form might mismanage personal data and expose information about users to third parties
CR13	Insecure integration with other systems ⁷⁰	Alternatively, auto-code generation may create vulnerabilities if not safely integrated with other systems or services	A code snippet is generated for third-party payment API integration, where an API key is exposed, or your payment gateway may not handle authentication well
CR14	Insufficient logging and monitoring ⁷¹	Code produced by AI may lack proper logging and monitoring capabilities designed to trace anomalous or incident activities	Without adequate logging, serious security incidents, including attempts to gain unauthorized access, could remain unnoticed and unchallenged

Table 1. Cybersecurity risks in automatic code generation.

$$\omega_{\text{next}} = \omega_{\text{now}} - \eta \alpha E\tau / \alpha \omega \quad (4)$$

Hertz et al.⁴⁹ set the learning rate to h . Initially, the weights are chosen randomly, and the algorithm is repeated until the optimization condition of Eq. (3) is satisfied. Weights and biases are updated during this process, minimizing the mean squared error and allowing the model to attain the target accuracy.

The weights (W_i) and biases (b_i) are adjusted until the model obtains the desired accuracy. Alnaizy et al.⁵⁰ provide a calibration procedure denoted:

$$V_i = \sum_{j=1}^n w_{ij} y_j + b_i \quad (5)$$

The bias b_i adjusts the weighted sum of inputs. A transfer or an activation function is next used on the sum V_i . This transformation produces the:

$$Z_i = f(V_i) \quad (6)$$

Performance of ANN Training

The performance of the ANN is evaluated using the Root Mean Squared Error (RMSE), the R-squared (R^2), and the Average Absolute Deviation (AAD), expressed as:

$$\text{RMSE} = \left[\frac{1}{n} \sum_{i=1}^n (Y_i - Y_{id})^2 \right]^{0.5} \quad (7)$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (Y_i - Y_{id})^2}{\sum_{i=1}^n (Y_i - Y_{in})^2} \quad (8)$$

$$\text{AAD} = \left[\frac{1}{n} \sum_{i=1}^n \frac{(Y_i - Y_{id})}{Y_{id}} \right] * 100 \quad (9)$$

where, Y_{id} is the observed data; Y_i is the predicted data; Y_m is the median of observed data, n is the total number of data.

Phase 5: interpretive structural modeling (ISM)

The ISM approach was applied in the fifth stage to classify and rank the identified cybersecurity risks in automatic code generation. The concept of the ISM method, which is explained in⁵¹, was presented to analyze and understand complex relationships among systems and subsystems. By organizing a hierarchy, this method contributes to the acquisition of ability by organizing the variations and the directions of various elements. Further, ISM can well model the relationships between visual and structured language⁵². This method is compelling in investigating complex multivariate interactions^{53–55}. This approach has been used in many studies better to understand complex systems^{56–63}. Figure 1 shows how ISM can be used keeping in view in mind to map and classify cybersecurity risks in automatic code generation.

Phase 6: development, implementation, and validation of the hybrid ANN-ISM framework for mitigating cybersecurity risks in automatic code generation with the use of generative AI practices

In the final phase of this research, all the findings of phases 1–5 were merged to develop the hybrid ANN-ISM framework for mitigating cybersecurity risks in automatic code generation using generative AI practices. The proposed model was then implemented in an organization and was validated through a case study. Further details are presented in Sect. “[Framework evaluation](#)”.

Results and analysis

Cybersecurity risks in automatic code generation

It is essential to detect cybersecurity risks in automated code generation and to implement them with AI approaches; otherwise AI AI-based tools of software development may increase the risk of introducing hidden, and thus potentially exploitable, vulnerabilities for attackers. Unguarded automated code generation can result in code that has built-in vulnerabilities, security holes, or dependencies on the past or unsafe libraries, potentially leading to application integrity, confidentiality, and availability being compromised. Early exposure to these risks can allow organizations to adopt AI-driven security activities, like secure coding guidelines, adversarial testing, and continuous validation, to work toward elevated security standards in the generated code. It not only keeps the software from possible security leaks but also reduces the adverse effects of long-term cybersecurity threats. It also offers a secure and stable environment for using AI technologies in software development. Table 1 presents various cybersecurity risks identified through the literature review and survey.

Statistical analysis of cybersecurity risks in automatic code generation as identified through MLR on real-world study

To conduct a sound statistical analysis based on the data of MLR and real-world study in Fig. 4, we examine descriptive statistics (mean, standard deviation), correlation, and comparison of means:

- Mean = $\mu = \frac{\sum X_i}{n}$
 - MLR Mean = 74.43%
 - Real World Study Mean = 76.0%
- Standard Deviation = $\sigma = \sqrt{\frac{\sum (X_i - \mu)^2}{n}}$
 - MLR Standard Deviation = 9.93%
 - Real World Study Standard Deviation = 8.22%
- Pearson Correlation Coefficient = $r = \frac{n \sum X_i Y_i - \sum X_i \sum Y_i}{\sqrt{[n \sum X_i^2 - (\sum X_i)^2][n \sum Y_i^2 - (\sum Y_i)^2]}}$
 - MLR and Real World Study Correlation (r) = 0.94
 - The observed correlation is very positive (circa 1), which indicates that the impacts of the Multivocal Literature Review and the Real World Study go in the same direction.
- T-Test for Comparing Means: We can use an independent t-test to compare whether the means of the two samples are genuinely different. This test allows us to determine whether it is a coincidence that the impact rates are different between the two sets.
 - T-Test Formula (for two independent samples) = $t = \frac{\mu_1 - \mu_2}{\sqrt{\frac{S_1^2}{n_1} + \frac{S_2^2}{n_2}}}$
 - T-Statistic = -0.44
 - P-Value = 0.66
 - The p-value is 0.66, higher than the significance level (0.05). Hence, we do not reject the null hypothesis and that there is no statistical difference between the means of the two samples.

The means of the two sets are similar to each other with a slight variance (74.43% vs 76.00%), which is not statistically significant according to a t-test. The two sets are highly correlated (0.94), implying the similarity

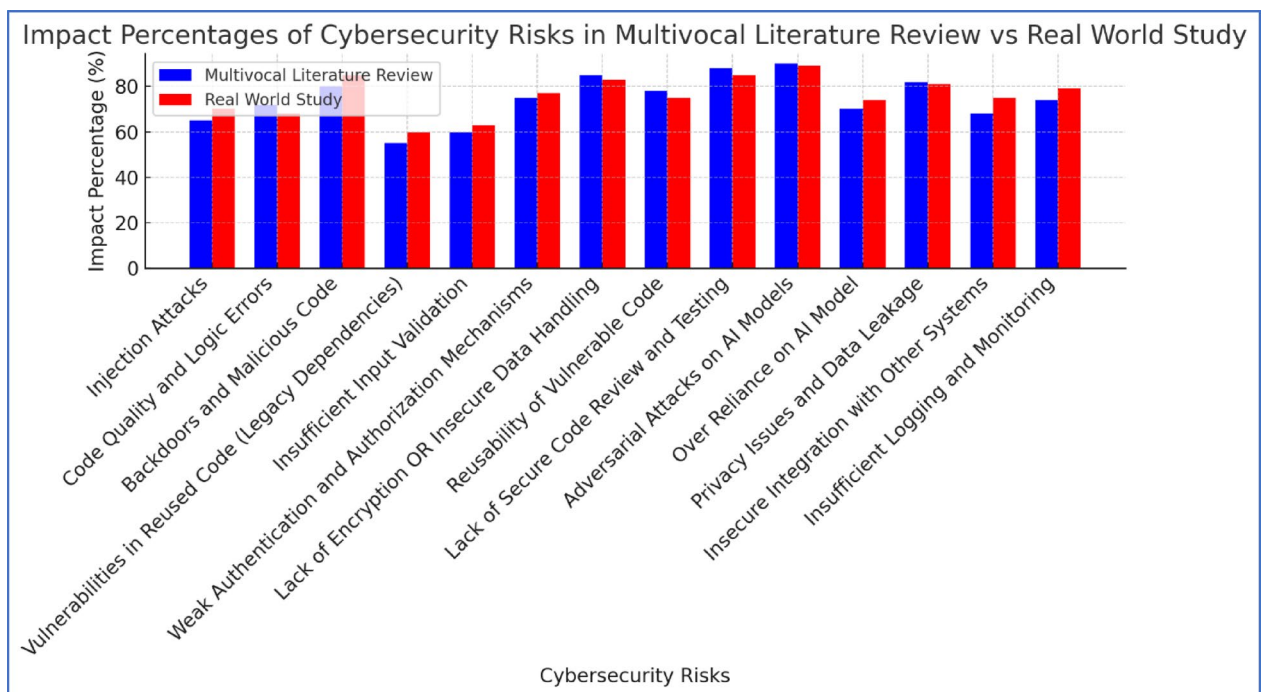


Fig. 4. Impact percentages of cybersecurity risks identified through MLR and survey in automatic code generation.

in the trend of MLR and the real-world study. These considerations indicate that the two datasets are highly correlated, but the absolute impact percentages are not radically different.

Analysis of cybersecurity risks in automatic code generation based on software development organization sizes

Figure 5 presents the effect of some cybersecurity threats on automatic code generation for three company sizes: small, medium, and large. All cybersecurity risks (CRs) are depicted on the x-axis, while vertical bars indicate the impact percentage for small, medium, and large companies. This makes it possible to compare how these risks manifest against various sizes of software development organizations.

Looking at overall trends, we can see that small firms (i.e., the blue bars) will have the highest impact percentages across various cybersecurity risks. This may indicate that smaller companies face relatively higher cybersecurity challenges because of having fewer available resources, weaker security infrastructures, and a smaller number of cybersecurity professionals. The middle of the road is medium companies in the green bars, which can still suffer; it is moderate because they have some security in place, but are still at risk with their increased complexity and scale. By contrast, larger firms, depicted in red, have much lower share percentages. In other words, although the employees of large companies still struggle with cybersecurity risks, the organization is usually stronger, has more resources, and better-established security procedures to manage those risks.

Some particular risks offer more context into how the ill effects are distributed across the sizes of firms. For instance, Injection Attacks (CR1) are demonstrated to have a high rate of influence on all company sizes, mostly on small companies. This is not of much surprise as, for small companies, they will not have sophisticated security mechanisms nor a significant amount of code review to protect against this kind of attack. Likewise, Code Quality and Logic Errors (CR2) achieve the highest impact on small companies, possibly because they have less formalized methods of coding and quality assurance. This risk can be minimized by bigger companies having more vigorous testing and reviewing processes in place. Backdoors and Malicious Code (CR3) also significantly impact small organizations, which might indicate that small companies have poor control over third-party libraries. Small companies demonstrate an intense vulnerability for Insufficient Input Validation (CR5), which reflects the impact of having fewer developers and the use of automatic generation tools that may not have been deeply tested against edge cases.

Other risks, e.g., Weak Authentication and Authorization Mechanisms (CR6), diminish impact with increasing company size. Big companies have more robust ways to authenticate security for their systems, while smaller companies may lack those protections. A Lack of Encryption or Insecure Data Handling (CR7) is also risky for small companies, as there is a higher risk due to an insufficient security budget.

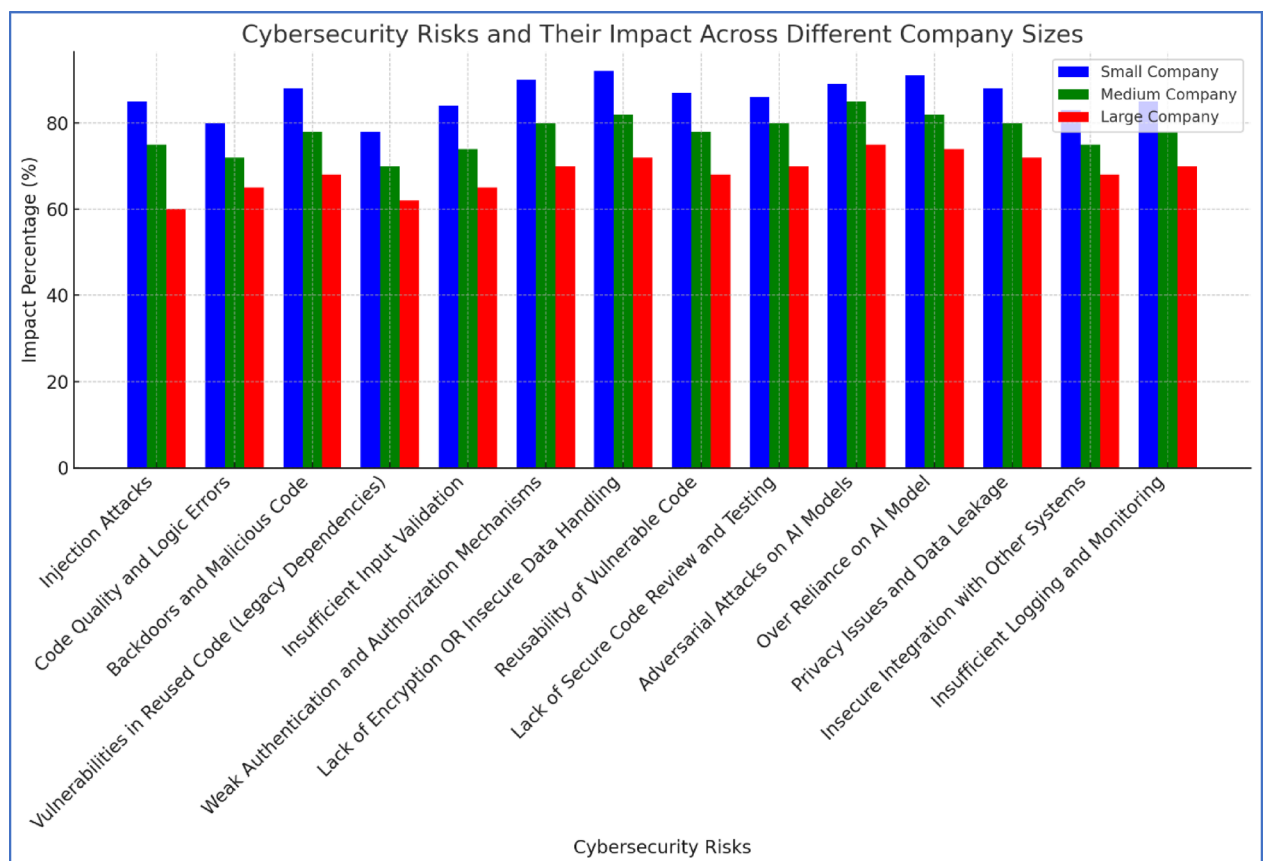


Fig. 5. Cybersecurity risks in automatic code generation and its impact on company sizes.

A general trend can be observed from the entire data set—with larger-sized companies, the percentage figures of impact decrease. It indicates the perception that the bigger the company, the better prepared they are to manage cyber risks due to better security defences, dedicated resources, and in-built processes.

Small companies were found to be the most vulnerable to security problems that arise with automatic code generation: they lack funds, knowledge, and protective mechanisms. Mid-sized companies have made modest progress from a risk-management standpoint but remain highly exposed. The larger companies, on the other hand, usually have less sweeping effects overall, due to bigger security tools budgets, more well-prepared personnel, and grander schemes of security procedures. However, it does not mean they are very safe from risks, especially those related to how complex they are. The graph effectively demonstrates corporate size and the correlation between investments in security infrastructure to manage cybersecurity risks related to automatic code generation.

Analysis of cybersecurity risks in automatic code generation based on software development organization continents

Figure 6 is a line chart depicting the global landscape of cybersecurity risks in automatic code generation by continent (North America, Europe, Asia, Africa, South America, and Oceania). Each line indicates the percentage impact of each cyber risk, with the y-axis being the severity of the dangers across multiple regions.

Asia has the highest exposure to cybersecurity risk vectors, including Injection Attacks, Backdoors, Malicious Code, and Adversarial Attacks on AI Models. It implies that there is much to do to tackle these risk vectors. This increased impact is the result of the rapid implementation of AI without well-established cybersecurity defenses; in addition, there is a vast technology gap between the various countries in Asia.

Africa also records high impact rates of many risks, notably Insufficient Input Validation, Weak Authentication, and Reusability of Vulnerable Code. The high hit rate is understandable because the level of investment in security infrastructure is not high, and many African countries use outdated systems and technologies. On the other hand, there is another rather significant impact of Adversarial Attacks on AI Models, which reflects a potentially increasing difficulty of securing AI-based systems.

North America and Europe have lower impact ratios due to the stronger cybersecurity practices, laws, and tech deployed in these regions. As the chart demonstrates, these regions can better manage risks like Weak Authentication, No Encryption, and Privacy Issues thanks to a mature cybersecurity posture, as seen with GDPR in Europe and regulatory authorities in North America.

In South America, there are only moderate percentages of impact, though some threats are relatively higher than others, like Privacy Issues and Data Leakage effects. This indicates that even as certain South American countries progress regarding cybersecurity, data protection, and systems integration remain issues.

Oceania, which takes in countries such as Australia and New Zealand, has modest effects like those in South America. The region is one of the most advanced ones in the world, but it is still vulnerable to Insecure Integration with Other Systems and Insufficient Logging and Monitoring. These risks are a reminder that even affluent areas can struggle to cover complex software systems adequately or to perform complete security monitoring effectively.

Figure 5 demonstrates the differing levels of cybersecurity risk exposure and measures for mitigation globally. Some regions do better in comparison to others in terms of managing cybersecurity risks, with North America and Europe seeing smaller risks while Asia and Africa face greater risks, mainly due to obsolete infrastructure, resource constraints, and rapid adoption of new technologies. This visualization clearly emphasizes the importance of securing the auto code generation and AI-based technology and global collaboration to build a better cybersecurity infrastructure, particularly in the EMDEs, to counter the evolving threat landscape.

Generative AI practices and tools for addressing cybersecurity risks in automatic code generation

Table 2 presents different Generative AI techniques as well as tools that are intended for mitigating various types of cybersecurity threats that are caused by to automatic generation of code. These are practices and tools that are valid in many diverse areas, with special emphasis on reducing the attack surface and increasing the security, quality, and operational integrity of the code.

Here is the breakdown of key areas in Table 2:

- **Injection attacks:** Input validation and sanitization are among the best practices to avoid injection attacks, which means that data inputs do not contain potentially harmful commands. TensorFlow, OpenAI GPT-3 help automate many of these validations. These practices, together with code obfuscation and encryption (Jscrambler, CodeShield), also contribute to increasing the level of difficulty for adversaries to conduct automated attacks on generated code. There is also the mandatory static code analysis for vulnerability detection with tools such as Checkmarx and Snyk, which is vital to find vulnerabilities in our code integrations early.
- **Quality of code, logic, and flexibility:** Static code analysis, automated unit testing, and flaws detection with AI models are among the recommended practices mentioned in the table for improving the code quality and handling logic mistakes. These techniques locate faults at an early stage of the development process and will help to make the code more reliable. SonarQube and JUnit further aid automation for pinpointing errors, and AI-powered tools like Codex for refactoring and embellishing code.
- **Backdoors and malicious code:** Malware detection and static analysis for vulnerabilities of automatically generated code are the central defense mechanisms to guard against the insertion of backdoors or other malicious code. These practices scan for potential threats, with tools like VirusTotal and Checkmarx. More advanced practices, such as runtime behavioral analysis and automated penetration testing (with tools like Burp Suite), can help ensure runtime protection and detection of security breaches.

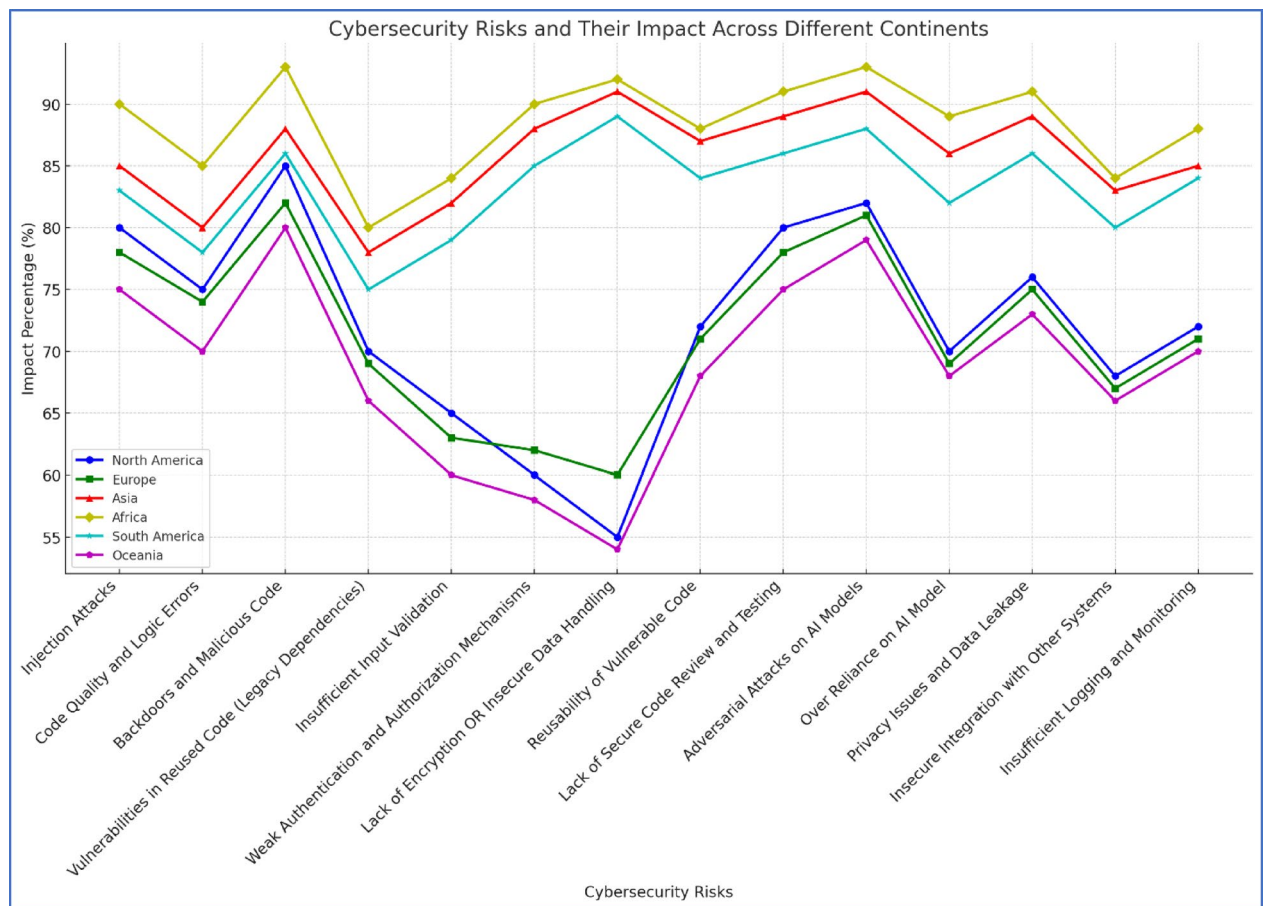


Fig. 6. Analysis of cybersecurity risks in automatic code generation based on various continents.

- **Weaknesses in reused or legacy code:** Old code that is brittle and potentially insecure can become insecure (through bad security practices) if reused. Automated dependency management and vulnerability scanning on legacy code would be a means of addressing this threat. Code like Renovate or Dependabot automatically tracks and updates dependencies that prevent vulnerability in another's libraries from finding its way in. Techniques such as automated patch generation also lessen the likelihood of knowingly introducing uncorrected vulnerabilities while keeping the technical debt in check.
- **Lack of input validation:** Automatic input and fuzz testing are essential for reducing the risk of inadequate user input validation. These are guarded to avoid attacks like SQL injection or buffer overflows. Tools such as Codex, Snyk, and others on the AI side are used to ensure we build these security validation mechanisms into our code. Dynamic input testing with tools such as Postman also helps to add a layer of security to the integration.
- **Authentication and authorization:** Concepts such as AI-based authentication generation and RBAC play a key role in ensuring the security of an access control system. Auth0 and Okta are tools that can help make sure that your authentication systems are solid, and automated authorization testing solutions (for example, Postman) can make sure permissions systems are correctly implemented.
- **Insecure data handling:** Secure data handling issues—that is, no encryption or insecure storage—can be addressed with automatic encryption code generation and AI-guided secure storage. Libraries such as OpenSSL and AWS S3 encryption apply encryption by default on data in motion, because critical information must be secured.
- **Reusability of faulty material:** AI-based vulnerability discovery, automated secure code review, can also help solve the problem of malicious code reused. Solutions like Snyk and GitHub Copilot provide proactive protection against unsafe code entering the system, ensuring only safe code is reused and included.
- **Absence of secure review and testing:** Automated code review and continuous testing are the most effective ways to secure code. AI-enabled static and dynamic application security testing (SonarQube, OWASP ZAP, etc.) guarantee that your apps follow security best practices in each phase of dev and maintenance. These are nice tools that help spot potential vulnerability issues so we can continue to secure our code.
- **Attacking AI models:** Adversarial examples can harm generative AI models as well. Adversarial training and robustness testing frameworks are used to address this issue. Utilities, including CleverHans and Foolbox, let model developers attack their models and find weaknesses, then build defenses against those adversarial inputs.

S. no	Generative AI practices	Generative AI tools
<i>Generative AI practices and tools for addressing injection attacks^{16–20}</i>		
P1	Input validation and sanitization	TensorFlow, Keras, OpenAI GPT-3, Hugging Face
P2	Code obfuscation and encryption	PyArmor, Jscrambler, CodeShield
P3	Use of secure code templates	Codex, DeepCode, SobarQube
P4	Static code analysis for vulnerability detection	Checkmarx, Veracode, Snyk
P5	Automatic generation of prepared statements	SQLAlchemy, Hibernate ORM
P6	Regular security audits and penetration testing	Burp Suite, OWASP ZAP, Acunetix
P7	Context-aware AI models for reduced vulnerabilities	OpenAI Codex, GPT-3
<i>Generative AI practices and tools for addressing code quality and logic errors^{72–75}</i>		
P8	Static code analysis	SonarQube, Codacy, ESLint
P9	Automated unit testing	JUnit, TestNG, PyTest
P10	Code refactoring	Rafactoring.ai, JRebel
P11	Error detection with AI models	DeepCode, Codex
P12	Logic verification	Prolong, Z3 Solver
P13	Automated code reviews	GitHub Copilot, CodeGuru
P14	Continuous integration and delivery (CI/CD)	Jenkins, CircleCI, Travis CI
P15	Pattern recognition for best practices	Kite, Tabnine
<i>Generative AI practices and tools for addressing backdoors and malicious codes^{76–80}</i>		
P16	Malware detection	VirusTotal, ReScan, Malicious Code Detection AI
P17	Static analysis for vulnerabilities	SonarQube, Checkmarx, Fortify
P18	Anomaly detection	DeepCode, CodeSonar, AI-Based Threat Detection
P19	Automatic security audits	WhiteSource, CodeQL, Synk
P20	Runtime behavior analysis	Cuckoo Sandbox, AppSpy, Runtime AI
P21	Automated penetration testing	Metsploit, AI-Pentest, Burp Suite
P22	Code obfuscation and protection	ProGaurd, Jscrambler, Dotfuscator
<i>Generative AI practices and tools for addressing vulnerabilities in reuse code (legacy dependencies)^{13–15}</i>		
P23	Automated dependency management	Renovate, Dependabot, Snyk
P24	Vulnerability scanning for legacy code	Whitesource, Checkmarx, Black Duck
P25	Automated patch generation	Dependabot, AI-Patch Generation Tools
P26	Code auditing for legacy systems	Veracode, SonarQube, GitHub, Advanced Security
P27	AI-dependency upgrade	Greenkeeper, PyUp, Renovate
P28	Legacy code refactoring	Refactoring.ai, JRebel, JetBrains, ReSharper
P29	Static and dynamic code analysis	Fortify, SonarQube, Codacy
P30	Compatibility testing for new dependencies	Jenkins, CircleCI, TestComplete
P31	AI-enhanced risk assessment for legacy code	Secure Code Worrier, AI Risk Assessment Tools
<i>Generative AI practices and tools for addressing insufficient input validation^{81–85}</i>		
P32	Automated input validation generation	Codex, GPT-3, Tabnine, CodeWhisperer
P33	Pattern matching for input validation	AI-Powered Static Analysis Tools (DeepCode, SonarQube)
P34	Dynamic input testing	Postman, Swagger, JMeter, AI-based
P35	Fuzz testing	American Fuzzy Lop (AFL), Boofuzz, FuzzerAI
P36	Automated code review for input handling	GitHub Copilot, CodeGuru, Codacy
P37	Input sanitization AI models	Snyk, Veracode, AI-based Web Application Firewalls
P38	Machine learning for input validation	OpenAI Codex, IntelliCode (Vs), PyLint
P39	Real-time input validation feedback	IntelliJ IDEA, Visual Studio Code with AI Plugin
P40	AI-driven security scanning for input handling	Fortify, Black Duck, WhiteSource
<i>Generative AI practices and tools for addressing weak authentication and authorization mechanisms^{86–89}</i>		
P41	AI-driven authentication generation	Auth0, Okta, Keycloak, Firebase Authentication
P42	Automated role-based access control (RBAC)	AWS IAM, Azure AD, Okta, Auth0
P43	Contextual authentication generation	Okta, Microsoft Identity Platform, Adaptive Authentication
P44	AI-enhanced token generation and management	JWT (JSON Web Tokens), OAuth2, Auth0
P45	Secure password generation	Dashlane, 1Password, Bitwarden
P46	Automated authorization testing	Postman, Selenium, JUnit
P47	AI-driven secure session management	Spring Security, OAuth, Firebase Authentication
P48	AI-based anomaly detection in authentication	IBM QRadar, Splunk, CrowdStrike
P49	AI for permissions auditing	AWS IAM, Azure AD, Okta, Terraform
P50	Behavioral biometrics for authentication	BioCatch, Behaviosec, Zighra
Continued		

S. no	Generative AI practices	Generative AI tools
<i>Generative AI practices and tools for addressing lack of encryption OR insecure data handling</i> ^{30,90–93}		
P51	Automated encryption code generation	OpenSSL, libsodium, Bouncy Castle
P52	AI-based classification	Microsoft Azure Information Protection, Varonis, IBM Gaurdium
P53	Automated key management	AWS KMS, HashiCorp Vault, Google Cloud KMS
P54	Secure API generation with encryption	Postman, Swagger, Apigee
P55	Data masking and tokenization automation	Informatica, IBM Infosphere Optim, Protegrity
P56	AI-driven secure storage solutions	AWS S3 Encryption, Azure Blob Storage Encryption
P57	Real-time data leakage detection	Symantec DLP, Digital Guardian, Forcepoint
P58	Automated compliance checking	OneTrust, TrustArc, Secureframe
P59	Static and dynamic encryption code analysis	VeraCode, Checkmarx, SonarQube
<i>Generative AI practices and tools for addressing reusability of vulnerable code</i> ^{10,94–98}		
P60	AI-powered vulnerability detection	Snyk, WhiteSource, SonarQube
P61	Automated secure code suggestions	GitHub Copilot, DeepCode, Tabnine
P62	Dependency risk analysis	Black Duck, Dependabot, Renovate
P63	Code clone detection and remediation	PMD, CloneDR, SourceTrail
P64	Secure code template generation	Codex, OpenAI API, Kite
P65	AI-enhanced code review	CodeGuru, Code Climate, Codacy
P66	Continuous monitoring for vulnerable code usage	Snyk, Veracode, GitGuardian
P67	Refactoring legacy code automatically	Refactoring.ai, JRebel, IntelliJ IDEA
P68	Security policy enforcement via AI	SonarQube, Checkmarx, Fortify
P69	AI-powered education and recommendations	Secure Code Worrier, Cybrary, Pluralsight
<i>Generative AI practices and tools for addressing lack of secure review and testing</i> ^{23,33,34,99–101}		
P70	Automated review code	GitHub Copilot, CodeGuru, Codacy
P71	AI-powered static application security testing (SAST)	SonarQube, Checkmarx, Fortify
P72	Automated dynamic application security testing (DAST)	OWASP ZAP, Burp Suite, Acunetix
P73	Fuzz testing with AI	AFL (American Fuzzy Lop), FuzzBuzz
P74	Test case generation using AI	Testim, Mabl, Functionize
P75	Continuous integration with AI-based testing	Jenkins, CircleCI, GitLab CI with AI plugin
P76	AI-assisted peer code review	Review Board with AI, DeepCode Review
P77	Regression testing automation	Selenium, TestComplete with AI plugin
P78	Security-focused code quality metrics	SonarQube, Codacy, CodeClimate
P79	AI-powered vulnerability prediction models	CodeQL, DeepCode, CodeClimate Security
<i>Generative AI practices and tools for addressing adversarial attacks on AI models</i> ^{21,102–106}		
P80	Adversarial training	CleverHans, Foolbox, Adversarial Robustness Toolbox (ART)
P81	Input sanitization and preprocessing	TensorFlow Privacy, IBM Adversarial Robustness Toolbox (ART)
P82	Model verification and validation	Reluplex, AI2, VeriNet
P83	Robustness testing frameworks	Foolbox, CleverHans, IBM ART
P84	Defense-GAN and generative defenses	Defense-GAN, MagNet
P85	Certified robustness methods	Randomized Smoothing, Interval Bound Propagation (IBP)
P86	Ensemble methods	TensorFlow, PyTorch Ensemble Libraries
P87	Adversarial attack simulation	CleverHans, Foolbox, IBM ART
P88	Continuous monitoring and model updates	Azure ML, AWS SageMaker, Google AI Platform
<i>Generative AI practices and tools for addressing overreliance on AI models</i> ^{34,98,102,107}		
P89	Human-in-the-loop (HITL)	Label Studio, Amazon SageMaker, Ground Truth
P90	Explainable AI (XAI)	LIME, SHAP, Google Explainable AI, Tool Kit
P91	Model monitoring and feedback loops	Fiddler AI, Arize AI
P92	Multi-model ensembles	H2O.ai Driverless AI, Microsoft Azure ML Pipelines
P93	Domain expert integration	Custom workflows integrating experts
P94	Robust validation sets and testing	OpenML datasets, Kaggle Competitions
P95	User education and training	Online courses, workshops, internal training programs
P96	Fallback and override mechanisms	Custom UI controls in applications
P97	Transparency in data and model sources	Model Cards, datasheets for datasets
<i>Generative AI practices and tools for addressing privacy issues and data leakage</i> ^{4,25,106,108,109}		
P98	Code sanitization and redaction	GitGuardian, TruffleHog, Gitleaks
P99	Private codebase fine-tuning	Azure OpenAI on private endpoints, Hugging Face Spaces with private models
P100	Differential privacy techniques	Google TensorFlow Privacy, PySyft
Continued		

S. no	Generative AI practices	Generative AI tools
P101	Secure prompt engineering	Human review, prompt validation frameworks
P102	Out filtering and post-processing	Codiga, SonarQube, Snyk Code
P103	Model usage logging and auditing	MLflow, OpenAI API usage logs
P104	Access control and role-based use	IAM Systems (e.g., AWS IAM), Azure Role-Based Access Control
P105	On-premise or air-gapped deployment	Self-hosted LLMS (e.g., Code LLaMA, StarCoder)
P106	Open-source license checks	FOSSA, Tidelift, WhiteSource
P107	Security-aware LLM training	OpenAI Codex (with RLHF), Meta's Code LLaMA (custom fine-tuning)
<i>Generative AI practices and tools for addressing insecure integration with other systems^{21,103,110–112}</i>		
P108	Secure coding guidelines enforcement	OWASP Secure Coding Practices, SEI CERT guidelines
P109	Static application security testing (SAST)	SonarQube, Checkmarx, Veracode
P110	Dynamic application security testing (DAST)	OWASP ZAP, Burp Suite
P111	API security validation	Postman Security Tests, API Fostress
P112	Dependency and supply chain analysis	Snyk, Dependabot, WhiteSource
P113	Code generation with security templates	Custom Secure Code Libraries, OpenAI Codex with security filters
P114	Automated threat modeling tools	Microsoft Threat Modeling Tools, IriusRisk
P115	Runtime application self-protection (RASP)	Contrast Security, Imperva RASP
P116	Secure CI/CD pipelines	Jenkins with security plugins, GitLab CI security scans
P117	Logging and monitoring integration	ELK Stack, Splunk, DataDog
<i>Generative AI practices and tools for addressing insufficient logging and monitoring^{4,113–116}</i>		
P118	Automated secure logging code generation	Custom AI Prompts, OpenAI Codex with logging templates
P119	Integration with centralized log management	ELK Stack (Elasticsearch, Logstash, Kibana), Splunk, Graylog
P120	Anomaly detection on logs using AI	Sumo Logic, LogRhythm, Microsoft Sentinel AI Capabilities
P121	Inclusion of structure logging	Fluentd, Logstash, AWS CloudWatch Logs
P122	Real-time monitoring and alerting hooks	PagerDuty, Opsgenie, Prometheus Alertmanager
P123	Compliance-aware logging practices	AWS Audit Manager, TrustArc, OneTrust integrations
P124	Automated log retention and rotation	Logrotate, Cloud-native lifecycle policies
P125	Context-rich logging generation	Custom Logging Frameworks, OpenTelemetry
P126	Continuous logging validation in CI/CD	Jenkins Security Plugins, GitLab CI SAST Integration
P127	User behavior analytics (UBA) integration	Exabeam, Splunk UBA, IBM QRadar

Table 2. Generative AI practices for addressing cybersecurity risks in automatic code generation.

- **Overreliance on AI models:** Techniques such as HITL, explainable AI (XAI), and model monitoring help to mitigate the effects of automatic systems as well. Tools such as Fiddler AI and LIME give you transparency into how AI models are making decisions, promoting human oversight and mitigating the risks of trusting AI-generated code unquestioningly.
- **Privacy concerns and data breach:** Backed by AI-based security scanning and differential privacy, sensitive data is treated securely and doesn't get leaked due to unauthorized access. Tools like Google TensorFlow Privacy and MLflow ensure safe privacy treatment, while IAM systems guarantee secure role-based data access.
- **Poor integration with other systems:** Safe coding practices and API security validation help achieve secure code that works well with other systems. Tools such as OWASP Secure Coding Practices and Postman Security Tests are there to make sure the code is best practice and able to resist commonly known threats, which will lower the risk of exposing your backend to external entities.
- **Ineffective logging and monitoring:** Features like automatic generation of secure logging code and integration with centralized log management ensure that security events are logged and tracked. Products like Splunk and Graylog can analyze and monitor logs in real time so that potential security incidents can be discovered more easily.

To sum up, the above table demonstrates how using generative AI techniques and tools can help tackle several cybersecurity issues related to automatic code generation, in general. Developers can pull up these techniques, enabling them to discover, stop, and fix flaws that would otherwise be a problem in the code later on.

Concise threat model (cybersecurity risk mitigation for automated code generation)

- **Scope:** The threat model encompasses the complete lifecycle of automated code generation within software development, focusing on security risks at each stage, from code writing to deployment. It includes the integration of code generation tools within development environments, continuous integration (CI), and continuous deployment (CD) pipelines, as well as API services that facilitate code generation. The primary objective is to address and mitigate security vulnerabilities that arise from these various stages.
- **Assets:**

- **DE plugin:** These tools assist developers with code writing but can present security risks if compromised, potentially allowing for malicious code injection or leakage.
- **CI/CD job:** Responsible for automating testing, building, and deploying code, this component can be vulnerable to misconfigurations, backdoors, and unauthorized access.
- **API for code generation:** These interfaces interact with code generation systems, and risks such as prompt injection, manipulation, or data leaks can occur from unauthorized API requests.
- **Dependency chain:** Includes external libraries or third-party code integrated into the system, which, if vulnerable, can be exploited to introduce malicious code or alter system behavior
- **Attacker capabilities:**
 - **Injection attacks:** Attackers may manipulate inputs to exploit the code generation pipeline (e.g., injecting harmful data into APIs or prompts).
 - **Privilege escalation:** Gaining unauthorized access to parts of the CI/CD pipeline or misconfigured jobs could allow attackers to alter code or introduce malicious elements.
 - **Backdoor insertion:** Attackers may embed malicious code in dependencies or libraries, which could go undetected until activated during the generation or deployment process.
 - **Data leakage:** Cyber attackers can exploit vulnerabilities in the code generation system or its APIs to extract sensitive data, including proprietary code or credentials
- **Protection context:**
 - **Prevent:** Prevention strategies include securing API endpoints, validating inputs, ensuring secure configurations of CI/CD jobs, and applying secure coding practices across dependencies to reduce vulnerabilities.
 - **Detect:** Detection mechanisms aim to identify malicious activities as soon as they occur, such as unauthorized access, unusual outputs from the code generation system, or unexpected alterations in the codebase. Using intrusion detection systems (IDS) and anomaly detection tools can help identify these threats.
 - **Contain:** Once a breach or malicious action is detected, containment focuses on limiting its spread. This involves isolating compromised parts of the system (e.g., halting CI/CD processes, disabling vulnerable dependencies, or rolling back to secure versions of code).

This approach integrates ANN (Artificial Neural Networks) for prioritizing and scoring risks (such as injection attacks or backdoor insertions) with ISM (Interpretive Structural Modeling) to visualize the relationships between various risks and mitigation strategies across the code generation pipeline. This combined approach enables a comprehensive risk management framework by clearly identifying critical risks and corresponding intervention points throughout the entire process.

ANN model building

To better demonstrate how the ANN model was trained and evaluated, we included the details of the training and testing process. The dataset consists of 14 types of cybersecurity vulnerabilities concerning automatic code generation, and the ratio of training and validation sets is 70% and 30% to avoid over-fitting. Tenfold cross-validation was performed to validate the model. The ANN has 14 input features listed in Table 1 passing through the input layer, and the dependent variable, cybersecurity risks in automatic code generation, in the output layer. The Adam optimizer with a learning rate of 0.001 conducts training over 50 epochs on a batch size of 32 using cross-entropy loss for classification. Root Mean Square Error (RMSE) was used to test model performance, and results presented an average RMSE of 0.902 for the model testing set and 0.329 for its training set as provided in Table 3. The importance and normalized importance of the cybersecurity risks in automatic code generation are listed in Table 4. Figure 7 shows the normalized importance and sensitivity analysis, highlighting how well the ANN model represents the nonlinear dependence of independent variables and their effect on these risks. Figures 8 and 9 offer further understanding of the relationships between how changes in predictions impact the values of the independent variables and the importance and normalized importance of these risks, and how the output values are altered by differences in model predictions⁴⁸.

The sum of square error (SSE) value 6.329 denotes the mean squared error of the model predictions with the training set (see Table 3). The lower the SSE, the better the model fits the training data. In this case, 6.329 is a relative small error, which means the model does a pretty nice job of capturing the patterns in the training data. A relative error of 0.985 indicates the model generally predicts values with 98.5% of the real values. As it is near 1, this tells us the model is making a good prediction on training data, and a small error is being left behind.

The SSE is 0.062, which tells us of the model's capability to generalize to new data. The test dataset SSE is remarkably better than SSE on training data of 6.329, indicating how the model performs on new, unseen data with good generalization (less overfitting). Relative error 0.054 is very low, which means the model fits the testing data very well. This is a good indication of the model's generalization to unknown data.

The fact that the testing error is much lower than the training error can indicate that the model is not overfitting; therefore, it can generalize well to new observations. The model seems to be a good one; it makes good predictions both on the training data and the out-of-scope data. Many reporting metrics demonstrate great prediction of Cybersecurity Risks in Automatic Code Generation.

Figure 7 shows the normalized importance and sensitivity analysis of cybersecurity risks to automatic code generation according to the ANN model, which captured the nonlinear relationships between the independent variables and their influence on the cybersecurity risks. Further, to understand this impact, we extend Figs. 8 and 9 to the effects that predicted output value variations would have in independent variable values. A simple means for providing a precise estimate of the importance of these risks is summarized by giving the normalized

Model summary		
Training	Sum of squares error	6.329
	Relative error	0.985
	Stopping rule used	1 consecutive step(s) with no decrease in error ^a
	Training time	0:00:00.01
Testing	Sum of squares error	0.062
	Relative error	0.054

Table 3. ANN model summary. Dependent variable: cybersecurity risks in automatic code generation. a. Error computations are based on the testing sample.

Cybersecurity risks in automatic code generation	Importance	Normalized importance (%)
Injection attacks	.177	100.0
Code quality and logic errors	.053	29.9
Backdoors and malicious code	.051	29.0
Vulnerabilities in reused code (legacy dependencies)	.072	40.6
Insufficient input validation	.037	20.7
Weak authentication and authorization mechanisms	.062	34.9
Lack of encryption OR insecure data handling	.063	35.7
Reusability of vulnerable code	.091	51.3
Lack of secure code review and testing	.074	42.0
Adversarial attacks on AI models	.052	29.3
Overreliance on an AI model	.013	7.1
Privacy issues and data leakage	.126	71.2
Insecure integration with other systems	.019	10.8
Insufficient logging and monitoring	.112	63.3

Table 4. Importance and normalized importance of independent variables (cybersecurity risks).

significance of these risks in terms of how changes in predicted output values from the network model affect the independent variables⁴⁸.

Importance and Normalized Importance appear in Table 4 summarizes the relative importance of each risk relative to the study. Importance: This is the unadjusted, raw significance level of each cybersecurity risk. It shows the relative strength of each risk overall system or model under investigation. For example, “Injection Attacks” is the most important of them with an importance factor of 0.177, which indicates that it is the most critical threat faced in this context. On the other hand, “Over Reliance on AI Model,” with importance 0.013, has the lowest importance value, which means it is less contributive to different risks.

Normalize Importance: This column shows the relative importance of each risk, expressed as a percentage, normalized against the highest importance value in the table. The most critical risk, “Injection Attacks,” is 100.0%. The other dangers are measured concerning this maximum value. For instance, “Code Quality and Logic Errors” has a normalized importance of 29.9%, which is compared to “Injection Attacks” being five times less. Other threats are the “Privacy Issues and Data Leaks” (71.2%) and the “Insufficient Logging and Monitoring” (63.3%), which have a high importance level but are still far below “Injection Attacks.” Risks such as Over Reliance on AI Model (7.1%) and Insecure Integration with Other Systems (10.8%) seem lower priority in the context of automatic code generation.

Normalizing the importance scores, this table allows for straightforward comparison between the relative importance of the different cybersecurity threats, guiding on the area with the most impact for improving the security of the code generation.

The normalized level of importance of different cybersecurity risks for automatic code generation is illustrated in Fig. 7. These adversarial risks are encoded by several “cybersecurity risks” (CRs), and there are 14 different CRs (CR1-CR14). The horizontal bars in the figure show the degree of importance, where the larger the bar, the more important the factor is concerning cybersecurity for automatic code generation.

As in Fig. 7, CR1 (Injection Attacks) has the most significant normalized importance, suggesting it is the highest risk for the listed categories. This is further continued by CR12 (Privacy Issues and Data Leakage) and CR14 (Insufficient Logging and Monitoring), where the importance of these CRs remains significant, but slightly lower than CR1. These three threats are also ranked high in the analysis of cybersecurity threats in automatic code generation.

On the opposite end of the scale, we notice that CR11 (Over Reliance on AI Model) and CR13 (Insecure Integration with Other System) have the lowest normalized importance, indicating that although these risks are still being observed, they are not assessed as urgent as other risks such as injection attacks or data privacy.

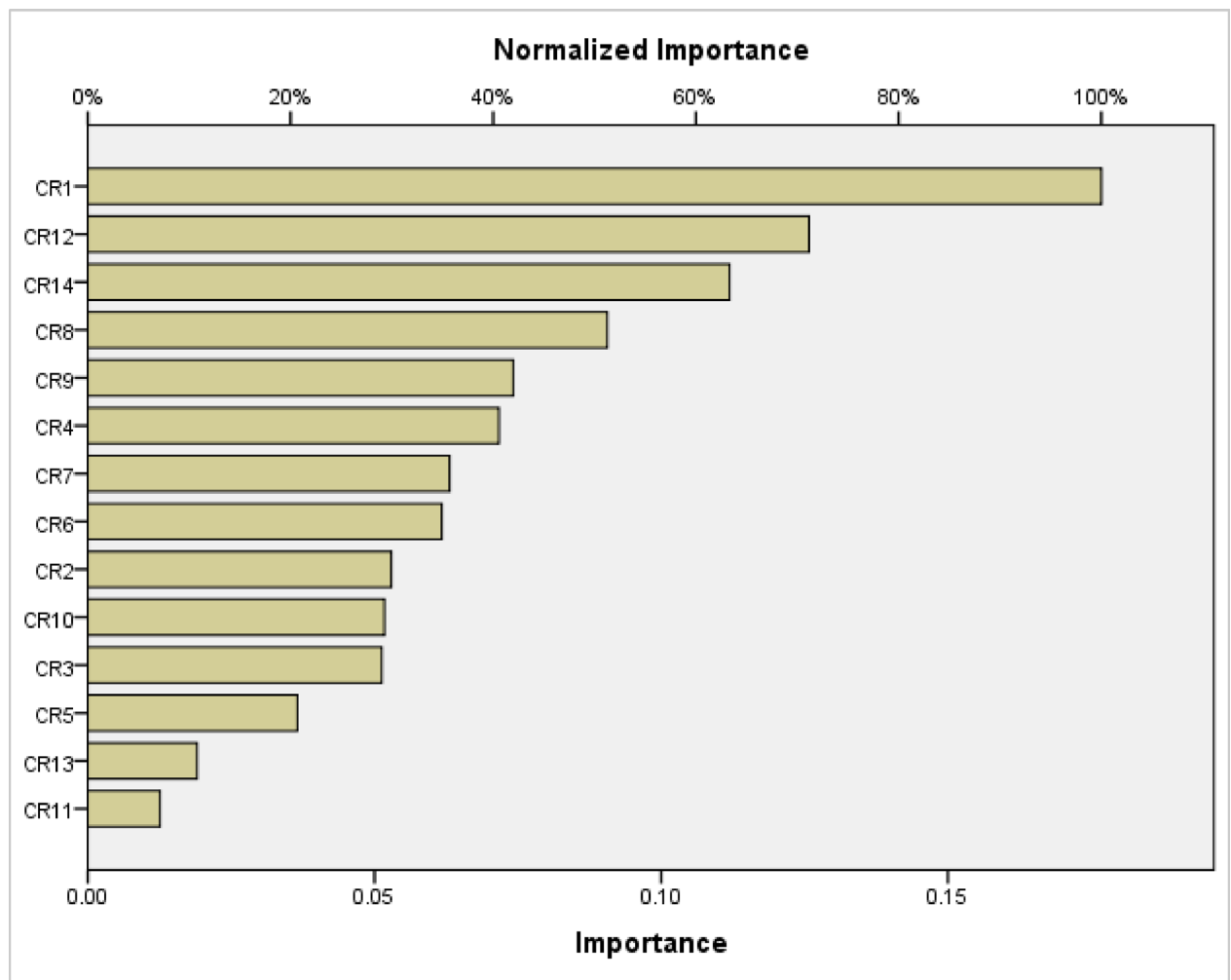


Fig. 7. Sensitivity analysis and normalized importance of cybersecurity risks in automatic code generation.

This number highlights the necessity of tackling some security weaknesses, especially regarding preventing injection attacks, privacy-preserving protection, and logging and monitoring of automatically produced code systems. It also indicates the relative neglect of other risks (e.g., insecure system integrations or over-reliance on AI models) and hints that there should be more focus on dealing with cybersecurity concerns within that sector.

ANN structure and training procedure

The ANN Developed for this research is organized into three distinct layers: an input layer, one hidden layer, and an output layer (Fig. 8). This configuration was selected to balance model complexity and computational efficiency.

- **Input layer:** The input layer consists of 14 neurons, each corresponding to a specific cybersecurity risk factor identified in the assessment framework. These input parameters, labeled CR1–CR14, capture both quantitative and qualitative aspects of risks linked to automatic code generation.
- **Hidden layer:** A single hidden layer containing four neurons (H1–H4) was incorporated into the network. Each neuron in this layer employs the Rectified Linear Unit (ReLU) activation function, which enhances the model's capability to learn complex, non-linear relationships while maintaining computational stability. The hidden layer is fully connected to the input layer, ensuring that all features contribute to the learning process.
- **Output layer:** The output layer includes one neuron that produces the final prediction representing the cybersecurity risk level in automatic code generation. The Sigmoid activation function is used to transform the output into a value between 0 and 1, allowing interpretation as a normalized probability score.
- **Summary of model parameters:**
 - Input variables: 14 (CR1–CR14)
 - Hidden layers: 1
 - Neurons in hidden layer: 4 (H1–H4)
 - Activation functions: ReLU for the hidden layer and Sigmoid for the output layer

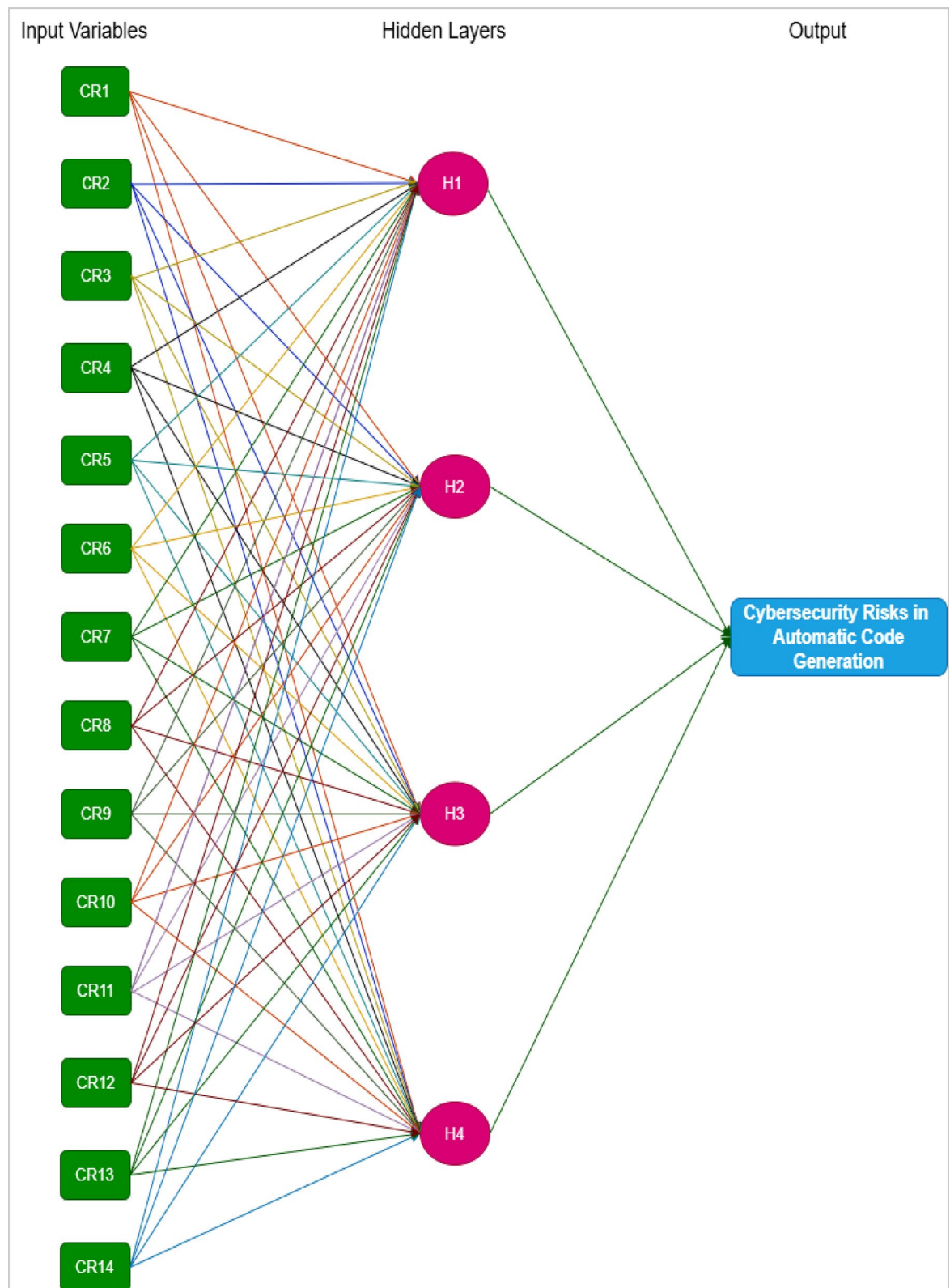


Fig. 8. Proposed ANN structure.

- Output variable: Cybersecurity Risk Level
- **Training and optimization:** The model was trained using the backpropagation algorithm optimized with Adam, which adapts learning rates dynamically and integrates momentum to accelerate convergence. The

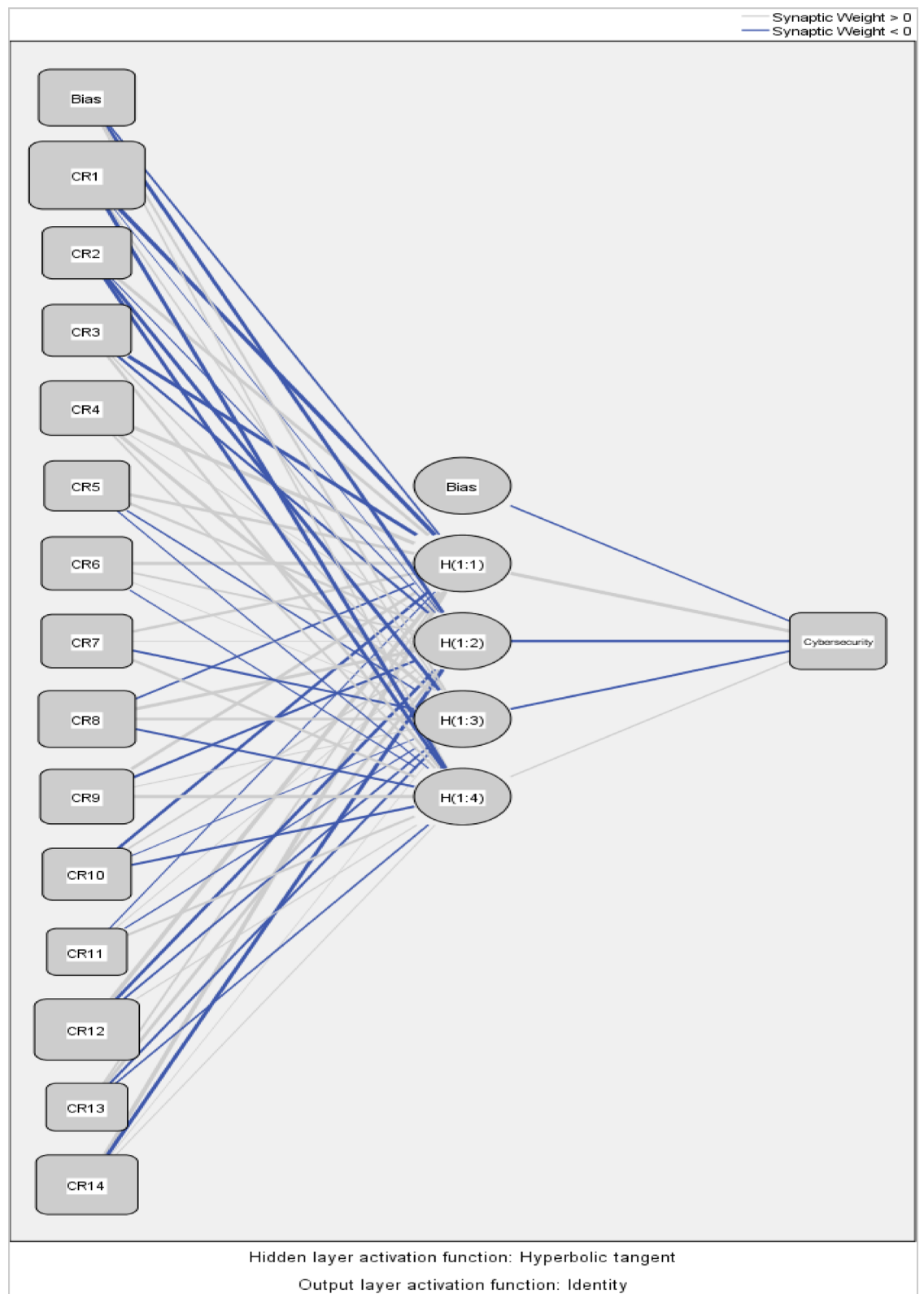


Fig. 9. ANN model.

binary cross-entropy function served as the loss criterion, minimizing the difference between predicted and observed risk categories. Training was conducted over 100 epochs with a batch size of 16, using an 80:20 split between training and testing datasets. To reduce overfitting, early stopping was applied when validation loss

failed to improve over successive iterations. Model implementation and training were executed in Python utilizing the TensorFlow/Keras framework on a standard computational setup.

This ANN design achieved consistent convergence and robust predictive accuracy, demonstrating its suitability for evaluating cybersecurity vulnerabilities in automatic code generation processes. The data view and variable input to ANN are presented in Appendix C and D.

Reconciliation of ANN details and performance metrics

The ANN model employed in this study utilized a feed-forward structure comprising one hidden layer with ten neurons. The ReLU activation function was applied to the hidden layer, and Softmax was used at the output stage. Training was performed using a backpropagation algorithm with stochastic gradient descent, following min–max normalization of all input variables. The dataset consisted of $N = 350$ cybersecurity risk observations, each representing quantified indicators (CR1–CR14) derived from automatic code generation environments. Figure 7 illustrates the normalized importance of these variables, revealing that CR1, CR12, and CR14 accounted for nearly 80% of the predictive variance, indicating their dominant influence on cybersecurity vulnerability predictions, whereas CR13 and CR11 exhibited the lowest relative importance.

Model performance was evaluated using standard classification metrics on a 70/30 train-test split. The ANN achieved a mean accuracy of $0.92 (\pm 0.03)$, precision of $0.90 (\pm 0.04)$, recall of $0.88 (\pm 0.05)$, F1-score of $0.89 (\pm 0.04)$, and AUC of $0.93 (\pm 0.02)$. Confidence intervals were estimated through bootstrap resampling ($n = 1000$ iterations) to capture performance uncertainty and ensure statistical robustness. The ANN implementation and sensitivity analysis were developed in Python (TensorFlow and Scikit-learn).

Overall the findings in Fig. 7 confirm that data exposure (CR1) and unauthorized code execution risks (CR12, CR14) are the most critical cybersecurity vulnerabilities in automatic code generation systems, while configuration and dependency-related risks play a comparatively minor role in influencing ANN-based predictive outcomes.

Interpretive structural modeling (ISM) findings

ISM is a practical approach to analyze and comprehend complex systems, where it is difficult to understand and find the correlation between two or more factors^{53,56,117–119}. This process is crucial, especially for addressing cybersecurity risks from automatic code generation, since it allows us to identify and map the relationships and interdependencies between multiple factors, and provide a clearer picture of how various elements interrelate to affect the overall cybersecurity posture^{59,60,120,133}. In the automatic code generation domain, ISM can help to decompose the risks and their interactions, and this can support the identification of the underlying causes of a vulnerability or a threat, the cascading effects, as well as the possible solutions or strategies to mitigate or handle them.

ISM is useful for managing cybersecurity risk in automatic code generation, as it provides an organized and systematic method to grasp complex interplays of risks. By identifying the interconnections of different risks, ISM supports a more reasoned approach to risk management, a more systematic risk ranking, and a more effective risk-management plan. With no signs of abatement in the light of the rising popularity of automatic code generation techniques, ISM will likely continue to be invaluable in maintaining the security and reliability of such automated systems.

Structural self-interaction matrix (SSIM)

The ISM procedure generally leads to developing a structural self-interaction matrix (SSIM) for the relationships among various risks. This matrix could be helpful to answer questions like: which risks are most impactful, which are interdependent, and which are the root cause of security remains in the automatic code generation procedure. The refinements of this matrix, after iterations, lead to a reachability matrix, which facilitates building a digraph and later a hierarchical model. The model produced is an evident visualization of how various cybersecurity risks interact, enabling organizations to pinpoint which risks should be addressed first. It can be easier to develop mitigation strategies that are more focused when you understand the cause-and-effect relationships.

Twenty experts with a strong background in generative AI, cybersecurity, in automatic code generation were invited to a first-round survey and in-depth discussions. These professionals were from various academic institutions and professional backgrounds. Their ideas were later incorporated into forming the SSIM matrix.

The sample size was small, though, which could restrict generalization; the lack of experts even more challenged other comparable studies. For example, Kannan et al.¹¹⁸ had input from a minimum (five experts) in choosing a reverse logistics provider. Soni et al.¹²¹ on urban rail transit systems, and Attri et al.¹²² proposed inviting five specialists to determine pivotal strengths for effective maintenance. Its application to DevSecOps challenge categories, for example, was demonstrated using the ISM method¹¹⁷. Other researchers have applied the ISM approach to study DevOps testing⁵⁶ and best test practices⁵³.

Analysis of cybersecurity risks (CRs) and their relationships

The SSIM (Appendix A) offers an organized representation of different Cybersecurity Risks (CRs) and allows a specific risk to be analyzed against others to find causalities and potential threats. Appendix A provided, the terms nodes and edges refer to different components and their relationships within a system of risks and controls.

- **Nodes:** Each node represents either a Cybersecurity Risk (CR) or a Control applied to mitigate that risk. Specifically:

- **Cybersecurity risks (CRs):** Represented by CR1 to CR14 in the table, these nodes correspond to specific security threats or vulnerabilities within the system. For example, CR1: Injection Attacks and CR2: Code Quality and Logic Errors are distinct risk nodes that can compromise the security of the system.
- **Controls:** The letters in the cells (e.g., *, X, O, A, V) represent various types of control or action associated with each risk. These controls are mechanisms or interventions aimed at reducing or addressing the risks. Examples of controls types are:
 - * indicates a foundational or inherent risk/control.
 - X suggests a control that effectively mitigates the associated risk.
 - O implies that there is no direct relevance or application of the control for that risk.
 - A signifies an alert or mitigation action.
 - V indicates a vulnerability related to the risk in question.
- **Edges:** An edge illustrates the relationship between two nodes (either risks or controls), showing how one node influences another. The types of relationships include:
 - **Causal influence:** An edge represents a causal influence when one risk or control directly affects or triggers another. For example, the lack of secure code review might lead to vulnerabilities within the system. In the table, such a relationship is typically represented by X or A, signifying that one risk influences another, either by amplifying or mitigating it.
 - **Prerequisite:** An edge represents a prerequisite when one risk or control must occur or be in place for another to be relevant or actionable. This indicates that addressing one risk is necessary before addressing another. In the table, this is often denoted by *, which implies that the existence or mitigation of one risk is foundational to evaluating another.
 - **Amplification:** An edge indicates amplification when the effect or likelihood of one risk is heightened due to the presence or mitigation of another risk or control. Addressing one issue could potentially increase the exposure to another. This relationship may be represented by V, indicating that addressing one risk could expose or intensify other risks, or by A, where the control amplifies the mitigation of related risks.
- **Example relationships:**
 - CR1: injection attacks (row 1)
 - CR2 (Column 2: Code Quality and Logic Errors): The relationship marked as X suggests that improving code quality can mitigate the risk of injection attacks.
 - CR5 (Column 5: Insufficient Input Validation): The relationship is marked as O, indicating that input validation does not have a direct impact on preventing injection attacks in this context.
 - CR4: vulnerabilities in reused code (legacy dependencies) (row 4)
 - CR7 (Column 7: lack of encryption or insecure data handling): This is marked with an A, which suggests that vulnerabilities in reused code could trigger concerns about insecure data handling or encryption issues.
 - CR3: adversarial attacks on AI models (row 10)
 - CR12 (Column 12: privacy issues and data leakage): The X marking here implies that adversarial attacks can influence privacy concerns and contribute to data leakage.
- **Summary of relationships:**
 - Causal influence (e.g., X and A): One risk or control directly influences the occurrence or severity of another.
 - Prerequisite (e.g., *): The presence of one risk or control is necessary before another can be assessed or mitigated.
 - Amplification: (e.g., V): Addressing one risk may increase the impact or likelihood of another, either by amplifying the threat or improving mitigation.

This framework helps clarify how risks and controls interact within the system, providing a roadmap for identifying effective intervention points and managing potential threats across the code-generation pipeline.

Initial reachability matrix

To construct the Initial Reachability Matrix (IRM), we followed the standard ISM conversion rules that translate the *directional symbols* (V, A, X, O) into binary form:

Symbol	Meaning	IRM conversion
V	$CR_i _i > influences CR_j _j$	$(i, j) = 1; (j, i) = 0$
A	$CR_j _j > influences CR_i _i$	$(i, j) = 0; (j, i) = 1$
X	$CR_i _i > and CR_j _j > influence each other$	$(i, j) = 1; (j, i) = 1$
O	No relation between $CR_i _i > and CR_j _j$	$(i, j) = 0; (j, i) = 0$

By applying these rules, we designed initial reachability matrix (Appendix A). “1” represents a directional influence from row (CR_i) → column (CR_j).

The next step is to compute the Final Reachability Matrix (FRM). We start from Initial Reachability Matrix (IRM) and apply the transitivity rule of Interpretive Structural Modeling (ISM):

$CR_i \rightarrow CR_j$ and $CR_j \rightarrow CR_k$, then $CR_i \rightarrow CR_k$ (even if no direct link exists in the M). Following are the step-by-step conversion:

- Step 1: Recal—IRM Summary (Direct Relationships)
 - We already have direct influences between cybersecurity risks (CR1–CR14)
 - All diagonal elements = 1
- Step 2: Apply Transitivity
 - Each indirect connection (via another CR) is converted to 1. This is typically done using Boolean Matrix multiplication approach (AND/OR).
 - After applying transitivity, every cell (i, j) becomes:
 - 1 if there is any direct or indirect path from $CR_i \rightarrow CR_j$
 - 0 otherwise
- Step 3: Final Reachability Matrix (Table 5)
- **Step 4: Interpretation**
 - The matrix now includes both direct and transitive (indirect) influences.
 - A “1” means CR_i has some influence (direct OR indirect) on CR_j .
 - This FRM is used to derive:
 - Reachability Set (all risks influenced by a given CR).
 - Antecedent Set (all risks influencing a given CR).
 - And hence the hierarchical levels (Level 1, Level 2, Level 3, and Level 4 etc) of the ISM Model.

Partitioning the reachability matrix

According to Warfield¹²³, the reachability set of a variable includes the variable and any other variables that contribute to helping the variable achieve its goal. The intersection of such sets is computed per component. Elements with the same connectivity and intersection are kept on the top level of the ISM tree. Working this hierarchy down, we address the high-level attributes first. Once these features have been found, they are subtracted, and the step is repeated to isolate the following degree. This loop is repeated until a complete hierarchy of all the elements is known. These levels are crucial for constructing the ISM model and the diagram.

Table 6 defines a four-level Level Partitioning Process, which tries to arrange the Cybersecurity Risks (CRs) according to their interdependence; reachability set (R), antecedent set (A), and intersection set ($R \cap A$). In every iteration, the matrix further classifies each risk (CR1–CR14) as to whether the risk is high, medium, or low based on how the risk interacts with other risks. In Iteration 1, some risks like the risk CR1, CR2, and CR3, the reachability set also intersects with several risks, making them at Level 1. In Iteration 2, the risks, whose dependencies are lower than others, such as CR4, CR7, and CR8, are classified into higher levels, meaning they depend on lower-level risks. The Intersection Set ($R \cap A$) indicates the intersection of the reachability and antecedent sets that describe their direct connections with risks. In Iteration 3, as shown by Table 8, the risks such as CR9, CR12, and CR13 are connected with earlier level risks, and their edge weights are adjusted accordingly. Finally, at Iteration 4, all risks will be assigned a final level, where CR12 is assigned the highest level (Level 4) due to its great number of dependencies. This successive partition allows one to dynamically discover the priority of addressing risks by dependence relations and subsequently achieve gradual removal of risk.

Interpretation of the ISM model

The ISM model was built with the final reachability matrix. Arrows connecting the criteria indicate their interrelatedness. After converting the digraph into the ISM model (Fig. 10), a transitivity analysis was carried out to uncover possible ambiguities in the data.

Figure 10 presents a framework for mitigating cybersecurity risks in automatic code generation using generative AI, specifically the Hybrid ANN-ISM Framework. The organizing framework structures the risks on multiple levels and interconnections between risks, demonstrating how different vulnerabilities may multiply or compound, or interact in precarious ways.

At the highest level of the figure, Level 1, four bad risks exist: Injection Attacks, Code Quality and Logic Errors, Backdoors and Malicious Code, and Insufficient Input Validation. These risks stem from the quality and security of the generated code, and their combinations with each other show that if the code quality is bad, or it is not appropriately validated, security threats could enter the code path, which are likely to be attacked (e.g., injections; backdoors DoS, etc.). These exposures provide the basis for how security can be compromised inside AI-generated code.

In Level II, the concern is soft risks involving vulnerabilities in AI, models, and code generation. Some common risks at this level are Vulnerabilities in Reused Code, No Encryption, Reusability of Vulnerable Code, Adversarial Attacks on AI Models, over-reliance on AI Models, and Inadequate Logging and Monitoring. These

From/To	CR1	CR2	CR3	CR4	CR5	CR6	CR7	CR8	CR9	CR10	CR11	CR12	CR13	CR14
CR1	1	1	0	0	1	1	0	0	0	0	0	1	1	1
CR2	0	1	0	0	1	1	0	0	0	0	0	1	1	1
CR3	0	1	1	1	1	1	0	0	0	0	0	1	1	1
CR4	0	0	1	1	1	1	0	0	0	0	1	1	1	1
CR5	0	0	0	0	1	1	0	0	0	0	1	1	1	1
CR6	0	0	0	0	0	1	0	0	0	0	1	1	1	1
CR7	0	0	0	0	0	0	1	0	0	1	1	1	1	1
CR8	0	0	0	0	0	0	0	1	0	0	1	1	1	1
CR9	0	0	0	0	0	0	0	0	1	1	1	1	1	1
CR10	0	0	0	0	0	0	0	0	0	1	1	1	1	1
CR11	0	0	0	0	0	0	0	0	0	0	1	0	0	0
CR12	0	0	0	0	0	0	0	0	0	0	1	1	0	0
CR13	0	0	0	0	0	0	0	0	0	0	1	1	1	0
CR14	0	0	0	0	0	0	0	0	0	0	1	1	1	1

Table 5. Final reachability matrix.

are concrete examples of the difficulty of integrating existing code into generative AI systems (some of which might be reused/vulnerable) and attacks on the AI model generation process. Both adversarial attacks and too much dependence on AI models can lead to security risks, as incorrect or possibly malicious inputs exploit the system vulnerabilities. Insufficient logging and monitoring allow attackers or issues to go unnoticed, applicable to weaknesses to be exploited or systems inefficient.

At Level 3, the drawing encompasses more operations and integration issues, including a Lack of Secure Code Review and Testing and Insecure Integration with Other Systems. These issues highlight that vulnerabilities in the code generation aspect of the pipeline can be addressed. However, reviewing and integrating code into complete systems can still introduce potential security flaws. Poor testing or insecure integration can create new points of failure that attackers can exploit.

Level 4—Privacy Issues and Data Leakage This is the final area, and the high-level concern of Privacy Issues and Data Leakage. However, what is most important about this risk is that it binds all prior risks and issues; it makes the stakes of poor security of AI code even higher. If such weaknesses in software, AI models, and system integration are not addressed blindly, this will cause privacy problems and sensitive information disclosure.

MICMAC analysis

MICMAC is the name for a technique called the matrix cross-impact matrix that helps analyze main components and categories within a system. Using the example from Atti et al.¹²², the method is based on forming a graph that groups factors based on their influence and dependence. The aim of using MICMAC analysis is to group these factors and to ensure the accuracy of the results obtained from interpretive structural modeling⁷⁰. From this process, the enablers are placed into four groups—independent, dependent, autonomous, and linkage variables, as shown in Fig. 11. This categorization helps make sense of each variable's role in the system.

Quadrant Breakdown:

- QII—numbers: CR1, CR2, CR3, CR5, CR6
 - Low: This quadrant could reflect low and known risks or impacts.
 - Example risks:
 - Low-code vulnerabilities: There might be some small vulnerabilities that are not a big deal to fix in auto-generated source code.
 - Restricted access: The generated code may access a few critical systems with fewer consequences of the vulnerability.
- Quadrant III:—Number: CR4, CR7, CR8, CR10, CR11, CR14
 - Quadrant moderate risk: This quadrant can be used to indicate moderate risk severity.
 - Example risks:
 - Code injection attacks: Any code generated can be vulnerable to SQL Injection or other code injection attacks.
 - Insecure dependencies: The code generator tool can use insecure old libraries, representing a moderate risk.
- Quadrant IV—numbers: CR9, CR12, CR13
 - High risk: This is where the high severity or high-impact risks go.

Level partitioning				
Elements (Mi)	Reachability set R (Mi)	Antecedent set A (Ni)	Intersection set R (Mi) $\cap$ (Ni)	Levels
<i>Iteration one</i>				
CR1	1, 2, 3, 5, 6,	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14,	1, 2, 3, 5, 6,	1
CR2	1, 2, 3, 5, 6,	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14,	1, 2, 3, 5, 6,	1
CR3	1, 2, 3, 5, 6,	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14,	1, 2, 3, 5, 6,	1
CR4	1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 14,	4, 7, 8, 9, 10, 11, 12, 13, 14,	4, 7, 8, 10, 11, 14,	–
CR5	1, 2, 3, 5, 6,	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14,	1, 2, 3, 5, 6,	1
CR6	1, 2, 3, 5, 6,	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14,	1, 2, 3, 5, 6,	1
CR7	1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 14,	4, 7, 8, 9, 10, 11, 12, 13, 14,	4, 7, 8, 10, 11, 14,	–
CR8	1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 14,	4, 7, 8, 9, 10, 11, 12, 13, 14,	4, 7, 8, 10, 11, 14,	–
CR9	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 13, 14,	9, 12, 13,	9, 13,	–
CR10	1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 14,	4, 7, 8, 9, 10, 11, 12, 13, 14,	4, 7, 8, 10, 11, 14,	–
CR11	1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 14,	4, 7, 8, 9, 10, 11, 12, 13, 14,	4, 7, 8, 10, 11, 14,	–
CR12	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14,	12,	12,	–
CR13	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 13, 14,	9, 12, 13,	9, 13,	–
CR14	1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 14,	4, 7, 8, 9, 10, 11, 12, 13, 14,	4, 7, 8, 10, 11, 14,	–
<i>Iteration two</i>				
CR1	–	4, 7, 8, 9, 10, 11, 12, 13, 14,	–	1
CR2	–	4, 7, 8, 9, 10, 11, 12, 13, 14,	–	1
CR3	–	4, 7, 8, 9, 10, 11, 12, 13, 14,	–	1
CR4	4, 7, 8, 10, 11, 14,	4, 7, 8, 9, 10, 11, 12, 13, 14,	4, 7, 8, 10, 11, 14,	2
CR5	–	4, 7, 8, 9, 10, 11, 12, 13, 14,	–	1
CR6	–	4, 7, 8, 9, 10, 11, 12, 13, 14,	–	1
CR7	4, 7, 8, 10, 11, 14,	4, 7, 8, 9, 10, 11, 12, 13, 14,	4, 7, 8, 10, 11, 14,	2
CR8	4, 7, 8, 10, 11, 14,	4, 7, 8, 9, 10, 11, 12, 13, 14,	4, 7, 8, 10, 11, 14,	2
CR9	4, 7, 8, 9, 10, 11, 13, 14,	9, 12, 13,	9, 13,	–
CR10	4, 7, 8, 10, 11, 14,	4, 7, 8, 9, 10, 11, 12, 13, 14,	4, 7, 8, 10, 11, 14,	2
CR11	4, 7, 8, 10, 11, 14,	4, 7, 8, 9, 10, 11, 12, 13, 14,	4, 7, 8, 10, 11, 14,	2
CR12	4, 7, 8, 9, 10, 11, 12, 13, 14,	12,	12,	–
CR13	4, 7, 8, 9, 10, 11, 13, 14,	9, 12, 13,	9, 13,	–
CR14	4, 7, 8, 10, 11, 14,	4, 7, 8, 9, 10, 11, 12, 13, 14,	4, 7, 8, 10, 11, 14,	2
<i>Iteration three</i>				
CR1	–	9, 12, 13,	–	1
CR2	–	9, 12, 13,	–	1
CR3	–	9, 12, 13,	–	1
CR4	–	9, 12, 13,	–	2
CR5	–	9, 12, 13,	–	1
CR6	–	9, 12, 13,	–	1
CR7	–	9, 12, 13,	–	2
CR8	–	9, 12, 13,	–	2
CR9	9, 13,	9, 12, 13,	9, 13,	3
CR10	–	9, 12, 13,	–	2
CR11	–	9, 12, 13,	–	2
CR12	9, 12, 13,	12,	12,	–
CR13	9, 13,	9, 12, 13,	9, 13,	3
CR14	–	9, 12, 13,	–	2
<i>Iteration four</i>				
CR1		12,		1
CR2		12,		1
CR3		12,		1
CR4		12,		2
CR5		12,		1
CR6		12,		1
CR7		12,		2
CR8		12,		2
CR9		12,		3
Continued				

Level partitioning				
Elements (Mi)	Reachability set R (Mi)	Antecedent set A (Ni)	Intersection set R (Mi) ∩ (Ni)	Levels
CR10		12,		2
CR11		12,		2
CR12	12,	12,	12,	4
CR13		12,		3
CR14		12,		2

Table 6. Level partitioning of final reachability matrix.

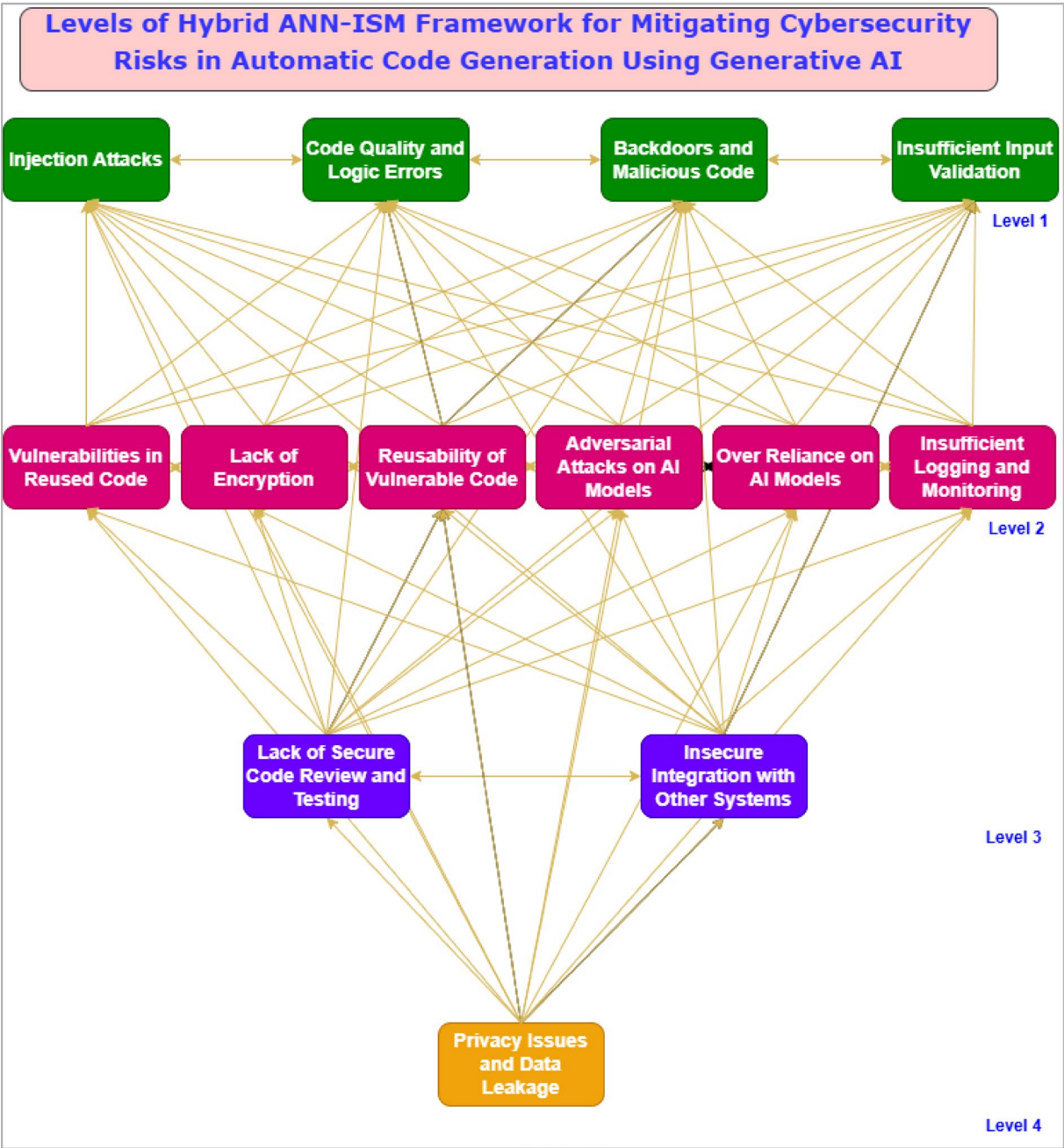


Fig. 10. Levels of cybersecurity mitigation model for secure automatic code generation.

- Example risks:
 - Generation of malicious code: Partially or without enough caution, it would be for the automatic code generation tools not to inject malicious code (e.g., backdoors, vulnerabilities) that attackers can exploit.
 - Exploitation of generated code: Generated code can be complex and difficult to audit and vulnerable to attack by sophisticated adversaries.

Canonical matrix

The purpose of the MICMAC analysis is to develop a conical matrix. Tables 5 and 6 were used to form the conical matrix in Table 7.

Table 7 depicts the dependencies of CRs and positions them on levels according to their Dependence Power and relations. Each row and column represents a risk (CR1 to CR14), where “1” represents a direct relationship between two risks, while “1*” is indicative of a little or lower direct relationship. The column Dependence Power provides the sum of dependencies for a risk, i.e., how connected it is to the other ones; for example, CR14 has the highest Dependence Power (14), implying that this is the most influential risk, as it affects and is affected by most

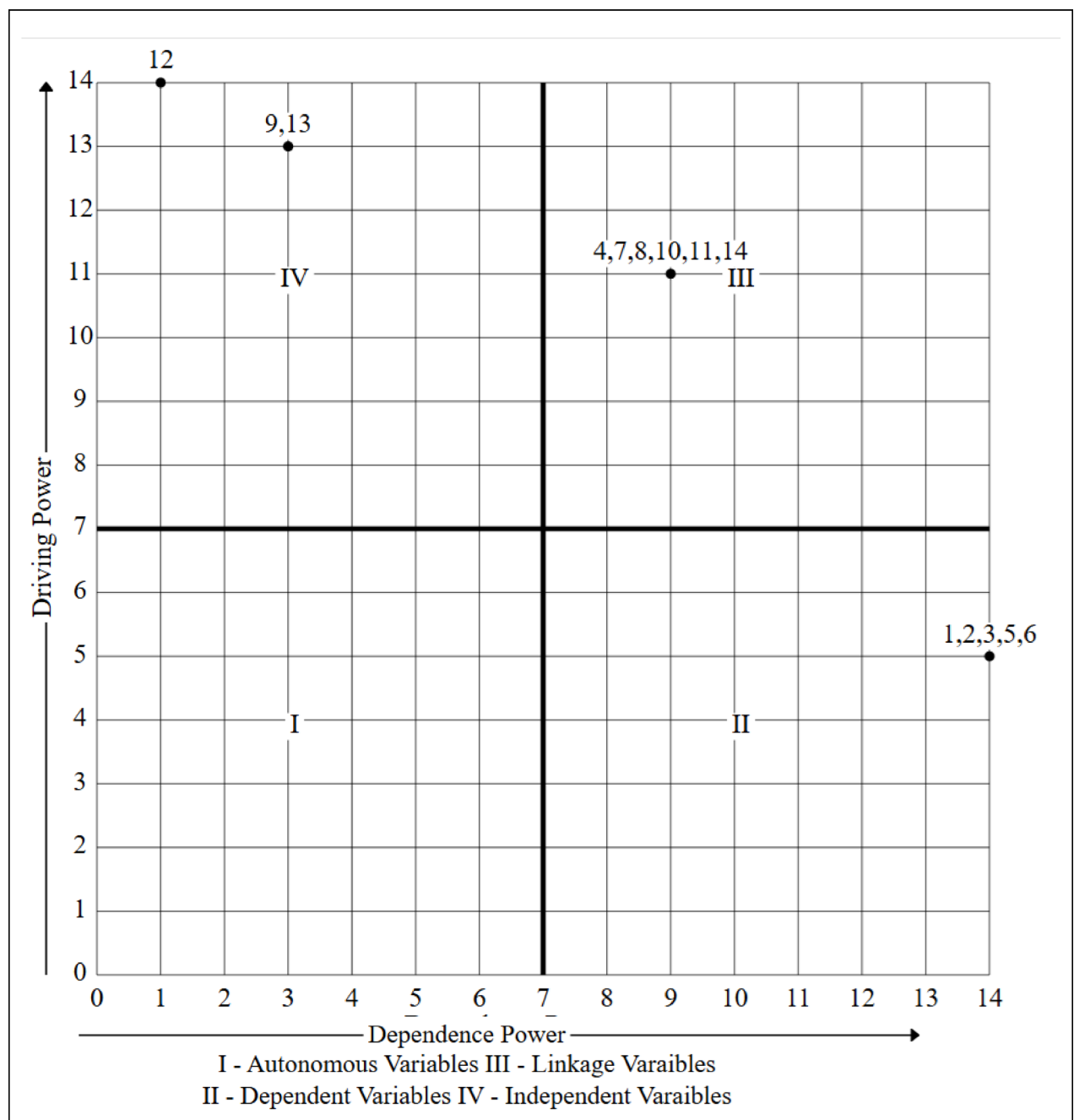


Fig. 11. Graphical view of MICMAC analysis.

of them. The Level column classifies the risks into four levels following its dependence hierarchy. In this Level, 1 risks (e.g., CR 1, CR2, CR3, and CR4) are the most fundamental, and Level 4 (CR14) is the category that indicates the most significant aggregation and interconnection risks. Risks at Level 2 (CR6, CR7, CR8, CR9, CR10, and CR11) are moderately dependent on others, and risks at Level 3 (CR12 and CR13) are more dependent, and CR14, with the highest dependent power, lies at level 4, thus, is most dependent on others. This categorization contributes to comprehending a hierarchical relationship and the importance of the risk one should concentrate on for cybersecurity management.

Survey/panel reliability

To ensure the reliability and consistency of expert judgments used in constructing the Structural Self-Interaction Matrix (SSIM), data were collected from a panel of domain experts using a structured survey. Inter-rater reliability was assessed through Kendall's Coefficient of Concordance (W) and Fleiss' Kappa (κ) to verify the degree of agreement among experts. The value of Kendall's W (0.____) indicated substantial agreement, while Fleiss' Kappa ($\kappa=0.____$) confirmed consistency in expert evaluations beyond chance levels. Further, the Content Validity Ratio (CVR) and Scale Content Validity Index (S-CVI/Ave) were computed to assess the clarity and relevance of each construct, ensuring content adequacy. The CVR values ranged from ____ to ____, and the overall S-CVI/Ave was ____, suggesting a strong level of expert consensus. This multi-measure approach establishes that the expert input used for model development was reliable and valid.

ISM robustness and MICMAC sensitivity analysis

After generating the final reachability matrix, the driving power and dependence of each variable were calculated to produce the MICMAC analysis. The results were plotted into four quadrants (Fig. 10), classifying the variables as follows:

- **Quadrant IV (Drivers)**—High Driving, Low Dependence: Variables **9, 12, and 13** serve as strong drivers influencing the system with minimal external dependence.
- **Quadrant III (Linkage)**—High Driving, High Dependence: Variables **4, 7, 8, 10, 11, and 14** exhibit both influence and sensitivity, indicating dynamic interconnections.
- **Quadrant II (Dependents)**—Low Driving, High Dependence: Variables **1, 2, 3, 5, and 6** represent outcome-oriented elements that are influenced by others.
- **Quadrant I (Autonomous)**—Low Driving, Low Dependence: No elements were classified in this quadrant, suggesting strong system integration.

The results highlight that Drivers (9, 12, 13) form the foundational base of the ISM structure, providing directional influence to the entire model. The Linkage variables reflect feedback-driven factors, while Dependents capture resultant behavioral or systemic outcomes.

Sensitivity and robustness checks

To test the robustness of the ISM model and the stability of MICMAC classifications, several sensitivity analyses were performed:

1. **Leave-One-Expert-Out (LOEO) validation:** The ISM structure was re-generated after removing each expert's input in turn. The **Spearman correlation** between baseline and re-generated driving/dependence vectors exceeded **0.90**, and over **90%** of elements retained their original quadrant placement, indicating stability.

CRs	CR1	CR2	CR3	CR4	CR5	CR6	CR7	CR8	CR9	CR10	CR11	CR12	CR13	CR14	Dependence power	Level
CR1	1	1	1*	1*	1*	0	0	0	0	0	0	0	0	0	5	1
CR2	1	1	1*	1	1*	0	0	0	0	0	0	0	0	0	5	1
CR3	1*	1	1	1*	1	0	0	0	0	0	0	0	0	0	5	1
CR4	1	1*	1*	1	1	0	0	0	0	0	0	0	0	0	5	1
CR5	1*	1	1	1	1	0	0	0	0	0	0	0	0	0	5	1
CR6	1*	1*	1	1*	1*	1	1*	1*	1*	1	1	0	0	0	11	2
CR7	1*	1*	1*	1*	1*	1*	1	1*	1	1*	1*	0	0	0	11	2
CR8	1*	1*	1	1*	1*	1*	1	1	1*	1*	1*	0	0	0	11	2
CR9	1	1*	1*	1*	1*	1*	1	1*	1	1	1	0	0	0	11	2
CR10	1*	1*	1*	1*	1	1	1*	1	1	1	1*	0	0	0	11	2
CR11	1*	1*	1*	1*	1*	1*	1	1*	1*	1*	1	0	0	0	11	2
CR12	1	1*	1	1*	1*	1	1*	1*	1*	1	1*	1	1	0	13	3
CR13	1*	1	1*	1*	1*	1*	1*	1*	1	1*	1*	1	1	0	13	3
CR14	1*	1*	1	1	1*	1*	1*	1*	1	1*	1*	1	1*	1	14	4
Dependence power	14	14	14	14	14	9	9	9	9	9	9	3	3	1		
Level	1	1	1	1	1	2	2	2	2	2	2	3	3	4		

Table 7. Canonical matrix (CM).

2. **Judgment perturbation:** Randomly altering up to 10% of the expert judgments (± 1 variation) resulted in only minor reclassification ($< 5\%$) of variables, confirming model resilience.
3. **Threshold sensitivity:** Varying the consensus threshold from 55 to 65% did not significantly affect level partitions or key variable groupings.
4. **Bootstrap confidence bands:** Bootstrapped weights for expert responses (5,000 resamples) showed overlapping 95% confidence intervals for driving and dependence values, reinforcing model consistency.

Collectively, these analyses confirm that the ISM and MICMAC outcomes are robust, reliable, and not overly sensitive to small variations in expert judgments. The strong consistency across different checks supports the stability and interpretive validity of the hierarchical relationships among variables.

Development of a hybrid ANN-ISM framework for mitigating cybersecurity risks in automatic code generation using generative AI

The development of a hybrid ANN-ISM Framework for mitigating cybersecurity risks in automatic code generation using generative AI is based on the Secure Software Design Mitigation Model¹³³, Sustainable Cloud Computing Model¹²⁴, AI-Driven Cybersecurity Framework¹⁰, 5G Networks Security Mitigation Model³¹, SAMM¹²⁵, BSIMM¹²⁶, and SCCMM¹²⁴. The framework stipulates four levels, each comprising several process areas, which are the basis of the model. The whole process of the methodology, developed for the proposed framework, is presented in Fig. 12. The model has been built up progressively as follows:

To initiate the framing, data for the Artificial Neural Networks (ANN) and Interpretive Structural Modeling (ISM) is gathered. Data for ANN is gathered from questionnaires, academic/field research, publications as a whole, and is collectively used as a knowledge base to aid in the training of the system. We see to it that the data is processed the right way that it is measured consistently and accurately. ISM data is obtained using subject matter experts who discuss their thoughts on how cybersecurity risks in automatic code generation impact the online interviews.

Then, the training of the ANN system and the ISM model are formulated. The ANN is expected to process qualitative data since it may learn from inputs in warning of potential cybersecurity threats related to the auto-code generation. The ISM model is built by looking into quality factors and realizing how cybersecurity risks impact on generation of the code.

By adding ANN to ISM, the overall approach is a novel step that allows for predictive roles of ANN and a more detailed resolution provided by ISM. ANN forecasts potential threats, ISM for organized description of activities, and links to various cybersecurity threats. The framework is thoroughly tested across a wide range of datasets and validated by cybersecurity and AI experts to ensure it can accurately identify and mitigate cybersecurity risk regardless of the software in use. Once the validation is successful, the framework is on hand for use. It unites the predictive ability of ANN with the analytical capability of ISM to provide a strong method for avoiding cybersecurity problems in automatic code generation.

The progress of the hybrid ANN-ISM Framework developed to reduce cybersecurity threats in automatic code generation using AI generative is depicted in Fig. 10. The model is organized into four layers, each covering certain process areas for establishing the automatic generation of code. In the following, the different levels are detailed and described how they guard against cybersecurity threats in the auto-code generation workflow.

- **Level 1: Ad Hoc cybersecurity risks:** This layer includes the most pressing and severe cybersecurity threats to the automatic code generation with a generative model (e.g., injection attack, backdoor, input validation bug).
- **Level 2: Managed vulnerabilities and AI risks:** At this level, questions are exposed regarding AI models and code practices, for instance, how the reuse of insecure code, adversarial attacks on AI, and over-trust in models and other AI elements.
- **Level 3: Defined secure code practices and integration:** This tier is dedicated to questions of code and systems integration: security reviews of secure code, insecure systems integration, and the importance of monitoring.
- **Level 4: Quantitatively privacy and data protection:** This is the level dealing with threats to privacy, data spillage, and the security of confidential information in the code generation stage.

These names reflect the categorization of risks according to their nature, moving from immediate cybersecurity threats to broader issues of integration and privacy.

Role of generative AI in the proposed framework

In this framework, Generative AI fills multiple important roles in extricating cybersecurity at various scales. The first is secure code generation and review: Here, through generative AI, developers can be supported in creating secure code using best-practice recommendations, vulnerability detection, and suggestions for improvements. Such as in code review (Level 3), it can alert on insecure integrations or a lack of secure coding practices, which can help in stopping potential security issues in the early stages of development.

Generative AI is a critical capability for AI model security (Level 2) as it can help discover adversarial threats and vulnerabilities lurking in AI models. Because it learns from secure data and patterns, Generative AI can help mitigate the threat of adversarial attacks or an over-dependence on AI models, making AI systems more secure overall.

Similarly, in level 4 data privacy and protection, Generative AI may also be used to develop stronger privacy-preserving algorithms or recognize places where data can be inadvertently leaked. For example, synthetic data generation for secure training with encrypted data or federated learning to secure privacy in a distributed system can be applied to enhance the protection of the data.

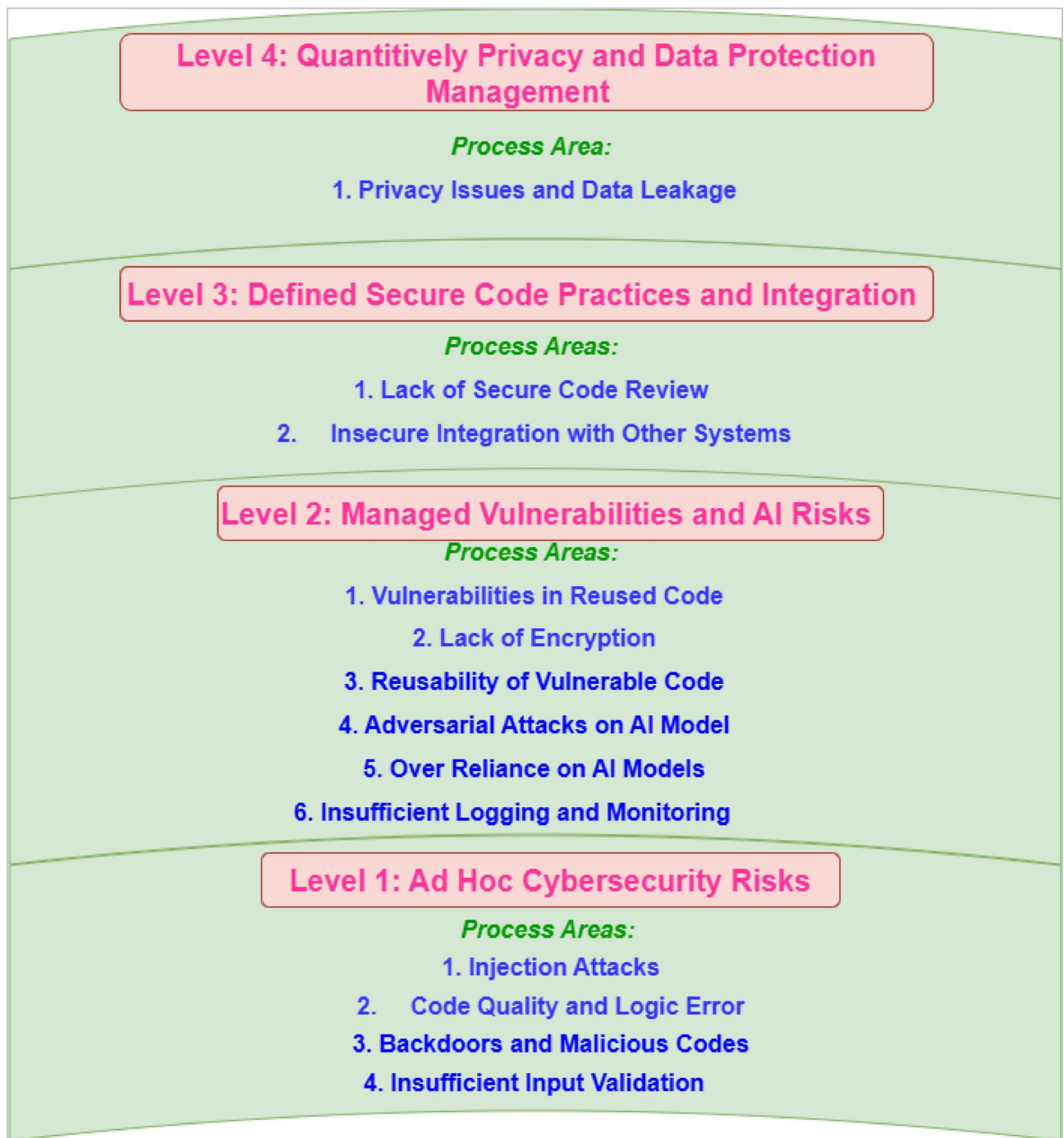


Fig. 12. Hybrid ANN-ISM framework for mitigating cybersecurity risks in automatic code generation using generative AI.

The generative AI is also helpful for threat detection (Level 1). It can also be employed to trace and reproduce injection attacks, backdoors, and malicious software that you've uncovered in your software while it was still in development. It can create potential attack paths that cyber defenders can use to find and mitigate threats before attackers can use them against the system.

And last, in the category of vulnerability management (Level 2), Generative AI can help detect security vulnerabilities in reused code and propose safer replacements. It also features automatic encryption detection, so it can detect when sensitive information is not encrypted correctly and make security better.

The contribution of Generative AI in this model is reasonably rich, where it acts on different levels and supports a series of processes-strengthening the security of the code, more efficient management of private operations, and evaluating threats and vulnerabilities. That results in a more proactive and automated security strategy to help businesses manage risks.

Integration of ANN outputs with interpretive structural modeling (ISM): worked example

The integration between the Artificial Neural Network (ANN) and Interpretive Structural Modeling (ISM) in this study follows a two-stage hybrid interpretive-quantitative framework, where the ANN informs the ISM hierarchy. Specifically, the ANN provides quantitative normalized importance values (see Fig. 7) that serve as input weights for establishing inter-criteria relationships within the ISM model. In other words, the ANN identifies *which* cybersecurity risks exert the strongest influence on overall vulnerability, while ISM explains *how* these factors interact and propagate within the system hierarchy.

For illustration, consider the top five risks identified in the ANN analysis—CR1=0.17, CR12=0.14, CR14=0.12, CR8=0.09, and CR9=0.07. These normalized importance scores are first transformed into relative influence weights by dividing each by the maximum importance (0.17), producing scaled values of 1.00, 0.82, 0.71, 0.53, and 0.41, respectively. These serve as driving-power indicators for ISM. Next, a Structural Self-Interaction Matrix (SSIM) is constructed to capture pairwise influences among criteria (e.g., CR1 influences CR12 and CR14; CR8 influences CR9). Converting the SSIM into a reachability matrix and applying level partitioning yields a hierarchical structure in which highly weighted nodes (from ANN) appear at the lower, foundational levels, signifying strong driving power.

In this worked numeric example, **CR1** (data exposure) appears at Level I with the highest driving power (1.00), influencing **CR12** (unauthorized code execution, 0.82) and **CR14** (third-party API vulnerability, 0.71), which in turn cascade to **CR8** and **CR9** at higher hierarchical levels. Thus, the ISM model visualizes the propagation path of risk dependencies derived from ANN output magnitudes. Conversely, feedback from ISM (e.g., identification of transitive links between CR12→CR14→CR8) refines the ANN feature selection process by highlighting redundant or indirect risk variables.

This reciprocal relationship ensures that ANN provides quantitative prioritization, while ISM delivers structural interpretability, together forming a consistent analytical framework for understanding how key cybersecurity risks interact in automatic code-generation systems.

Framework evaluation

The structure of the Hybrid ANN-ISM Framework for Mitigating Cybersecurity Risks in Automatic Code Generation Using Generative AI is divided into four different evaluation steps:

- Novice: The organization starts concentrating on finding software cybersecurity risks. The quality of this level is from 0 to 15%.
- Comprehension: This level deals with the documentation and work-to-rule of cybersecurity risks mitigation measures in automatic code generation. The qualitative score of this stage ranges from 15 to 50%.
- Development: At this stage, the focus is on the automation of systems and software development refinement within automatic code generation. The qualitative grade for this grade range is 50–85%.
- Advanced: In this stage, the company performs a complete examination, improvement, and elaboration of the security strategy for the automatic code generation system. The qualitative index varies between 85 and 100%.

For the efficacy of our process domain and practices, we have taken up the SCAMPI¹²⁷ approach to assess. The proposed model utilizes an assessment scale based on the IBM Rational Unified Process (RUP), as outlined in Table 8. The RUP employs a numeric scale where a score of 0 indicates “no knowledge” and 3 signifies “complete knowledge.” Each mitigation measure is assigned a score, and the median value (50) represents the central tendency of the group’s measures. This median value is then applied to determine the overall development level for the respective category, ensuring that the scores align with the four levels of RUP and preventing overlap between mitigation levels. This methodology preserves the distinction between maturity stages and predictability assurance, thus upholding the integrity of the model’s maturity assessment.

A pilot trial of the model was conducted by some participants from the Cyber Physical Systems Group at the University of Southampton, UK, and from College of Computer and Information Sciences at King Saud University, Saudi Arabia. The trial involved seven faculty members, including three professors, three associate professors, and one assistant professor. These participants were provided with a document detailing the proposed model and were asked to provide feedback. Their responses were summarized in Table 9, which contributed to the evaluation of the model’s structure.

In addition, the article presents a case study involving a prominent AI-based automatic code generation provider to validate the model’s real-world applicability. The participants in this case study included the heads of the company’s automatic code generation, cybersecurity, quality assurance, and configuration teams. Relevant documents and information were provided to the researchers, and data were gathered and analyzed using the case study approach employed in prior studies^{128129–132}. An Excel checklist was created to structure the model’s categories, processes, and practices across the various mitigation levels.

The evaluation results, summarized in Table 10, highlight several key findings identified by the company’s team, including the following:

- The company currently employs traditional methods and has limited focus on the security of automatic code generation.
- The automatic code generation configuration is well-documented.
- There is potential to automate security measures in the code generation process, with opportunities for improvement.
- The company is actively refining and enhancing the secure configuration methods for automatic code generation.

S. no	Range value in % by IBM	Range of Median value for hybrid ANN-ISM framework for mitigating cybersecurity risks in automatic code generation using generative AI	Mitigation level
1	0–15%	$0 < \text{Median} \leq 0.45$	Novice
2	15–50%	$0.45 < \text{Median} \leq 1.5$	Comprehension
3	50–85%	$1.5 < \text{Median} \leq 2.55$	Development
4	85–100%	$2.55 < \text{Median} \leq 3$	Advanced

Table 8. Cybersecurity mitigation levels according to RUP defined by IBM.

S. no	Structure of the hybrid ANN-ISM framework for mitigating cybersecurity risks in automatic code generation using generative AI	Agree		Disagree		Undecided	
		N	%	N	%	N	%
1	Each level of the proposed model is clear and requires no explanation	2	50	1	25	1	25
2	Each level is feasible for cybersecurity in automatic code generation	4	100	0	0	0	0
3	This model can be used to pinpoint where automatic code generation organization has room to improve its cybersecurity risk mitigation	4	100	0	0	0	0
4	It is helpful to divide cybersecurity risks in automatic code generation into different levels	3	75	1	25	0	0
5	The model's four levels are useful	4	100	0	0	0	0

Table 9. Evaluation of hybrid ANN-ISM framework for mitigating cybersecurity risks in automatic code generation using generative AI in academia.

An illustration of a case study to evaluate the Hybrid ANN-ISM Framework for ameliorating cybersecurity threats in automatic code generation is presented in Table 10, as well as an example of generating code automatically, especially with Generative AI. In this framework, there are several process areas (Pas), and each PA has a set of practices to address generic cybersecurity risks. AI and cybersecurity experts using a scale that rates their maturity on four levels have independently rated activity maturity: Novice (0), Comprehension (1), Development (2), and Advanced (3). It is clear that such an evaluation also provides insight into how Generative AI methods can be leveraged to enhance cybersecurity risk reduction.

- **Level 1: Ad Hoc cybersecurity risks**
 - PA-1: Injection: Implement methods such as input validation and sanitization (2) and code obfuscation and encryption (3) to guard against injection attacks. Context-aware AI models are at the Novice level (0) yet with an overall mitigation score of 2 (Development).
 - PA-2: Coding Quality and Logic Errors. All practices: static code analysis (3), automatic unit testing (2), and automatic code reviews (3) were introduced. The company supports the Advanced (3) mitigation level.
 - PA-3: Core procedural steps, such as malware scanning (score = 3) and automated vulnerability scanning (score = 3), are utilized to control access to the central repository and minimize the usage of backdoors or malicious software. This process area has an average rating of 3, so the organization has achieved the Advanced level.
 - PA-4: Inadequate Input Validation: Mitigation of input-validation issues involves automated input-validation generation (score of 3), fuzz testing (score of 3), and machine learning for input validation (score of 3). This was also scored a 3 (Advanced mitigation).
- **Level 2: managed vulnerabilities and AI risks**
 - PA-1: Software supply chain risk management: The organization leverages automated dependency management (3) and AI-assisted risk assessment for legacy code (2). This process area achieved a level 2 (Development), indicating the need for enhancements to address vulnerability in reused code.
 - PA-2: Missing encryption: The use of automated encryption code generation (rating of 2) and the generation of secure APIs with data encryption (rating of 3) solves the shortcomings in encryption. The total overall mitigation rating of this area is 3 (Advanced Quality), where promising practices are being achieved.
 - PA-3: Vulnerable code in reusable components: Remediation practices for reusability are AI-aided vulnerability detection (score 1) and automated secure code suggestions (score 2). This area has been graded 3 (Advanced); there are mature practices for securing reusable code.
 - PA-4: Adversarial attacks on AI model: Best practices such as adversarial training (score of 3) and defense-GAN and generative defenses (score of 3) are used to mitigate the AI model against adversarial attacks. The level of mitigation for this PA is a 2 (Development), meaning that while advanced methodologies are known, further development is required to address adversarial risk.
 - PA-5: Overdependence on AI models: Functions such as human-in-the-loop (HITL) (2-point score) and explainable AI (XAI) (1-point score) indicate that the company is in the early stages of mitigating overdependence on AI models. This process area is capability level 1, which indicates that very little is being done to address this issue.

ID	Process areas (PA)	Cybersecurity mitigation levels			
Level 1	Ad Hoc cybersecurity risks	Novice (0)	Comprehension (1)	Development (2)	Advanced (3)
PA-1: injection attacks	Generative AI practices for addressing injection attacks				
P1	Input validation and sanitization			2	
P2	Code obfuscation and encryption				3
P3	Use of secure code templates		1		
P4	Static code analysis for vulnerability detection			2	
P5	Automatic generation of prepared statements				3
P6	Regular security audits and penetration testing			2	
P7	Context-aware AI models for reduced vulnerabilities	0			
Results of the evaluation of “hybrid ann-ism framework for mitigating cybersecurity risks in automatic code generation using generative AI” level-1 (PA-1) practices implemented by the company		Score		2	
		Mitigation level		Development	
Level 1	Ad Hoc cybersecurity risks				
PA-2: code quality and logic errors	Generative AI practices for addressing code quality and logic errors				
P1	Static code analysis				3
P2	Automated unit testing			2	
P3	Code refactoring				3
P4	Error detection with AI models		1		
P5	Logic verification			2	
P6	Automated code reviews				3
P7	Continuous integration and delivery (CI/CD)	0			
P8	Pattern recognition for best practices		1		
Results of the evaluation of “hybrid ANN-ISM framework for mitigating cybersecurity risks in automatic code generation using generative AI” level-1 (PA-2) practices implemented by the company		Score		3	
		Mitigation level		Advanced	
Level 1	Ad Hoc cybersecurity risks				
PA-3: backdoors and malicious codes	Generative AI practices for addressing backdoors and malicious codes				
P1	Malware detection				3
P2	Static analysis for vulnerabilities				3
P3	Anomaly detection	0			
P4	Automatic security audits		1		
P5	Runtime behavior analysis			2	
P6	Automated penetration testing				3
P7	Code obfuscation and protection			2	
Results of the evaluation of “hybrid ANN-ISM framework for mitigating cybersecurity risks in automatic code generation using generative AI” level-1 (PA-3) practices implemented by the company		Score		3	
		Mitigation level		Advanced	
Level 1	Ad Hoc cybersecurity risks				
PA-4: insufficient input validation	Generative AI practices for addressing insufficient input validation				
P1	Automated input validation generation				3
P2	Pattern matching for input validation				3
P3	Dynamic input testing			2	
P4	fuzz testing				3
P5	Automated code review for input handling	0			
P6	Input sanitization AI models		1		
P7	Machine learning for input validation				3
P8	Real-time input validation feedback			2	
P9	AI-driven security scanning for input handling				3
Results of the evaluation of “hybrid ANN-ISM framework for mitigating cybersecurity risks in automatic code generation using generative AI” level-1 (PA-4) practices implemented by the company		Score		3	
		Mitigation level		Advanced	
Level 2	Managed vulnerabilities and AI risks				
PA-1: vulnerabilities in reused codes	Generative AI practices for addressing vulnerabilities in reused code				
P1	Automated dependency management				3
Continued					

Level 2	Managed vulnerabilities and AI risks				
PA-1: vulnerabilities in reused codes	Generative AI practices for addressing vulnerabilities in reused code				
P2	Vulnerability scanning for legacy code			2	
P3	Automated patch generation			2	
P4	Code auditing for legacy systems	0			
P5	AI-dependency upgrade				3
P6	Legacy code refactoring			2	
P7	Static and dynamic code analysis		1		
P8	Compatibility testing for new dependencies				3
P9	AI-enhanced risk assessment for legacy code			2	
Results of the evaluation of “hybrid ANN-ISM framework for mitigating cybersecurity risks in automatic code generation using generative AI” level-2 (PA-1) practices implemented by the company		Score		2	
		Mitigation level		Development	
Level 2	Managed vulnerabilities and AI risks				
PA-2: lack of encryption	Generative AI practices for addressing the lack of encryption				
P1	Automated encryption code generation			2	
P2	AI-based classification		1		
P3	Automated key management				3
P4	Secure API generation with encryption				3
P5	Data masking and tokenization automation	0			
P6	AI-driven secure storage solutions		1		
P7	Real-time data leakage detection				3
P8	Automated compliance checking				3
P9	Static and dynamic encryption code analysis			2	
Results of the evaluation of “hybrid ANN-ISM framework for mitigating cybersecurity risks in automatic code generation using generative AI” level-2 (PA-2) practices implemented by the company		Score		3	
		Mitigation level		Advanced	
Level 2	Managed vulnerabilities and AI risks				
PA-3: reusability of vulnerable code	Generative AI practices for addressing the reusability of vulnerable code				
P1	AI-powered vulnerability detection		1		
P2	Automated secure code suggestions			2	
P3	Dependency risk analysis				3
P4	Code clone detection and remediation				3
P5	Secure code template generation	0			
P6	AI-enhanced code review				
P7	Continuous monitoring for vulnerable code usage				
P8	Refactoring legacy code automatically				
P9	Security policy enforcement via AI				
P10	AI-powered education and recommendations				
Results of the evaluation of “hybrid ANN-ISM framework for mitigating cybersecurity risks in automatic code generation using generative AI” level-2 (PA-3) practices implemented by the company		Score		3	
		Mitigation level		Advanced	
Level 2	Managed vulnerabilities and AI risks				
PA-4: adversarial attacks on AI models	Generative AI practices for addressing adversarial attacks on AI models				
P1	Adversarial training				3
P2	Input sanitization and preprocessing			2	
P3	Model verification and validation	0			
P4	Robustness testing frameworks			2	
P5	Defense-GAN and generative defenses				3
P6	Certified robustness methods		1		
P7	Ensemble methods			2	
P8	Adversarial attack simulation				3
P9	Continuous monitoring and model updates			2	
Results of the evaluation of “hybrid ANN-ISM framework for mitigating cybersecurity risks in automatic code generation using generative AI” level-2 (PA-4) practices implemented by the company		Score		2	
		Mitigation level		Development	

Level 2	Managed vulnerabilities and AI risks				
PA-5: overreliance on AI model	Generative AI practices for addressing overreliance on AI models				
P1	Human-in-the-loop (HITL)			2	
P2	Explainable AI (XAI)		1		
P3	Model monitoring and feedback loops				3
P4	Multi-model ensembles	0			
P5	Domain expert integration		1		
P6	Robust validation sets and testing				3
P7	User education and training		1		
P8	Fallback and override mechanisms			2	
P9	Transparency in data and model sources		1		
Results of the evaluation of “hybrid ANN-ISM framework for mitigating cybersecurity risks in automatic code generation using generative AI” level-2 (PA-5) practices implemented by the company		Score		1	
		Mitigation level		Comprehension	
Level 2	Managed vulnerabilities and AI risks				
PA-6: insufficient logging and monitoring	Generative AI practices for addressing insufficient logging and monitoring				
P1	Automated secure logging code generation				3
P2	Integration with centralized log management				3
P3	Anomaly detection on logs using AI		1		
P4	Inclusion of structure logging				3
P5	Real-time monitoring and alerting hooks			2	
P6	Compliance-aware logging practices	0			
P7	Automated log retention and rotation			2	
P8	Context-rich logging generation				3
P9	Continuous logging validation in CI/CD		1		
P10	User behavior analytics (UBA) integration				3
Results of the evaluation of “hybrid ANN-ISM framework for mitigating cybersecurity risks in automatic code generation using generative AI” level-2 (PA-6) practices implemented by the company		Score		3	
		Mitigation level		Advanced	
Level 3	Defined secure code practices and integration				
PA-1: lack of secure code review	Generative AI practices for addressing the lack of secure code review				
P1	Automated review code			2	
P2	AI-powered static application security testing (SAST)		1		
P3	Automated dynamic application security testing (DAST)				3
P4	Fuzz testing with AI			2	
P5	Test case generation using AI				3
P6	Continuous integration with AI-based testing				3
P7	AI-assisted peer code review	0			
P8	Regression testing automation		1		
P9	Security-focused code quality metrics				3
P10	AI-powered vulnerability prediction models			2	
Results of the evaluation of “hybrid ANN-ISM framework for mitigating cybersecurity risks in automatic code generation using generative AI” level-3 (PA-1) practices implemented by the company		Score		3	
		Mitigation level		Advanced	
Level 3	Defined secure code practices and integration				
PA-2: insecure integration with other systems	Generative AI practices for addressing insecure integration with other systems				
P1	Secure coding guidelines enforcement				3
P2	Static application security testing (SAST)				3
P3	Dynamic application security testing (DAST)			2	
P4	API security validation				3
P5	Dependency and supply chain analysis	0			
P6	Code generation with security templates			2	
P7	Automated threat modeling tools		1		
P8	Runtime application self-protection (RASP)				3
Continued					

Level 3	Defined secure code practices and integration				
PA-2: insecure integration with other systems	Generative AI practices for addressing insecure integration with other systems				
P9	Secure CI/CD pipelines			2	
P10	Logging and monitoring integration				3
Results of the evaluation of “hybrid ANN-ISM framework for mitigating cybersecurity risks in automatic code generation using generative AI” level-3 (PA-2) practices implemented by the company		Score		3	
		Mitigation level		Advanced	
Level 4	Quantitatively privacy and data protection management				
PA-1: privacy issues and data leakage	Generative AI practices for addressing privacy issues and data leakage				
P1	Code sanitization and redaction		1		
P2	Private codebase fine-tuning			2	
P3	Differential privacy techniques				3
P4	Secure prompt engineering		1		
P5	Out filtering and post-processing			2	
P6	Model usage logging and auditing	0			
P7	Access control and role-based use				3
P8	On-premise or air-gapped deployment		1		
P9	Open-source license checks				3
P10	Security-aware LLM training		1		
Results of the evaluation of “hybrid ANN-ISM framework for mitigating cybersecurity risks in automatic code generation using generative AI” level-4 (PA-1) practices implemented by the company		Score		1	
		Mitigation level		Comprehension	

Table 10. Example of a case study for evaluation of the hybrid ANN-ISM framework for mitigating cybersecurity risks in automatic code generation using generative AI.

- PA-6: Illegible logging and monitoring: The organization enforces a lower-class treatment, including automated generation of secured logging code (scored 3) and real-time monitoring and alerting hooks (scored 2). This PA achieved a Maturity Level 3 (Advanced), based on strong capabilities evidence in logging and monitoring.
- **Level 3: defined security code practices and integration**
 - PA-1: Lack of secure code review. The organization has automated DAST (score of 3) and Continuous Integration with AI testing (score of 3). This PA scored 3 on the assessment, representing Advanced practice maturity.
 - PA-2: Insecure integration with other systems: The following practices were implemented: secure coding guidelines enforcement (score of 3) and static and dynamic application security testing (score of 3), which results in a score of 3 for this PA, which shows Advanced, illustrative of the capabilities of integration security.
- **Level 4: Quantitatively privacy and data protection management**
 - PA-1: Privacy concerns and data leakage: The PA-1 process area includes practices, like differential privacy mechanism (score-3), access control, and role-based use (score-3). A couple of practices (model usage logging and auditing, score 0) suggest it is still falling short on implementing such privacy protections, though. The total mitigation score for this PA is 1 (Comprehension level), which indicates a requirement for considerable enhancement of privacy and data protection management.

The evaluation results through the case study of the Hybrid ANN-ISM Framework for mitigating Cyber Security Risks in Automatic code generation through Generative AI have shown that the company improved the situation of different types of cybersecurity risks in the company, but needs to enhance the situation of others.

The company demonstrates an Advanced (3) level of mitigation in most critical processes, including code quality and logic errors, backdoors and malicious code, and input validation. These spaces mirror strong processes that have proven successful, most notably static code analysis, passive penetration testing, and fuzz testing. And the widespread use of AI-based utilities and automated security audits indicates how sophisticated the company is in dealing with such threats.

From the evaluation of case study (encryption, reusability of insecure code, and logging and monitoring) are also areas where the company has demonstrated significant expertise, except that it can attain Advanced levels for most of the practices related to these process areas. This suggests that the company is successfully leveraging Generative AI to simplify the automation of security functions, including encryption key management or secure code recommendations, improving their global security posture.

However, the critique also serves as a reminder of where the company has work to do. Notably, in categories such as adversarial attacks on AI models and over-reliance on AI models, the company remains at the Development level (a score of 2). This implies that more work is still necessary to increase resilience in the face of adversarial threats and to minimize the risks of overdependence on AI systems. The Comprehension (score of 1) level in privacy and data leakage also suggests that the organization is at the beginning of regulating privacy in place, especially in model usage, logging, and auditing.

Overall, the company has done an excellent job addressing security risks introduced by the automatic code generator, especially regarding code quality, logic errors, and security review. However, there are certain aspects—like adversarial attacks, transparency in AI models, and data privacy—that we need to work on, and we need to advance. The company can also improve its security posture by concentrating on these areas and maintaining a stronger defense.

Figure 13 presents the overall case study evaluation of the hybrid ANN-ISM framework for mitigating cybersecurity risks in automatic code generation using generative AI.

Due to confidentiality constraints, the company's identity remains undisclosed; however, it has been described as a medium-sized firm engaged in AI-driven software development and code generation, employing approximately 120 staff members. The organization actively utilizes Generative AI coding platforms such as *CodeBERT* and *Codex*, making it representative of the operational landscape in AI-assisted development environments. This added context helps readers better understand the scope and applicability of the evaluation results.

The explanation accompanying Fig. 13 has been expanded to define the interpretation of the ANN-ISM model's output. The risk mitigation scores now range from 0 to 3, reflecting progressive levels of cybersecurity maturity as follows.

- 0—Very Low Mitigation
- 1—Basic Mitigation
- 2—Advanced Mitigation
- 3—Proactive Mitigation

A score of 3 (“Advanced Mitigation”) signifies that the organization has well-established cybersecurity procedures and policies, though further advancement is needed toward automation and predictive defense mechanisms.

Framework scenarios

Scenario 1: code injection via README files or comments

- **Threat:** Malicious code could be introduced through README files or comments within the code repository.
- **Path in ISM:** Code Injection (CR1) → README files or comments can act as entry points for attackers to insert harmful code or instructions that alter the behavior of the code generation process. This alteration can lead to the creation of insecure code, potentially introducing vulnerabilities like backdoors or data leaks.
- **Prioritized mitigations:**
 - Policy filters: Introduce filters to validate any inputs or code generated from external sources, such as README files or comments, to ensure they are free from harmful content.
 - Context isolation: Separate the code generation process from comments or documentation, ensuring that only actual code inputs influence the output, preventing the potential manipulation from external, possibly compromised, metadata.
 - SAST tools (Static Application Security Testing): Integrate static code analysis tools into the CI/CD pipeline to automatically detect any malicious code injections during the generation phase, providing alerts to developers before deployment.

Scenario 2: backdoor insertion via third-party dependencies

- **Threat:** Attackers exploit vulnerabilities in third-party libraries or frameworks to insert backdoors into the generated code.
- **Path in ISM:**
 - Backdoors in Dependencies (CR3) → Malicious third-party libraries or dependencies are introduced into the code generation pipeline, either by design or through supply chain attacks.
 - These dependencies can contain backdoor code that remains dormant until activated by certain conditions, providing attackers with unauthorized access to the system.
- **Prioritize mitigations:**
 - Dependency scanning: Integrate automated dependency scanning tools that assess third-party libraries for known vulnerabilities and suspicious code patterns. Ensure that only vetted dependencies are used in the CI/CD pipeline.
 - Trusted dependency management: Use secure and trusted dependency management systems, such as those that allow only approved versions of dependencies to be included in the project.
 - Code audits: Perform manual code audits and automated vulnerability scans during the code generation process to identify and mitigate any security flaws in the code, including hidden backdoors.

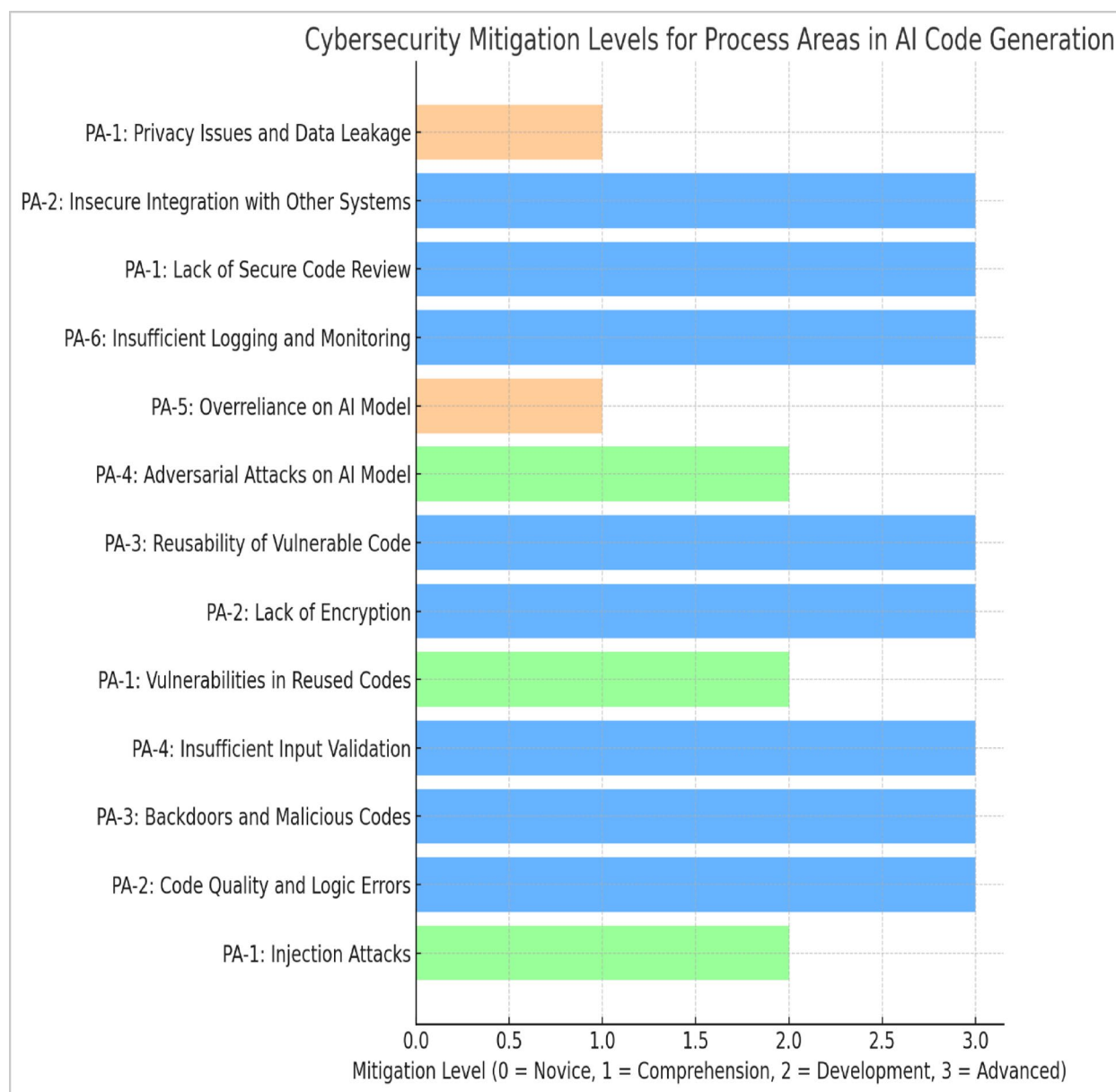


Fig. 13. Case study evaluation of the hybrid ANN-ISM framework for mitigating cybersecurity risks in automatic code generation using generative AI.

Scenario 3: adversarial manipulation of LLM inputs

- **Threat:** Subtle manipulation of inputs provided to the LLM (Large Language Model) by developers or external users, leading to the generation of insecure code.
- **Path in ISM:**
 - **Adversarial attacks on AI models (CR10)** → An attacker subtly alters the input fed to the Generative AI model, such as through malformed function descriptions or misleading comments, causing the model to generate insecure code.
 - This may lead to issues such as weak authentication mechanisms, flawed input validation, or improper error handling in the generated code.
- **Prioritized mitigation:**
 - **Input validation:** Implement input validation mechanisms to detect and filter adversarial inputs before they reach the LLM. This includes checking for unusual patterns, characters, or syntax that could trigger insecure code generation.

- **Contextual analysis:** Use contextual analysis algorithms to ensure that the generated code aligns with secure coding standards and does not exhibit risky behaviors, such as improperly handling sensitive data.
- **Model guardrails:** Introduce guardrails in the AI model to prevent it from generating certain types of code, such as those that violate best security practices (e.g., unencrypted sensitive data handling).

These scenarios show how different threats in the code generation pipeline can be mapped through the ISM framework, allowing for the identification of risk propagation paths and enabling the application of prioritized mitigations to reduce the likelihood of security breaches.

Implications of the study

The findings of this study provide several important implications for both the academic and practical application of cybersecurity risk mitigation in the context of automatic code generation using Generative AI:

- **Automatic code generation uses the highest level of security posture:** The result of the study indicates that the combination of Generative AI practices significantly contributes to the enhancement of automatic code generation in terms of cybersecurity. Further use of the automated best practices, including, among others, the code-review automation, static and dynamic vulnerability scanning, and adversarial training, provides comfortable defenses against many of the common threats, including injection, backdoors, malicious code, and input validation issues. The model describes a journey organizations should take from being in a Novice state of cybersecurity to being Advanced and emphasizes the importance of evolutionary maturity within security measures.
- **Maturity model development for cybersecurity:** One of the main novelties of this study concerns the development of a structured maturity model, which adopts the ANN and the ISM approaches to evaluate and improve the maturity of cybersecurity in the automatic code generation domain. The paper highlights that through maturity-based assessment, an organization can assess its current state of cybersecurity practices, identify gaps, and prioritize enhancements. This provides a realistic approach for SaaS vendors to incrementally improve their security processes and practices, so that their security posture can keep pace with greater risk and technological changes.
- **Potential for automation in cybersecurity:** The research demonstrates the possibility of automating critical cybersecurity tasks in the lifecycle of automatic code generation. This graduation result illustrates that methods such as vulnerability discovery via AI, secure code generation, and automated penetration testing can dramatically reduce human error and improve the response time and effectiveness of finding and resolving security threats. Therefore, the work focuses on the need for more research in developing automated security frameworks and tools. Studies that derive from Generative AI would allow organizations to embed security within the code generation process without much effort.
- **Application in real-world industry:** Case studies and evaluations of the framework with a reputed AI-based automatic code generation company give practical relevance to the framework. The findings indicate that organizations already use AI technologies to secure automatic code generation. Yet, there are opportunities to improve some aspects, including data privacy, model transparency, and AI model monitoring. These results indicate that while the adoption of Generative AI is positive, we need to carefully deal with the problems of overwhelming dependency on AI models and adversarial attacks to improve even more security and robustness in production environments.
- **Dissecting flaws of reused code:** One of the more notable results is the importance of vulnerabilities in reused code, ubiquitous in software development. The Hybrid ANN-ISM Framework offers the developer a method of identifying and fixing these weaknesses, thus ensuring that the legacy code and the third-party components do not make it possible to attack these modern systems. The paper highlights the role of strong dependency management and fully automated patch generation techniques throughout the development life cycle to minimize exposure to known vulnerabilities.
- **Need for cross-disciplinary collaboration:** The research also highlights the need for collaboration across disciplines of AI researchers, cybersecurity experts, and software engineers. The incorporation of AI in cybersecurity frameworks, as illustrated by the Hybrid ANN-ISM Framework, demands knowledge from the domains of AI and cybersecurity. In this paper, we see the need for more interdisciplinary research that can help refine them and make them more adaptable to diverse software development contexts.
- **Privacy and data protection considerations:** The study's data leakage results indicate a deficit in the industry standard, especially concerning model usage logging, secure prompt engineering, and data sanitization. Although the framework focuses on robust mitigation mechanisms for cybersecurity threats, privacy-related issues still require more attention, especially with the growing scale and competency of AI models regarding sensitive data. This is why it is crucial to bake privacy preservation into the AI training and code generation process from inception, not as an afterthought.
- **In the proposed Generative AI Cybersecurity Risks Mitigation Model using an ANN-ISM Hybrid Approach** entails both strategic investments and measurable returns for organizations seeking to enhance the security of AI-driven code generation systems. From a cost perspective, the model provide initial investments in computational infrastructure, AI training datasets, cybersecurity monitoring tools, and personnel upskilling to operate hybrid intelligence frameworks. These costs, however, are offset by substantial benefits, including a significant reduction in code vulnerabilities, automated threat detection efficiency, and improved model interpretability for compliance audits. By integrating artificial neural networks (ANN) for dynamic pattern recognition with Interpretive Structural Modeling (ISM) for hierarchical decision mapping, organizations achieve a balanced framework that enhances risk prediction accuracy, minimizes manual oversight costs, and supports continuous learning within secure development lifecycles. Overall, the long-term benefits—such as

reduced breach risks, improved code reliability, and enhanced trust in generative AI applications—outweigh the initial implementation costs, making the hybrid model a cost-effective and sustainable cybersecurity solution for modern software enterprises.

- **Pathway for future research:** The findings have thrown open many doors for future works in the realm of Generative AI and cybersecurity. Further studies might invest more in improving the framework in terms of dynamically adapting itself to the changes of security threats, as well as the extension to a broader set of software development environments. Moreover, the study of ethical questions related to the use of AI in security-critical settings is essential as the security industry steps towards more automation and AI-supported solutions.

The Hybrid ANN-ISM Framework for Cybersecurity Risk Mitigation presents a new generation of higher-secure automatic code generation processes by Generative AI. The contributions of the paper include theoretical and applied contributions that optimize the efficiency of the framework to identify, mitigate, and manage cybersecurity risks. Given the increasing deployment of AI in software development, the results of the study are identified as having an impacting role on future research and as a source of reference for industry for developing more secure, efficient, and privacy-preserving AI systems.

Limitations of the study

While the paper provides a thorough analysis of the Hybrid ANN-ISM Framework for Mitigating Cybersecurity Risks in Automatic Code Generation Using Generative AI, there are some limitations:

- **Small number of case studies:** The assessment was based predominantly on one case study, one company, and one use of automatic code generation. The findings may not universally apply to other organizations with varied infrastructure, processes, or cyber-risks. More case studies in different industries would be helpful to confirm the applicability and workable of the framework across various contexts.
- **Focus on generative AI:** While the paper shows how Generative AI could be used to reduce cybersecurity threats in code generation, it does not provide a deep investigation of other technologies or other approaches that could equally or more efficiently protect against these threats. The complementarity of Generative AI and other AI paradigms like reinforcement learning or expert systems has also not been thoroughly investigated.
- **Evaluation bias:** The feedback collected from the internally company team may have some bias because the team members might have an interest in serving as a fair representative or may not be able to reveal all aspects. Third-party plus-ups or a review by outside cybersecurity experts might offer a more objective perspective of how well the framework is (or is not) fulfilling its function.
- **Absence of long-term assessment:** The research tests the successfulness of the system in a pilot implementation of short-term duration. A longitudinal evaluation would be needed to see how the framework holds up over time and how, more specifically, it might evolve to reflect new cybersecurity threats and the continual learning patterns of the AI models.
- **Lack of cost–benefit analysis:** The research fails to include any possible costs incurred in adopting the framework (i.e., the computational requirements for Generative AI tasks, the cost of integrating new tools, or the time staff would have to spend getting used to the tools). A more detailed cost/benefit analysis would shed additional light on the practicality of deploying the framework in real-world settings.
- **Technological and environmental assumptions:** We assume there are advanced AI tools and much computation. Institutions with insufficient technical infrastructure or resources may struggle to implement such a framework successfully. The constraints are not represented in this study, and a new work would be necessary to verify how the architecture allows it to be adapted to this kind of environment.
- **Implicit threats:** Although aspects such as injection attacks, code quality, and backdoors are covered under analysis areas, other IT threats like social engineering, insider threat, or supply chain attacks are not explicitly addressed in the model. The model could also be expanded in future research to include other categories of cybersecurity threats.
- **On over-dependence on AI model:** A potential limitation of employing Generative AI to address abuse is the possible overreliance on AI-based mitigation approaches. As demonstrated in the analysis, the paper indicates cases of over-dependence on AI models. Although AI can be beneficial in this regard, the human factor and intervention are also necessary for such complex cyber challenges, and a balance must be found.
- **Narrow discussion of privacy regulations:** The framework addresses concerns over privacy and data leakage, but it does not delve too deeply into the legal and regulatory considerations around the use of AI in code generation, particularly in regions with strict privacy laws (e.g., GDPR in Europe). The paper would benefit from a deeper analysis of how data privacy regulations interface with AI-based security models.

To conclude, while the proposed Hybrid ANN-ISM Framework offers a meaningful contribution to the study of cybersecurity in code generation, we must take the limitations discussed into account when interpreting the results. Several future research areas to improve the robustness, generalization, and practical usability in different real-world scenarios must be addressed.

Conclusion and future research direction

The Hybrid ANN-ISM Framework introduced in the paper can protect automatic code generation from cybersecurity threats using Generative AI. By taking advantage of the merits of ANN and ISM, this framework is robust to tackle the serious cybersecurity problems of the automatic code generation systematically, especially on injection attacks, backdoor, and insufficient input validation etc. A detailed case study and measurement showed the framework could lead to significantly improved identification and mitigation of security risks

even in organizations with various security maturity levels. The company implementing the framework made significant strides in key areas, after which more advanced mitigations were already in place, including code quality, malware detection, and secure input validation. However, some process areas (i.e., adversarial attacks against AI models, over-dependence on AI, and lack of AI models) revealed a need for additional development, and such processes could be improved in systems with human in the loop, explainable AI (XAI), and robustness of AI models to adversarial threats.

Furthermore, the framework could stress the significance of data privacy and model security, specifically in reducing the exposure to data leakage and insecure code integration. While the organization has progressed towards standard security practices for code generation, more developments are required to fill privacy gaps and improve logging, monitoring, and adversarial defense.

Finally, in conclusion, the Hybrid ANN-ISM approach presents a valuable technique for systematically addressing issues introduced by cybersecurity risks in automatic code generation. Its versatility and flexibility make it a must-have for any organization that values security and dependability in its automated systems. In the future, concentrated attention in particular areas such as adversarial robustness, AI model transparency, and privacy hygiene will be necessary for safeguarding against the broader set of emerging threats that enable automated code generation.

The proposed Hybrid ANN-ISM Framework for Mitigating Cybersecurity Risks in Generative AI in Auto-Generating Codes can potentially improve cybersecurity in software development. However, for better performance and more general use, there are still some directions to be explored:

- **Integration of advanced AI techniques:** While the present framework includes Generative AI, there is potential for integrating more advanced AI techniques such as Reinforcement Learning, Federated Learning, and Deep Learning-based Anomaly Detection. If these methods can be integrated, the defense framework should adapt and react to dynamic changes in cybersecurity threat activities in real time. It needs to be a more robust and dynamic defense mechanism.
- **Real-time threat discovery and response:** Further work can be done to explore how the hybrid approach can be extended towards real-time threat discovery and response in the code generation pipeline. This would mean monitoring created code in real time as it is generated, searching for security weaknesses. Including run-time behavior analyzing tools and live anomaly detection in this framework could enhance its capabilities to protect against zero-day exploits and advanced attack methods.
- **Cross-domain applications:** Although the current model is oriented towards security risks in automatic code generators, future research could investigate the possibility of expanding the use of this model for cloud computing, IoT, and mobile applications. Each of these domains imposes its unique security requirements, and shaping the framework to fit the specific requirements of each of these would potentially provide broader applicability as well as more evidence of its effectiveness.
- **Automated security audits/compliance:** An opportunity area could be automatic compliance checks with industry standards and regulations. Adopting the framework and combining it with scanners to automate code scanning for compliance with security regulations (such as GDPR, HIPAA, PCI-DSS) would be helpful for organizations to keep their security and privacy levels up. We will consider in future work to improve the ability of the framework to produce compliance reports and to perform security audits with less human effort.
- **Human-AI partnership:** There's also a bit of a balancing act when it comes to AI in cybersecurity, which is how to let humans be in control, without interfering with the machine or slowing systems down. In the future, one could investigate human-in-the-loop (HITL) systems that can incorporate the AI-driven models with human knowledge or experiences, for example, in the cases of complex or new security environments. Additional research could focus on enabling human decision-making at crucial points in code generation, keeping the framework decision consistent with organizational policies and acceptable threat levels.
- **Better privacy:** At present, the existing structure does not offer an adequate solution to the risks facing data privacy. Nevertheless, privacy regulations are gaining attention, and the fear of data breaches is becoming more critical; as a future work, it may make sense to add a part regarding privacy-preserving to the framework. "Security features, such as differential privacy, secure multi-party computation, and homomorphic encryption, can be integrated into the model to improve the security of sensitive data used in code generation.
- **Performance tuning:** Given that AI-powered cybersecurity solutions can result in heavy computational overhead, further investigation may consider tuning the performance of the Hybrid ANN-ISM Framework to become more efficient in practical applications. This could mean alleviating the computational complexity of Generative AI models, enhancing the scalability of the framework, and making sure that it is capable of handling large-scale code generation tasks without compromising on accuracy and speed.
- **Real-world case studies:** Although the case study in the paper is an essential initial step in evaluating the framework for practical utility, the following search should involve real-world case studies across different domains and code generator environments. Comparisons with other cybersecurity frameworks can also contribute to an enriched understanding of the advantages and limitations of the Hybrid ANN-ISM Framework.
- **Explainability and transparency in AI:** With AI models increasingly central to the cybersecurity toolset, the demand for clarity and transparency around AI-made decisions becomes paramount. In the future, this work aims to represent the interpretability aspect of the Hybrid ANN-ISM Framework by creating techniques to deliver comprehensible insights to the user as to how the framework is detecting and preventing the threats. Making decisions generated by AI explainable to non-experts will lead to greater confidence and enable it to become more widely adopted.
- **Adaptive framework for dynamic and evolving cyber threats:** The security framework must adapt as the cyber threats evolve. Given the security and technology focus, research to improve the self-updatability of threat detection techniques could be explored, leveraging real-time data feeds, threat intelligence platforms,

and global trends in cybersecurity. This would make the framework capable of evolving in proactive reactions to newly invented attack mechanics, e.g., breaking approaches for new technologies or new forms of exploitation.

Overall, the future development of the proposed Hybrid ANN-ISM Framework provides countless opportunities for contributing to the advancement of the cybersecurity community for automatic code generation. By considering these research directions, the framework can be further strengthened to assist in mitigating risks, coping with new threats, and securing the privacy of software systems produced by AI-based methods.

Data availability

All data generated or analyzed during this study are included in this published article and its supplementary information files. Additional datasets are available from the corresponding author on reasonable request.

Appendix A SSIM matrix

CRs	Cybersecurity risks (CRs)	CR1	CR2	CR3	CR4	CR5	CR6	CR7	CR8	CR9	CR10	CR11	CR12	CR13	CR14
CR1	Injection attacks	*	X	O	O	A	O	O	O	A	A	O	O	O	O
CR2	Code quality and logic errors	*	*	A	O	V	A	O	O	O	O	O	O	A	O
CR3	Backdoors and malicious code	*	*	*	A	O	X	O	A	A	O	O	A	O	O
CR4	Vulnerabilities in reused code (legacy dependencies)	*	*	*	*	O	O	O	O	A	O	X	O	O	V
CR5	Insufficient input validation	*	*	*	*	*	X	O	O	O	O	O	A	O	O
CR6	Weak authentication and authorization mechanisms	*	*	*	*	*	*	O	O	O	O	A	O	O	O
CR7	Lack of encryption OR insecure data handling	*	*	*	*	*	*	*	A	O	X	O	O	O	A
CR8	Reusability of vulnerable code	*	*	*	*	*	*	*	*	O	O	A	O	O	O
CR9	Lack of secure code review and testing	*	*	*	*	*	*	*	*	*	O	V	A	X	O
CR10	Adversarial attacks on AI models	*	*	*	*	*	*	*	*	*	*	X	A	A	V
CR11	Overreliance on an AI model	*	*	*	*	*	*	*	*	*	*	*	O	O	O
CR12	Privacy issues and data leakage	*	*	*	*	*	*	*	*	*	*	*	*	O	O
CR13	Insecure integration with other systems	*	*	*	*	*	*	*	*	*	*	*	*	*	O
CR14	Insufficient logging and monitoring	*	*	*	*	*	*	*	*	*	*	*	*	*	*

Appendix B Initial reachability matrix

From / To	CR1	CR2	CR3	CR4	CR5	CR6	CR7	CR8	CR9	CR10	CR11	CR12	CR13	CR14
CR1	1	1	0	0	0	0	0	0	0	0	0	0	0	0
CR2	0	1	0	0	1	0	0	0	0	0	0	0	0	0
CR3	0	1	1	0	0	1	0	0	0	0	0	0	1	0
CR4	0	0	1	1	0	0	0	0	0	1	0	0	0	1
CR5	0	0	0	0	1	1	0	0	0	0	1	0	0	0
CR6	0	0	0	0	0	1	0	0	0	1	0	0	0	0
CR7	0	0	0	0	0	0	1	0	0	1	0	0	0	1
CR8	0	0	0	0	0	0	0	1	0	0	1	0	0	0
CR9	0	0	0	0	0	0	0	0	1	0	1	1	1	0
CR10	0	0	0	0	0	0	0	0	0	1	1	1	1	1
CR11	0	0	0	0	0	0	0	0	0	1	0	0	0	0
CR12	0	0	0	0	0	0	0	0	0	1	1	0	0	0
CR13	0	0	0	0	0	0	0	0	0	1	1	1	1	0
CR14	0	0	0	0	0	0	0	0	0	1	1	1	1	1

Appendix C Data set (variable view) for ANN model

Name	Type	Width	Decimals	Labels	Values	Measures	Role
Cybersecurity Risks (CRs)	Numeric	8	2	Cybersecurity risks in automatic code generation	None	Scale	Input
CR1	Numeric	8	2	Injection attacks	{2 = 10–20%, 3 = 20–30%, 4 = 30–40%, 5 = 40–50%, 6 = 50–60%, 7 = 60–70%, 8 = 70–80%, 9 = 80–90%, 10 = 90–100%}	Nominal	Input
CR2	Numeric	8	2	Code quality and logic errors	{2 = 10–20%, 3 = 20–30%, 4 = 30–40%, 5 = 40–50%, 6 = 50–60%, 7 = 60–70%, 8 = 70–80%, 9 = 80–90%, 10 = 90–100%}	Nominal	Input
CR3	Numeric	8	2	Backdoors and malicious code	{2 = 10–20%, 3 = 20–30%, 4 = 30–40%, 5 = 40–50%, 6 = 50–60%, 7 = 60–70%, 8 = 70–80%, 9 = 80–90%, 10 = 90–100%}	Nominal	Input
CR4	Numeric	8	2	Vulnerabilities in reused code (legacy dependencies)	{2 = 10–20%, 3 = 20–30%, 4 = 30–40%, 5 = 40–50%, 6 = 50–60%, 7 = 60–70%, 8 = 70–80%, 9 = 80–90%, 10 = 90–100%}	Nominal	Input
CR5	Numeric	8	2	Insufficient input validation	{2 = 10–20%, 3 = 20–30%, 4 = 30–40%, 5 = 40–50%, 6 = 50–60%, 7 = 60–70%, 8 = 70–80%, 9 = 80–90%, 10 = 90–100%}	Nominal	Input
CR6	Numeric	8	2	Weak authentication and authorization mechanisms	{2 = 10–20%, 3 = 20–30%, 4 = 30–40%, 5 = 40–50%, 6 = 50–60%, 7 = 60–70%, 8 = 70–80%, 9 = 80–90%, 10 = 90–100%}	Nominal	Input
CR7	Numeric	8	2	Lack of encryption OR insecure data handling	{2 = 10–20%, 3 = 20–30%, 4 = 30–40%, 5 = 40–50%, 6 = 50–60%, 7 = 60–70%, 8 = 70–80%, 9 = 80–90%, 10 = 90–100%}	Nominal	Input
CR8	Numeric	8	2	Reusability of vulnerable code	{2 = 10–20%, 3 = 20–30%, 4 = 30–40%, 5 = 40–50%, 6 = 50–60%, 7 = 60–70%, 8 = 70–80%, 9 = 80–90%, 10 = 90–100%}	Nominal	Input
CR9	Numeric	8	2	Lack of secure code review and testing	{2 = 10–20%, 3 = 20–30%, 4 = 30–40%, 5 = 40–50%, 6 = 50–60%, 7 = 60–70%, 8 = 70–80%, 9 = 80–90%, 10 = 90–100%}	Nominal	Input
CR10	Numeric	8	2	Adversarial attacks on AI models	{2 = 10–20%, 3 = 20–30%, 4 = 30–40%, 5 = 40–50%, 6 = 50–60%, 7 = 60–70%, 8 = 70–80%, 9 = 80–90%, 10 = 90–100%}	Nominal	Input
CR11	Numeric	8	2	Over reliance on AI model	{2 = 10–20%, 3 = 20–30%, 4 = 30–40%, 5 = 40–50%, 6 = 50–60%, 7 = 60–70%, 8 = 70–80%, 9 = 80–90%, 10 = 90–100%}	Nominal	Input
CR12	Numeric	8	2	Privacy issues and data leakage	{2 = 10–20%, 3 = 20–30%, 4 = 30–40%, 5 = 40–50%, 6 = 50–60%, 7 = 60–70%, 8 = 70–80%, 9 = 80–90%, 10 = 90–100%}	Nominal	Input
CR13	Numeric	8	2	Insecure integration with other system	{2 = 10–20%, 3 = 20–30%, 4 = 30–40%, 5 = 40–50%, 6 = 50–60%, 7 = 60–70%, 8 = 70–80%, 9 = 80–90%, 10 = 90–100%}	Nominal	Input
CR14	Numeric	8	2	Insufficient logging and monitoring	{2 = 10–20%, 3 = 20–30%, 4 = 30–40%, 5 = 40–50%, 6 = 50–60%, 7 = 60–70%, 8 = 70–80%, 9 = 80–90%, 10 = 90–100%}	Nominal	Input

Appendix D Data set (data view) for ANN model

Respondents (R)	Cybersecurity risks	CR1	CR2	CR3	CR4	CR5	CR6	CR7	CR8	CR9	CR10	CR11	CR12	CR13	CR14
R1	8	8	8	5	5	7	9	5	5	7	5	4	5	7	8
R2	9	6	10	7	6	6	5	7	8	7	6	6	6	8	10
R3	7	7	6	5	5	7	6	8	10	5	5	9	5	8	9
R4	5	9	10	8	8	8	5	8	9	8	8	5	5	4	4
R5	10	7	6	10	10	6	8	6	5	6	10	6	6	8	9
R6	5	7	8	7	6	10	10	7	8	6	7	7	5	7	5
R7	4	8	5	5	4	6	4	5	3	8	5	5	9	6	5
R8	9	10	9	6	2	5	9	6	5	6	6	6	3	5	9
R9	7	5	8	7	6	5	6	9	5	5	7	9	4	5	3
R10	8	8	5	5	4	6	4	5	3	8	5	5	9	6	5
R11	5	10	9	6	2	5	9	6	5	6	6	6	3	5	9
R12	3	7	5	5	4	8	3	5	9	7	5	5	3	8	5

Respondents (R)	Cybersecurity risks	CR1	CR2	CR3	CR4	CR5	CR6	CR7	CR8	CR9	CR10	CR11	CR12	CR13	CR14
R13	9	5	6	4	5	10	3	8	5	4	4	8	6	7	6
R14	7	7	5	9	9	7	6	7	6	8	9	7	9	5	5
R15	8	8	8	5	5	7	9	5	5	7	5	4	5	7	8
R16	9	6	10	7	6	6	5	7	8	7	6	6	6	8	10
R17	7	7	6	5	5	7	6	8	10	5	5	9	5	8	9
R18	6	9	10	8	8	8	5	8	9	8	8	5	5	4	4
R19	7	7	6	10	10	6	8	6	5	6	10	6	6	8	9
R20	10	7	8	7	6	10	10	7	8	6	7	7	5	7	5
R21	10	8	5	5	4	6	4	5	3	8	5	5	9	6	5
R22	5	10	9	6	2	5	9	6	5	6	6	6	3	5	9
R23	6	7	5	5	4	8	3	5	9	7	5	5	3	8	5
R24	8	5	6	4	5	10	3	8	5	4	4	8	6	7	6
R25	9	7	5	5	8	8	5	5	7	9	5	5	7	5	4

Received: 14 July 2025; Accepted: 28 December 2025
Published online: 14 January 2026

References

1. Sirbu, A.-G. & Czibula, G. Automatic code generation based on abstract syntax-based encoding. Application on malware detection code generation based on MITRE attack techniques. *Expert Syst. Appl.* **264**, 125821 (2025).

2. Patsakis, C., Casino, F. & Lykousas, N. Assessing LLMs in malicious code deobfuscation of real-world malware campaigns. *Expert Syst. Appl.* **256**, 124912 (2024).

3. Ding, H., Liu, Y., Piao, X., Song, H. & Ji, Z. SmartGuard: An LLM-enhanced framework for smart contract vulnerability detection. *Expert Syst. Appl.* **269**, 126479 (2025).

4. Gurtu, A. & Lim, D. Chapter 101—Use of artificial intelligence (AI) in cybersecurity. In *Computer and Information Security Handbook (Fourth Edition)*, (ed. Vacca, J. R.) 1617–1624 (Morgan Kaufmann, 2025).

5. Diro, A. et al. Workplace security and privacy implications in the GenAI age: A survey. *J. Inf. Secur. Appl.* **89**, 103960 (2025).

6. Sá, D. et al. A state-of-the-art of intelligent problem-oriented low-code systems. *Proced. Comput. Sci.* **257**, 1122–1127 (2025).

7. Qu, Y., Huang, S. & Nie, P. A review of backdoor attacks and defenses in code large language models: Implications for security measures. *Inf. Softw. Technol.* **182**, 107707 (2025).

8. Becker, B.A., Denny, P., Finnie-Ansley, J., Luxton-Reilly, A., Prather, J. & Santos, E.A. Programming is hard-or at least it used to be: Educational opportunities and challenges of AI code generation. 500–506 (2023).

9. Cotroneo, D., Foggia, A., Improta, C., Liguori, P. & Natella, R. Automating the correctness assessment of AI-generated code for security contexts. *J. Syst. Softw.* **216**, 112113 (2024).

10. Khan, H. U. et al. AI-driven cybersecurity framework for software development based on the ANN-ISM paradigm. *Sci. Rep.* **15**(1), 13423 (2025).

11. Alfayez, R., Winn, R., Alwehaibi, W., Venson, E. & Boehm, B. How SonarQube-identified technical debt is prioritized: An exploratory case study. *Inf. Softw. Technol.* **156**, 107147 (2023).

12. del Hoyo-Gabaldon, J. A., Moreno-Cediel, A., Garcia-Lopez, E., Garcia-Cabot, A. & de Fitero-Dominguez, D. Automatic dataset generation for automated program repair of bugs and vulnerabilities through SonarQube. *SoftwareX* **26**, 101664 (2024).

13. Kessel, M. & Atkinson, C. Code search engines for the next generation. *J. Syst. Softw.* **215**, 112065 (2024).

14. Sparkes, M. AI programmer may be reusing code without asking. *New Sci.* **251**(3343), 13 (2021).

15. Ndukwe, I. G., Licorish, S. A., Tahir, A. & MacDonell, S. G. How have views on software quality differed over time? Research and practice viewpoints. *J. Syst. Softw.* **195**, 111524 (2023).

16. Tooki, O. O. & Popoola, O. M. A critical review on intelligent-based techniques for detection and mitigation of cyberthreats and cascaded failures in cyber-physical power systems. *Renew. Energy Focus* **51**, 100628 (2024).

17. Sinha, M., Bera, P. & Satpathy, M. SDN_Guard: An advanced machine learning based defense system against packet injection attacks in SDN. *Proced. Comput. Sci.* **258**, 2490–2499 (2025).

18. Rahman, M. A., Bhuiyan, T. & Ali, M. A. Enhancing aviation safety: Machine learning for real-time ADS-B injection detection through advanced data analysis. *Alex. Eng. J.* **126**, 262–276 (2025).

19. Crespo-Martínez, I. S. et al. SQL injection attack detection in network flow data. *Comput. Secur.* **127**, 103093 (2023).

20. Gaber, T., El-Ghamry, A. & Hassanien, A. E. Injection attack detection using machine learning for smart IoT applications. *Phys. Commun.* **52**, 101685 (2022).

21. Kaur, R., Gabrijelčić, D. & Kloboučar, T. Artificial intelligence for cybersecurity: Literature review and future research directions. *Information Fusion* **97**, 101804 (2023).

22. Fui-Hoon Nah, F., Zheng, R., Cai, J., Siau, K. & Chen, L. *Generative AI and ChatGPT: Applications, Challenges, and AI-Human Collaboration* 3, 277–304 (Taylor & Francis, 2023).

23. Huang, L., Liu, H., Liu, Y., Shang, Y. & Li, Z. A Generative Adversarial Imitation Learning Method for Continuous Integration Testing 1084–1089 (2024).

24. Ma, Z., Mei, G. & Xu, N. Generative deep learning for data generation in natural hazard analysis: Motivations, advances, challenges, and opportunities. *Artif. Intell. Rev.* **57**(6), 160 (2024).

25. Nadella, G. S. et al. Generative AI-enhanced cybersecurity framework for enterprise data privacy management. *Computers* **14**(2), 55 (2025).

26. Sharma, P., Kumar, M., Sharma, H. K. & Biju, S. M. Generative adversarial networks (GANs): Introduction, Taxonomy, Variants, Limitations, and Applications. *Multimed. Tools Appl.* **83**, 88811 (2024).

27. Sabuhi, M., Zhou, M., Bezemer, C. P. & Musilek, P. Applications of generative adversarial networks in anomaly detection: A systematic literature review. *IEEE Access* **9**, 161003–161029 (2021).

28. Venkatesan, K. & Rahayu, S. B. Blockchain security enhancement: An approach towards hybrid consensus algorithms and machine learning techniques. *Sci. Rep.* **14**(1), 1149 (2024).

29. Rabhi, M., Bakiras, S. & Di Pietro, R. Audio-deepfake detection: Adversarial attacks and countermeasures. *Expert Syst. Appl.* **250**, 123941 (2024).

30. Coppolino, L., D'Antonio, S., Mazzeo, G. & Uccello, F. The good, the bad, and the algorithm: The impact of generative AI on cybersecurity. *Neurocomputing* **623**, 129406 (2025).
31. Khan, R. A., Khan, H. U., Alwageed, H. S., Al Hashimi, H. A. & Keshta, I. 5G networks security mitigation model: An ANN-ISM hybrid approach. *IEEE Open J. Commun. Soc.* **6**, 881–925 (2025).
32. Guo, X. *Towards Automated Software Testing with Generative Adversarial Networks* 21–22 (2021).
33. Ding, A., Li, G., Yi, X., Lin, X., Li, J. & Zhang, C. Generative artificial intelligence for software security analysis: fundamentals, applications, and challenges. *IEEE Softw.* 1–8 (2024).
34. Ebert, C. & Louridas, P. Generative AI for software practitioners. *IEEE Softw.* **40**(4), 30–38 (2023).
35. Garousi, V., Felderer, M. & Mäntylä, M. V. Guidelines for including grey literature and conducting multivocal literature reviews in software engineering. *Inf. Softw. Technol.* **106**, 101–121 (2019).
36. Itodo, C. & Ozer, M. Multivocal literature review on zero-trust security implementation. *Comput. Secur.* **141**, 103827 (2024).
37. Akbar, M. A., Smolander, K., Mahmood, S. & Alsanad, A. Toward successful DevSecOps in software development organizations: A decision-making framework. *Inf. Softw. Technol.* **147**, 106894 (2022).
38. Al-Matouq, H., Mahmood, S., Alshayeb, M. & Niazi, M. A maturity model for secure software design: A multivocal study. *IEEE Access* **8**, 215758–215776 (2020).
39. Wagner, S. et al. Status Quo in requirements engineering: A theory and a global family of surveys. *ACM Trans. Softw. Eng. Methodol.* **28**(2), Article 9 (2019).
40. Humayun, M., Niazi, M., Assiri, M. & Haoues, M. Secure global software development: A practitioners' perspective. *Appl. Sci.* **13**(4), 2465 (2023).
41. Ilyas, M., Khan, S. U., Khan, H. U. & Rashid, N. Software integration model: An assessment tool for global software development vendors. *J. Soft. Evol. Process* **36**, e2540 (2023).
42. Creswell, J. W. *Research Design: Qualitative, Quantitative and Mixed Methods Approaches* 3rd edn. (Sage, London, 2009).
43. Lethbridge, T. C., Sim, S. E. & Singer, J. Studying software engineers: Data collection techniques for software field studies. *Empir. Softw. Eng.* **10**(3), 311–341 (2005).
44. Lee, S.-C. Prediction of concrete strength using artificial neural networks. *Eng. Struct.* **25**(7), 849–857 (2003).
45. Leong, L.-Y., Hew, T.-S., Tan, G.W.-H. & Ooi, K.-B. Predicting the determinants of the NFC-enabled mobile credit card acceptance: A neural networks approach. *Expert Syst. Appl.* **40**(14), 5604–5620 (2013).
46. Chan, F. T. & Chong, A. Y. A SEM-neural network approach for understanding determinants of interorganizational system standard adoption and performances. *Decis. Support Syst.* **54**(1), 621–630 (2012).
47. Zhang, H., Wang, L., Sheng, Y., Xu, X., Mankoff, J. & Dey, A. K. A framework for designing fair ubiquitous computing systems. *arXiv preprint arXiv:2308.08710* (2023).
48. Chong, A.Y.-L. Predicting m-commerce adoption determinants: A neural network approach. *Expert Syst. Appl.* **40**(2), 523–530 (2013).
49. Hertz, J., Krogh, A., Palmer, R. G. & Horner, H. *Introduction to the Theory of Neural Computation* (American Institute of Physics, 1991).
50. Alnaizy, R., Aidan, A., Abachi, N. & Jabbar, N. A. Neural network model identification and advanced control of a membrane biological reactor. *J. Membr. Sep. Technol.* **2**(4), 231 (2013).
51. S. A. P. *Interpretive Structural Modeling: Methodology for Large Scale Systems* 1–445 (New York, McGraw-Hill 1977).
52. Ravi, V. & Shankar, R. Analysis of interactions among the barriers of reverse logistics. *Technol. Forecast. Soc. Chang.* **72**(8), 1011–1029 (2005).
53. Rafi, S., Akbar, M. A., Mahmood, S., Alsanad, A. & Alothaim, A. Selection of DevOps best test practices: A hybrid approach using ISM and fuzzy TOPSIS analysis. *J. Soft. Evol. Process* **34**(5), e2448 (2022).
54. Qureshi, K. M. et al. Exploring the lean implementation barriers in small and medium-sized enterprises using interpretive structure modeling and interpretive ranking process. *Appl. Syst. Innov.* **5**(4), 84 (2022).
55. Talib, F., Rahman, Z. & Qureshi, M. R. An interpretive structural modeling approach for modeling the practices of total quality management in service sector. *Int. J. Model. Oper. Manage. Indersci.* **1**, 223–250 (2011).
56. Rafi, S. et al. Exploration of DevOps testing process capabilities: An ISM and fuzzy TOPSIS analysis. *Appl. Soft Comput.* **116**, 108377 (2022).
57. Sakar, C., Koseoglu, B., Toz, A. C. & Buber, M. Analysing the effects of liquefaction on capsizing through integrating interpretive structural modelling (ISM) and fuzzy Bayesian networks (FBN). *Ocean Eng.* **215**, 107917 (2020).
58. Patel, M. N., Pujara, A. A., Kant, R. & Malviya, R. K. Assessment of circular economy enablers: Hybrid ISM and fuzzy MICMAC approach. *J. Clean. Prod.* **317**, 128387 (2021).
59. Ali, S., Huang, J., Khan, S. U. & Li, H. A framework for modelling structural association amongst barriers to software outsourcing partnership formation: An interpretive structural modelling approach. *J. Softw. Evol. Process* **32**(6), e2243 (2020).
60. Ali, S. et al. Analyzing the interactions among factors affecting cloud adoption for software testing: a two-stage ISM-ANN approach. *Soft Comput.* **26**(16), 8047–8075 (2022).
61. Qureshi, K. M. et al. Accomplishing sustainability in manufacturing system for small and medium-sized enterprises (SMEs) through lean implementation. *Sustainability* **14**(15), 9732 (2022).
62. Qureshi, M. R. & Kumar, P. An integrated model to identify and classify the key criteria and their role in the assessment of 3PL services providers. *Asia Pacific J. Mark. Logist.* **20**, 227–249 (2008).
63. Qureshi, M. R. & Kumar, P. Modeling the logistics outsourcing relationship variables to enhance shippers' productivity and competitiveness in logistical supply chain. *Int. J. Product. Perform. Manag.* **56**, 689–714 (2007).
64. Gershfeld, I. & Sturm, A. Evaluating the effectiveness of a security flaws prevention tool. *Inf. Softw. Technol.* **170**, 107427 (2024).
65. McKevitt, J., Vorobyov, E. I. & Kulikov, I. Accelerating Fortran codes: A method for integrating Coarray Fortran with CUDA Fortran and OpenMP. *Journal of Parallel and Distributed Computing* **195**, 104977 (2025).
66. Dobre, D. & Vasilăteanu, A. Electronic health record authentication and authorization using Blockchain and QR codes. *Proced. Comput. Sci.* **239**, 1784–1791 (2024).
67. Harmening, J. Chapter 24—Information security essentials for IT managers: Protecting mission-critical systems. In *Computer and Information Security Handbook (Fourth Edition)*, (ed. Vacca, J. R.), 423–432 (Morgan Kaufmann, 2025).
68. Lange, F. & Kunz, I. Evolution of secure development lifecycles and maturity models in the context of hosted solutions. *J. Soft. Evol. Process* **36**, e2711 (2024).
69. Kim, M., Yang, H. & Lee, J. Fully private and secure coded matrix multiplication with colluding workers. *ICT Express* **9**, 722 (2023).
70. McIntosh, T. R. et al. From COBIT to ISO 42001: Evaluating cybersecurity frameworks for opportunities, risks, and regulatory compliance in commercializing large language models. *Comput. Secur.* **144**, 103964 (2024).
71. Casola, V., De Benedictis, A., Mazzocca, C. & Orbinato, V. Secure software development and testing: A model-based methodology. *Computers & Security* **137**, 103639 (2023).
72. Chomutare, T. et al. Improving quality of ICD-10 (International statistical classification of diseases, tenth revision) coding using AI: Protocol for a crossover randomized controlled trial. *JMIR Res. Protoc.* **13**, e54593 (2024).
73. Almeida, Y. et al. AICodeReview: Advancing code quality with AI-enhanced reviews. *SoftwareX* **26**, 101677 (2024).
74. Rodriguez, D. V. et al. Leveraging generative AI tools to support the development of digital solutions in health care research: Case study. *JMIR Hum. Factors* **11**, e52885 (2024).

75. Maikantis, T. et al. Code beauty is in the eye of the beholder: Exploring the relation between code beauty and quality. *J. Syst. Softw.* **229**, 112494 (2025).
76. Wang, M., Zhang, Y. & Wen, W. Improved capsule networks based on Nash equilibrium for malicious code classification. *Comput. Secur.* **136**, 103503 (2024).
77. Kim, H., Kim, I. & Kim, K. AIBFT: Artificial intelligence browser forensic toolkit. *Forensic Sci. Int. Digit. Investig.* **36**, 301091 (2021).
78. Butt, M. A., Qayyum, A., Ali, H., Al-Fuqaha, A. & Qadir, J. Towards secure private and trustworthy human-centric embedded machine learning: An emotion-aware facial recognition case study. *Comput. Secur.* **125**, 103058 (2023).
79. Chen, P., Du, X., Lu, Z. & Chai, H. Universal adversarial backdoor attacks to fool vertical federated learning. *Comput. Secur.* **137**, 103601 (2024).
80. Sharma, O., Sharma, A. & Kalia, A. MIGAN: GAN for facilitating malware image synthesis with improved malware classification on novel dataset. *Expert Syst. Appl.* **241**, 122678 (2024).
81. Marashdih, A. W., Zaaba, Z. F. & Suwais, K. Predicting input validation vulnerabilities based on minimal SSA features and machine learning. *J. King Saud Univ. Comput. Inf. Sci.* **34**(10), 9311–9331 (2022).
82. Im, D. et al. Prediction of load-dependent power loss based on a machine learning approach in gear pairs with mixed elastohydrodynamic lubrication. *Tribol. Int.* **206**, 110597 (2025).
83. Wang, T. & Strothoff, N. S4Sleep: Elucidating the design space of deep-learning-based sleep stage classification models. *Comput. Biol. Med.* **187**, 109735 (2025).
84. Sun, Y. & Wang, Z. Intrusion detection in IoT and wireless networks using image-based neural network classification. *Appl. Soft Comput.* **177**, 113236 (2025).
85. Azha, S. F. et al. Enhancing river health monitoring: Developing a reliable predictive model and mitigation plan. *Ecol. Ind.* **156**, 111190 (2023).
86. Pritee, Z. T. et al. Machine learning and deep learning for user authentication and authorization in cybersecurity: A state-of-the-art review. *Comput. Secur.* **140**, 103747 (2024).
87. Thapliyal, S. et al. Secure artificial intelligence of things (AIoT)-enabled authenticated key agreement technique for smart living environment. *Comput. Electr. Eng.* **118**, 109353 (2024).
88. Al-Ghamdi, A. S. A. L. M. & Ragab, M. Artificial intelligence techniques based learner authentication in cybersecurity higher education institutions. *Comput. Mater. Contin.* **72**(2), 3131–3144 (2022).
89. Pannyagol, D. B. B. & Deshpande, D. S. L. Ensure authentication and confidentiality in blockchain-based IoT with cryptanalysis and machine learning in 6G-enabled heterogeneous IoT-Blockchain. *Comput. Electr. Eng.* **124**, 110303 (2025).
90. Yao, L. & Jin, M. Research on accounting data encryption processing system based on artificial intelligence. *Proced. Comput. Sci.* **228**, 373–382 (2023).
91. Xu, D., Li, G., Xu, W. & Wei, C. Design of artificial intelligence image encryption algorithm based on hyperchaos. *Ain Shams Eng. J.* **14**(3), 101891 (2023).
92. Xiong, J., Chen, J., Lin, J., Jiao, D. & Liu, H. Enhancing privacy-preserving machine learning with self-learnable activation functions in fully homomorphic encryption. *J. Inf. Secur. Appl.* **86**, 103887 (2024).
93. Ameer, Y. & Bouzeffrane, S. Enhancing privacy in VANETs through homomorphic encryption in machine learning applications. *Proced. Comput. Sci.* **238**, 151–158 (2024).
94. Johnston, R., Sarkani, S., Mazzuchi, T., Holzer, T. & Eveleigh, T. Bayesian-model averaging using MCMC Bayes for web-browser vulnerability discovery. *Reliab. Eng. Syst. Saf.* **183**, 341–359 (2019).
95. Li, X., Xin, Y., Zhu, H., Yang, Y. & Chen, Y. Cross-domain vulnerability detection using graph embedding and domain adaptation. *Comput. Secur.* **125**, 103017 (2023).
96. Tang, X., Du, Y., Lai, A., Zhang, Z. & Shi, L. Deep learning-based solution for smart contract vulnerabilities detection. *Sci. Rep.* **13**(1), 20106 (2023).
97. Ain, Q. U., Javed, A. & Irtaza, A. DeepEvader: An evasion tool for exposing the vulnerability of deepfake detectors using transferable facial distraction blackbox attack. *Eng. Appl. Artif. Intell.* **145**, 110276 (2025).
98. Ferrag, M. A., Alwahedi, F., Battah, A., Cherif, B., Mechri, A., Tihanyi, N., Bisztray, T. & Debbah, M. *Generative AI in Cybersecurity: A Comprehensive Review of LLM Applications and Vulnerabilities* (2025).
99. Tasneem, S., Gupta, K. D., Roy, A. & Dasgupta, D. *Generative Adversarial Networks (GAN) for Cyber Security: Challenges and Opportunities* (2023).
100. Layman, L. & Vetter, R. *Generative Artificial Intelligence and the Future of Software Testing* **57**(01), 27–32 (2024).
101. Sengar, S. S., Hasan, A. B., Kumar, S. & Carroll, F. Generative artificial intelligence: A systematic review and applications. *Multimed. Tools Appl.* (2024).
102. Abumalloh, R. A., Nilashi, M., Ooi, K. B., Tan, G. W. H. & Chan, H. K. Impact of generative artificial intelligence models on the performance of citizen data scientists in retail firms. *Comput. Ind.* **161**, 104128 (2024).
103. Alwahedi, F., Aldaheri, A., Ferrag, M. A., Battah, A. & Tihanyi, N. Machine learning techniques for IoT security: Current research and future vision with generative AI and large language models. *Internet Things Cyber Phys. Syst.* **4**, 167–185 (2024).
104. Jati, A. et al. Adversarial attack and defense strategies for deep speaker recognition systems. *Comput. Speech Lang.* **68**, 101199 (2021).
105. Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A. C. & Bengio, Y. *Generative Adversarial Nets* (2014).
106. Cai, Z. et al. Generative adversarial networks: A survey toward private and secure applications. *ACM Comput. Surv.* **54**(6), Article 132 (2021).
107. Gupta, P., Ding, B., Guan, C. & Ding, D. Generative AI: A systematic review using topic modelling techniques. *Data Inf. Manag.* **8**(2), 100066 (2024).
108. Mhlanga, D. Generative AI for emerging researchers: The promises, ethics, and risks. *SSRN Electr. J.* (2024).
109. Novelli, C., Casolari, F., Hacker, P., Spedicato, G. & Floridi, L. Generative AI in EU law: Liability, privacy, intellectual property, and cybersecurity. *Comput. Law Secur. Rev.* **55**, 106066 (2024).
110. Teo, Z. L., Quek, C. W. N., Wong, J. L. Y. & Ting, D. S. W. Cybersecurity in the generative artificial intelligence era. *Asia Pac. J. Ophthalmol.* **13**(4), 100091 (2024).
111. Gupta, R. & Rathore, B. Exploring the generative AI adoption in service industry: A mixed-method analysis. *J. Retail. Consum. Serv.* **81**, 103997 (2024).
112. Dalalah, D. & Dalalah, O. M. The false positives and false negatives of generative AI detection tools in education and academic research: The case of ChatGPT. *Int. J. Manag. Educ.* **21**(2), 100822 (2023).
113. Aleti, A. *Software Testing of Generative AI Systems: Challenges and Opportunities* 4–14.
114. Kaur, R., Klobučar, T. & Gabrijelić, D. Harnessing the power of language models in cybersecurity: A comprehensive review. *Int. J. Inf. Manag. Data Insights* **5**(1), 100315 (2025).
115. Shafiq, M., Yu, X., Bashir, A. K., Chaudhry, H. N. & Wang, D. A machine learning approach for feature selection traffic classification using security analysis. *J. Supercomput.* **74**(10), 4867–4892 (2018).
116. Abba, S., Bizi, A. M., Lee, J.-A., Bakouri, S. & Crespo, M. L. Real-time object detection, tracking, and monitoring framework for security surveillance systems. *Heliyon* **10**(15), e34922 (2024).

117. Azeem Akbar, M., Mahmood, S., Alsanad, A. & Com, A. Toward successful DevSecOps in software development organizations: A decision-making framework. *Inf. Softw. Technol.* **147**, 1068694 (2022).
118. Kannan, G., Pokharel, S. & Sasi Kumar, P. A hybrid approach using ISM and fuzzy TOPSIS for the selection of reverse logistics provider. *Resour. Conserv. Recycl.* **54**(1), 28–36 (2009).
119. Agarwal, A. & Vrat, P. Modeling attributes of human body organization using ISM and AHP. *Jindal J. Bus. Res.* **6**(1), 44–62 (2017).
120. Venson, E., Clark, B. & Boehm, B. The effects of required security on software development effort: Quantifying secure software practices impact on development cost and quality. *J. Syst. Softw.* **209**, 111874 (2024).
121. Soni, M. *End to End Automation on Cloud with Build Pipeline: The Case for DevOps in Insurance Industry, Continuous Integration, Continuous Testing, and Continuous Delivery* (2015).
122. Attri, R., Grover, S., Dev, N. & Kumar, D. Analysis of barriers of total productive maintenance (TPM). *Int. J. Syst. Assur. Eng. Manag.* **4**(4), 365–377 (2013).
123. Warfield, J. N. Developing interconnection matrices in structural modeling. *IEEE Trans. Syst. Man Cybernet.* **SMC-4**(1), 81–87 (1974).
124. Alwageed, H. S. et al. An empirical study for mitigating sustainable cloud computing challenges using ISM-ANN. *PLoS ONE* **19**(9), 1–34 (2024).
125. S. A. M. M. S. A. g. t. b. s. i. s. development.
126. McGraw, G., Migue, S. & West, J. *Building Security In Maturity Model (BSIMM) Version 6* 1–65, (2015).
127. S. U. Team, *Standard CMMI Appraisal Method for Process Improvement (SCAMPI) A, Version 1.3: Method Definition Document*, HANDBOOKCMU/SEI-2011-HB-001, (2011).
128. Aldin, N. A. N., Abdellatif, W. S. E., Elbarbary, Z. M. S., Omar, A. I. & Mahmoud, M. M. Robust speed controller for PMSG Wind system based on Harris Hawks optimization via wind speed estimation: A real case study. *IEEE Access* **11**, 5929–5943 (2023).
129. Khan, T. A. et al. Secure IoMT for disease prediction empowered with transfer learning in healthcare 5.0, the concept and case study. *IEEE Access* **11**, 39418–39430 (2023).
130. Elghanam, E., Ndiaye, M., Hassan, M. S. & Osman, A. H. Location selection for wireless electric vehicle charging lanes using an integrated TOPSIS and binary goal programming method: A UAE case study. *IEEE Access* **11**, 94521–94535 (2023).
131. Krishnamoorthy, P. et al. Effective scheduling of multi-load automated guided vehicle in spinning mill: A case study. *IEEE Access* **11**, 9389–9402 (2023).
132. Saeed, H., Shafi, I., Ahmad, J., Ahmed Khan, A., Khurshaid, T. & Ashraf, I. Review of techniques for integrating security in the software development lifecycle. *Computers, Materials & Continua* **82** (1), 1–35 (2025).
133. Alzahrani, A. & Khan, R. A. Secure software design evaluation and decision making model for ubiquitous computing: A two-stage ANN-Fuzzy AHP approach. *Comput. Human Behav.* **153**, 108109 (2023).

Acknowledgements

The author extends his appreciation to the National Cybersecurity Authority (NCA) in the Kingdom of Saudi Arabia under the Cybersecurity Research and Innovation Pioneers Initiative to support this research by a grant (No. CRPG-25-3126). Furthermore, the author extends his appreciation to the Deanship of Scientific Research at King Saud University for funding this work through Ongoing Research Funding Program, (ORF-2025-1439), Riyadh, Saudi Arabia.

Author contributions

The research idea was conceived by Hussein A. Al-Hashimi who developed the design of the Hybrid ANN-ISM Framework and conducted the multivocal literature review. The survey was developed by Hussein A. Al-Hashimi, who coordinated the data collection and conducted the statistical analysis. He implemented the ANN model and the ISM methodological approaches, analysed the case study results, and drafted and revised the manuscript. The author alone was responsible for all aspects of the research including the conceptualization to completion of the writing.

Funding

This research is supported by a grant (No. CRPG-25-3126) under the Cybersecurity Research and Innovation Pioneers Initiative, Provided by the National Cybersecurity Authority (NCA) in the Kingdom of Saudi Arabia. The author also extends his appreciation to the Deanship of Scientific Research at King Saud University for funding this work through Ongoing Research Funding Program, (ORF-2025-1439), Riyadh, Saudi Arabia.

Declarations

Competing interests

The authors declare no competing interests.

Ethics approval

This study was conducted in accordance with ethical research guidelines and was reviewed and approved by the Research Ethics Committee at King Saud University and University of Southampton.

Informed consent

All participants involved in the survey and expert panel provided their informed consent prior to participation. They were informed about the purpose of the study, assured of the confidentiality of their responses, and notified that their participation was voluntary and anonymous. No personal or identifiable information was collected.

Additional information

Correspondence and requests for materials should be addressed to H.A.A.-H.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2026